MindVideo AI

إتقان المحوّلات والنماذج اللغوية الكبيرة

Transformers & Large Language Models

من الجبر الخطي والانتشار العكسي إلى بناء نموذج لغوي كبير
وتشغيله في الإنتاج - بالحدس والرياضيات والكود القابل للتشغيل

دليل عربي متكامل من الصفر إلى الإنتاج



المؤلف

م/أحمد صالح المغز

2026

إتقان المحوّلات والنماذج اللغوية الكبيرة

دليل عربي شامل من الصفر إلى الإنتاج: 12 جزءًا، 64 فصلًا، 12 مشروعًا تطبيقيًا، و8 ملاحق مرجعية.

الطبعة الأولى · ٢٠٢٦

فريق الكتاب

ahmedalmaghz.github.io/MLLBook

الفهرس

الأجزاء والفصول والمشاريع والملاحق · الأرقام تشير إلى صفحات هذه الطبعة

مقدمة الكتاب 7

كيف تقرأ هذا الكتاب 12

الرموز والاصطلاحات 19

خريطة الكتاب 28

دفاتر الكود القابلة للتشغيل 37

الجزء الأول — الأساسيات **45**

إعداد البيئة والبنية التحتية للتطوير 46

الأسس الرياضية للتعلم العميق 55

الشبكات العصبية من الصفر: MLP والانتشار العكسي 66

التحسين والتدريب: من SGD إلى AdamW 76

قياس الأداء: مقاييس التصنيف والحيرة 89

مصنّف نصوص عربي من الصفر [مشروع] 99

الجزء الثاني — التمثيل اللغوي **106**

التقطيع إلى وحدات: Tokenization 107

التمثيلات المتجهية للكلمات: Word2Vec وGloVe 119

الشبكات المتكررة: RNN وLSTM وGRU 130

تحليل مشاعر عربي بـ LSTM [مشروع] 144

الجزء الثالث — المحوّلات الأساسية **151**

من التكرار إلى الانتباه: آلية Attention التاريخية 152

الانتباه الذاتي متعدد الرؤوس 165

الترميز الموضعي والتطبيع وبنية المحوّل الكاملة 179

عائلات المحوّلات: BERT وGPT وT5 196

بناء MiniGPT عربي كامل من الصفر [مشروع] 214

الجزء الرابع — المعماريات الحديثة **223**

حدود المحوّل الكلاسيكي 224

الترميز الموضعي المتقدم وآليات الانتباه الفعّالة 235

خليط الخبراء: Mixture of Experts 254

نماذج الفضاء الحالة: SSM وMamba 268

المكوّنات الحديثة: RMSNorm وSwiGLU 282
بناء معمارية LLaMA-Style كاملة [مشروع] 295

الجزء الخامس — بناء LLM من الصفر 302

مصادر البيانات وتنظيفها 303
تدريب Tokenizer مخصص 327
قوانين التوسّع: Chinchilla 342
التوازي الموزّع للتدريب 357
حفظ نقاط التفتيش ومراقبة التدريب 379
خط إنتاج ما بعد التدريب 397
خط إنتاج تدريب LLM متكاملا [مشروع] 415

الجزء السادس — إعادة استخدام النماذج 425

التدريب المسبق المستمر: CPT 426
تقطير المعرفة: Knowledge Distillation 439
دمج النماذج: Model Merging 451
التقليم وضغط النماذج: Pruning 467
توسيع العمق: Depth Upscaling 479
إعادة استخدام النماذج: إطار القرار الشامل 488
خط أنابيب إعادة استخدام متكاملا [مشروع] 496

الجزء السابع — التخصيص الفعّال والمحاذاة 503

مقدمة إلى PEFT: التدريب الفعّال من حيث البارامترات 504
LoRA: التكيّف منخفض الرتبة بعمق 516
QLoRA والتكميم أثناء التدريب 527
RLHF وPPO بعمق 538
DPO وKTO وORPO: ما بعد RLHF 550
ضبط QLoRA ومحاذاة DPO [مشروع] 564

الجزء الثامن — النماذج متعددة الوسائط 572

أساسيات النماذج متعددة الوسائط 573
مُشفِّرات الرؤية: ViT وCLIP 583
نماذج الرؤية واللغة: VLMs 593
نماذج توليد الصور: Diffusion 607
الصوت والكلام متعدد الوسائط 618
النماذج الموحّدة: Any-to-Any 628

مساعد رؤية-لغة مصغّر [مشروع] 638

الجزء التاسع — التقييم الشامل **644**

أسس القياس الكمّي 645
معايير القياس المرجعية: Benchmarks 655
LLM-as-a-Judge والتقييم البشري 665
اختبار السلامة والأمان: Red Teaming 674
التقييم النوعي وتحليل الأخطاء 683
خط التقييم المستمر 692
منصّة تقييم شاملة لنموذج عربي [مشروع] 702

الجزء العاشر — التطبيقات والوكلاء الذكية **707**

أسس التوليد المعزّز بالاسترجاع: RAG 708
تقنيات RAG المتقدمة وGraphRAG 719
أسس الوكلاء الذكية: ReAct والأدوات 732
معماريات الوكلاء والأنظمة متعددة الوكلاء 744
الذاكرة طويلة المدى للوكلاء 755
هندسة الأوامر المتقدمة 766
تطبيقات متخصصة حسب المجال 778
وكيل بحث عربي متكاملا [مشروع] 788

الجزء الحادي عشر — النشر والتشغيل **792**

أساسيات تقديم النماذج للاستدلال 793
تحسين الاستدلال المتقدم 804
البنية التحتية والتوسّع 815
المراقبة والتشغيل: Observability 827
الأمان والحوكمة في الإنتاج 839
إدارة التكلفة ونضج LLMOps 850
مشروع الجزء الحادي عشر: خدمة LLM إنتاجية متكاملة [مشروع] 862

الجزء الثاني عشر — الاتجاهات المستقبلية **867**

نماذج الاستدلال وTest-Time Compute 868
المحاذاة المتقدمة وسلامة الأنظمة الفائقة 882
المعماريات التالية لما بعد Transformer 897
الحوسبة الفعالة والذكاء على الحافة 909
خارطة الطريق المهنية والتعلم المستمر 922

مشروع الجزء الثاني عشر: لوحة رصد اتجاهات البحث مشروع 929

الملاحق 934

مسرد المصطلحات 934
مرجع المعادلات 946
المكتبات والأدوات 962
بيئة GPU والسحابة 971
حلول مختارة للتمارين 979
مصادر ومسارات تعلّم 994
بنك أسئلة المقابلات 1004
الفهرس التفاعلي 1018

تمهيد

إتقان المحوّلات والنماذج اللغوية الكبيرة

دليل شامل من الصفر إلى الإنتاج — من الجبر الخطي والانتشار العكسي إلى بناء LLM وتشغيله في الإنتاج.

لماذا هذا الكتاب؟

يجمع هذا الكتاب بين ثلاث طبقات لا تجتمع عادة في مصدر واحد: **الحدس** الذي يبني الصورة الذهنية، **الرياضيات** التي تفسّر لماذا تعمل الأشياء، و**الكود** القابل للتشغيل الذي يحوّل الفهم إلى نظام حقيقي. كل فصل مكتوب ليكون مستقلًا وقابلًا للتنفيذ، مع تمارين وأسئلة مقابلات ومراجع للتعمّق.

أجزاء الكتاب

الجزء	العنوان	الفصول
1	الأساسيات	1–5
2	التمثيل اللغوي	6–8
3	المحوّلات الأساسية	9–12
4	المعماريات الحديثة	13–17
5	بناء LLM من الصفر	18–23
6	إعادة استخدام النماذج	24–29
7	التخصيص الفعّال والمحاذاة	30–34
8	النماذج متعددة الوسائط	35–40
9	التقييم الشامل	41–46

الجزء	العنوان	الفصول
10	التطبيقات والوكلاء الذكية	47–53
11	النشر والتشغيل	54–59
12	الاتجاهات المستقبلية	60–64

ابدأ من هنا

- مقدمة الكتاب
- كيف تقرأ هذا الكتاب
- الفهرس الكامل
- خريطة الكتاب
- دفاتر الكود القابلة للتشغيل

المسار السريع

1. **الأساسيات (الأجزاء 1–3):** الرياضيات، MLP، المحوّلات.
2. **البناء (الأجزاء 4–5):** المعماريات الحديثة، خط إنتاج LLM كامل.
3. **التخصيص (الأجزاء 6–7):** CPT، التقطير، الدمج، LoRA، المحاذاة.
4. **التوسّع (الأجزاء 8–9):** الوسائط المتعددة، التقييم.
5. **الإنتاج (الأجزاء 10–12):** RAG، الوكلاء، LLMOps، الاتجاهات.

المقدمة

مقدمة الكتاب

> **من الجبر الخطي إلى الإنتاج: رحلة واحدة متصلة، لا مجموعة من مقالات متفرقة.**

هذا الكتاب محاولة لبناء مرجع عربي واحد يشرح المحوّلات (Transformers) والنماذج اللغوية الكبيرة (LLMs) من أول مبدأ إلى نظام يعمل في الإنتاج. لا يفترض الكتاب أنك تعرف بالفعل ما هو الانتباه، ولا يقفز مباشرة إلى واجهات المكتبات الجاهزة. يبدأ من السؤال الأول: كيف يتحوّل النص إلى أرقام؟ ثم يبني الطبقة تلو الأخرى: الشبكة العصبية، الانتشار العكسي، المحسّنات، التقطيع، التضمينات، الانتباه، المحوّل، المعماريات الحديثة، التدريب الموزّع، إعادة استخدام النماذج، الضبط الفعّال، المحاذاة، الوسائط المتعددة، التقييم، الوكلاء، والنشر.

الهدف ليس أن تحفظ وصفًا معماريًا، بل أن تكون قادرًا على **رسم** المعمارية، و**اشتقاق** معادلاتها الأساسية، و**كتابة** نسخة مصغّرة منها، و**تشخيص** سلوكها عندما تفشل. لهذا يجمع كل فصل بين ثلاث طبقات:

1. **الحدس:** صورة ذهنية وقصة تشرح لماذا وُجد هذا المفهوم.
2. **الرياضيات:** المعادلات التي تحدد ما يحدث فعلًا، مع ربط كل رمز بمعناه.
3. **الكود:** تنفيذ قابل للتشغيل، أو هيكل واضح يمكن تحويله إلى تنفيذ.

لمن هذا الكتاب؟

- **للمبتدئ الجاد** الذي يعرف Python ويريد فهم التعلم العميق من الداخل، لا الاكتفاء باستدعاء `model.fit()`.
- **لمهندس البرمجيات** الذي ينتقل إلى مجال الذكاء الاصطناعي ويريد خريطة طريق واضحة تربط النظرية بالأنظمة.

- **للباحث أو طالب الدراسات العليا** الذي يحتاج إلى مرجع عربي منظم للمفاهيم والأوراق الأساسية.
- **للمهندس الممارس** الذي يبني تطبيقات RAG أو وكلاء أو خدمات استدلال، ويحتاج إلى فهم الحدود والكلفة والمخاطر.
- **للمستعد للمقابلات** الذي يريد أسئلة حقيقية وأجوبة مختصرة مدعومة بالمعادلات والتفاصيل الهندسية.

الكتاب لا يفترض خلفية متقدمة في الرياضيات، لكنه لا يتجنبها. إذا كنت لا تزال غير مرتاح للجبر الخطي أو الاحتمالات، فالفصل الثاني مصمم لك تحديدًا، ويمكنك العودة إليه كلما احتجت.

ما الذي يميز هذا الكتاب؟

1. البنية الموحّدة

كل فصل من فصول الكتاب الـ64 يتبع القالب نفسه:

1. لماذا يهمّك هذا الموضوع؟
2. الخريطة الذهنية
3. الشرح المتدرج
4. مثال محلول خطوة بخطوة
5. نصائح وحيل
6. أخطاء شائعة ومصائد
7. جدول مقارنة
8. ملخص أفضل الممارسات
9. أسئلة مقابلات
10. تمرين عملي

1. مراجع للتعمّق

هذه البنية ليست قيدًا شكليًا؛ إنها اتفاق مع القارئ. تعرف دائمًا أين تجد الفكرة الكبيرة، وأين تجد التفاصيل، وأين تختبر فهمك، وأين تتابع المصادر الأصلية.

2. العربية ليست ترجمة حرفية

الكتاب مكتوب بالعربية، لكنه لا يتعامل مع العربية كمجرد لغة واجهة. هناك فصول كاملة عن التقطيع العربي، وFertility، والهمزات والتشكيل، وبناء Tokenizer مخصص، وتقييم النماذج العربية، وRAG على وثائق عربية. المفاهيم العالمية تُشرح بالعربية، ثم تُختبر على بيانات عربية حيثما أمكن.

3. من الصفر إلى الإنتاج

الكتاب لا يتوقف عند النموذج. يغطي ما بعد التدريب: التقييم، الأمان، Red Teaming، التقديم، التكميم، التوسّع، المراقبة، التكلفة، والحوكمة. النموذج الجيد الذي لا يمكن تشغيله أو قياسه أو تأمينه ليس نظامًا جيدًا.

4. مشاريع تربط الأجزاء

كل جزء ينتهي بمشروع تطبيقي يدمج مفاهيمه. المشاريع ليست تمارين صغيرة؛ إنها نقاط تجميع (Integration Points) تكشف ما فهمته فعلًا وما لم تفهمه. المشروع النهائي في الفصل 64 يجمع ثلاثة أجزاء على الأقل في منتج واحد.

ما لن تجده هنا

- **لا وعود بأنك ستبني GPT-5 في نهاية الأسبوع.** بناء النماذج الكبيرة يحتاج بيانات وحوسبة وهندسة تتجاوز نطاق كتاب واحد.
- **لا كود جاهز للنسخ دون فهم.** بعض الأمثلة تعليمية ومصغّرة عمدًا؛ الهدف هو نقل المبدأ، لا تقديم مكتبة إنتاجية.
- **لا تغطية شاملة لكل ورقة بحثية.** المجال يتغير أسرع من أي كتاب. لذلك يُعلّمك الكتاب كيفية القراءة والتقييم أكثر من محاولة ملاحقة كل إصدار.

- **لا بديل عن التجربة.** لن يصبح أي مفهوم راسخًا دون تشغيل الكود، وقياس النتائج، وتحليل الأخطاء.

كيف بُني هذا الكتاب؟

الكتاب مُنظّم في 12 جزءًا و64 فصلًا و12 مشروعًا و8 ملاحق. الفصول مكتوبة لتُقرأ بالترتيب، لكنها مصممة أيضًا لتكون مستقلة نسبيًا. كل فصل يذكر متطلباته السابقة، ويحتوي على ملخص وأسئلة وتمرين ومراجع. إذا كنت تعرف موضوعًا بالفعل، يمكنك استخدام الخريطة الذهنية وجدول المقارنة وأسئلة المقابلات كمراجعة سريعة، ثم الانتقال إلى التمرين.

النسخة التي تقرأها مولّدة من مصدر واحد منظم، مع طبقة تحقق آلية تفحص اكتمال الأقسام، وتوازن كتل الكود، وسلامة الروابط الداخلية. هذا لا يضمن صحة كل جملة، لكنه يضمن أن البنية متسقة وأن الفجوات الظاهرة لا تمر دون رصد.

ماذا تحتاج قبل البدء؟

- **Python 3.10+** (يُفضّل 3.11 أو 3.12).
- **معرفة أساسية بالبرمجة:** الدوال، الكلاسات، القوائم، القواميس، البيئات الافتراضية.
- **أساسيات الجبر الخطي والاحتمالات:** يمكن بناؤها من الفصل الثاني.
- **جهاز يمكنه تشغيل Python.** وجود GPU مفيد لكنه ليس شرطًا للفصول الأولى. يمكن استخدام Google Colab أو Kaggle أو خدمة سحابية عند الحاجة.
- **صبر وممارسة:** الكتاب كثيف في بعض الأجزاء، وهذا مقصود. خذ وقتك، ونفّذ التمارين، ولا تنتقل قبل أن تستطيع شرح الفكرة لشخص آخر.

خارطة الطريق المقترحة

الهدف	المسار المختصر
فهم الأساسيات	الأجزاء 1–3، ثم مشاريعها

الهدف	المسار المختصر
بناء نموذج من الصفر	الأجزاء 1–5، مع مشروع MiniGPT
تخصيص نموذج جاهز	الأجزاء 6–7، ثم مشروع QLoRA + DPO
بناء تطبيق RAG/وكيل	الأجزاء 9–10، مع مشروع وكيل البحث
نشر خدمة إنتاجية	الأجزاء 9–11، مع مشروع الخدمة الإنتاجية
التحضير للمقابلات	الفصول 1–5، 9–17، 20–23، 30–34، 41–46، 54–59 + بنك الأسئلة
البحث والتطوير	الأجزاء 3–6 و12 + قراءة المراجع الأصلية

اصطلاحات سريعة

- **الفصل (Chapter):** وحدة تعليمية واحدة.
- **الجزء (Part):** مجموعة فصول مترابطة تنتهي بمشروع.
- **المشروع (Project):** تطبيق يدمج مفاهيم الجزء.
- **الملحق (Appendix):** مرجع مساعد (مسرد، معادلات، أدوات، حلول، مصادر).
- **المستوى:** مبتدئ، متوسط، متقدم، خبير. المستوى تقديري وليس شرطًا مسبقًا صارمًا.

شكر وتنبيه

هذا الكتاب عمل تعليمي مستقل. أسماء المكتبات والنماذج والشركات المذكورة ملك لأصحابها، وتُذكر لأغراض الشرح والمقارنة. الأرقام والأسعار والأداء المذكورة تقديرية وتتغير بسرعة؛ تحقق دائمًا من المصادر الرسمية قبل اتخاذ قرار هندسي أو مالي.

إذا وجدت خطأً أو فجوة، فالأفضل أن تفتح ملاحظة موثقة: رقم الفصل، القسم، النص المتوقع، والمصدر. الكتاب الجيد ليس الذي لا يخطئ، بل الذي يمكن تصحيحه.

ابدأ من: كيف تقرأ هذا الكتاب · خريطة الكتاب · الفهرس

المقدمة

كيف تقرأ هذا الكتاب

> **لا تقرأ الكتاب كرواية. اقرأه كورشة عمل: اقرأ، جرّب، قِس، ثم اشرح.**

1. اختر مسارك قبل أن تبدأ

ليس على كل قارئ أن يقرأ 64 فصلًا بالترتيب. اختر مسارًا يناسب هدفك الحالي، ثم عد إلى المسار الكامل لاحقًا.

المسار الكامل (من الصفر إلى الإنتاج)

الأجزاء 1 ← 12 بالترتيب.

مناسب لمن يريد بناء أساس شامل، أو لمن يخطط لمشروع كبير ويحتاج إلى صورة كاملة. المدة التقديرية: 6–12 شهرًا بمعدل 5–8 ساعات أسبوعيًا، مع تنفيذ التمارين والمشاريع.

مسار المبتدئ الجاد

الأجزاء 1–3، ثم مشاريع 1–3، ثم الفصل 18 و19 و23.

يغطي الرياضيات، الشبكات، المحسّنات، التقييم، التقطيع، التضمينات، RNN، الانتباه، المحوّل، وعائلاته. بعدها يمكنك التوسع حسب الاهتمام.

مسار مهندس النماذج

الفصول 1–5، 9–17، 20–23، 30–34، ثم 54–59.

يركز على ما تحتاجه لفهم المعمارية وتدريبها وتخصيصها ونشرها. يمكن تخطي فصول الوسائط المتعددة والتقييم النوعي في القراءة الأولى، مع العودة إليها عند الحاجة.

مسار بناء التطبيقات (RAG والوكلاء)

الفصول 7–6، 12، 43–41، 53–47، ثم 59–57.

مناسب لمن يبني منتجًا فوق نموذج جاهز. لا تحتاج إلى فهم كل تفاصيل الانتشار العكسي، لكنك تحتاج إلى فهم التقطيع والتضمينات والتقييم والاسترجاع والأمان والتكلفة.

مسار البحث

الفصول 5–2، 17–9، 25–20، 34–33، 46–41، 63–60 + المراجع الأصلية.

مع قراءة ورقة واحدة على الأقل لكل فصل متقدم، ومحاولة إعادة تنفيذ نتيجة بسيطة. هذا المسار يفترض استعدادًا لقضاء وقت أطول مع الرياضيات والأوراق البحثية.

مسار المقابلات

الفصول 5–1، 17–9، 23–20، 34–30، 46–41، 59–54 + الملحق ز (بنك الأسئلة).

ركّز على: لماذا صُمم كل مكوّن؟ ما تكلفته؟ متى يفشل؟ ما البديل؟ أسئلة المقابلات الجيدة لا تكتفي بتعريف المصطلح، بل تختبر قرارات التصميم والمقايضات.

2. منهجية القراءة الثلاثية

كل فصل مصمم ليُقرأ ثلاث مرات، كل مرة بعمق مختلف.

القراءة الأولى: الصورة الكبيرة (25–15 دقيقة)

1. اقرأ عنوان الفصل والملخص ومخرجات التعلم.
2. اقرأ القسم 1 (لماذا يهمّك؟) والقسم 2 (الخريطة الذهنية).
3. اقرأ العناوين الفرعية في القسم 3 دون التوقف عند كل تفصيل.
4. اقرأ القسم 8 (ملخص أفضل الممارسات) والقسم 9 (أسئلة المقابلات).
5. اكتب في دفترك: «ما السؤال الذي يجيب عنه هذا الفصل؟» و«ما الفكرة التي سأتذكرها بعد شهر؟»

الهدف: بناء هيكل ذهني قبل الحشو.

القراءة الثانية: الفهم والاشتقاق (120–60 دقيقة)

1. اقرأ القسم 3 ببطء، ومع كل معادلة اكتب معنى كل رمز.
2. نفّذ المثال المحلول في القسم 4 بيدك، لا بعينك فقط.
3. توقف عند كل كتلة كود وتوقع المخرجات قبل تشغيلها.
4. اقرأ القسم 5 و6، واربط كل خطأ شائع بتجربة أو ملاحظة.
5. اقرأ القسم 7 (جدول المقارنة) واسأل: ما المعيار؟ ما التكلفة؟ ما القيد؟

الهدف: القدرة على إعادة إنتاج الفكرة دون النظر إلى النص.

القراءة الثالثة: التثبيت والتقييم (60–30 دقيقة)

1. أغلق الفصل وحاول رسم الخريطة الذهنية من الذاكرة.
2. أجب عن أسئلة المقابلات بصوت مسموع أو كتابة.
3. نفّذ التمرين العملي، أو على الأقل اكتب خطة تنفيذه.
4. اقرأ المراجع الأصلية، ولو ملخصًا واحدًا.
5. اكتب ثلاث نقاط: ما فهمته، ما بقي غامضًا، ما سأطبقه.

الهدف: تحويل المعرفة إلى قدرة قابلة للاستدعاء.

3. كيف تتعامل مع الرياضيات؟

- **لا تحفظ المعادلة قبل أن تفهم شكلها.** اسأل: ما المدخلات؟ ما المخرجات؟ ما الأبعاد؟ ماذا يحدث عند القيم الحدية $(0,\ 1,\ \infty)$؟
- **اربط كل رمز بمتغير في الكود.** إذا كانت المعادلة تستخدم W_Q، فابحث عن `W_q` في المثال.

- **اشتق يدويًا مرة واحدة على الأقل** لأهم المعادلات: Softmax، Cross-Entropy، DPO، LoRA، LayerNorm، Attention.
- **استخدم الأبعاد كأداة تحقق.** كثير من الأخطاء تُكتشف بمجرد كتابة `(B, T, D)` على الورق.
- **إذا علقت، بسّط:** جرّب متجهًا ثنائي الأبعاد أو تسلسلًا من كلمتين. الحالة الصغيرة تكشف الفكرة.

4. كيف تتعامل مع الكود؟

1. **شغّل الكود في بيئة نظيفة.** أنشئ بيئة افتراضية لكل جزء أو لكل مشروع.
2. **اقرأ الكود قبل تشغيله.** توقع الشكل، القيم، والمخرجات.
3. **غيّر معاملًا واحدًا في كل مرة.** غيّر `rank` أو `learning_rate` أو `chunk_size`، ولاحظ الأثر.
4. **لا تكتفِ بالتشغيل.** أضف طباعة للأبعاد، وارسم منحنى، وقِس الوقت والذاكرة.
5. **اكتب اختبارًا صغيرًا.** حتى `assert output.shape == (B, T, D)` يحمي من أخطاء صامتة.
6. **إذا لم يعمل الكود، لا تنسخ حلًا جاهزًا فورًا.** اقرأ رسالة الخطأ، وعزل المشكلة، وقلّص المثال.

> **تنبيه:** بعض الأمثلة تعليمية ومصغّرة، وقد لا تكون محسّنة للإنتاج. لا تنشر كودًا تعليميًا في الإنتاج دون مراجعة أمنية واختبارات وأداء.

5. كيف تستخدم المشاريع؟

كل جزء ينتهي بمشروع. المشروع ليس اختبارًا نهائيًا، بل **نقطة تجميع**. اتبع هذه الدورة:

1. **اقرأ متطلبات المشروع كاملة** قبل كتابة أي كود.

2. **اكتب قرارات التصميم** في صفحة واحدة: البيانات، النموذج، المقياس، حدود الموارد، المخاطر.
3. **ابنِ نسخة رأسية صغيرة (Vertical Slice):** مسار كامل من البيانات إلى المخرج، ولو كان بسيطًا.
4. **أضف القياس مبكرًا:** ما معيار النجاح؟ كيف تعرف أنك تحسنت؟
5. **وثّق التجارب:** ما جرّبته، ما نجح، ما فشل، ولماذا.
6. **راجع النتيجة:** هل تحقق الهدف؟ ما القيد الحقيقي؟ ما الخطوة التالية؟

المشروع المكتمل ليس الذي يعمل فقط، بل الذي تستطيع شرح قراراته وحدوده.

6. جدول تنظيم الوقت

النشاط	النسبة المقترحة
قراءة الفصل	30%
تشغيل الكود والتجريب	35%
التمارين والمشاريع	25%
المراجعة وأسئلة المقابلات	10%

إذا كان وقتك محدودًا، لا تحذف التجريب بالكامل. يمكنك تقليل عدد التمارين، لكن احتفظ بتمرين واحد كامل لكل جزء.

7. كيف تراجع؟

- **مراجعة بعد 24 ساعة:** أعد رسم الخريطة الذهنية وأجب عن 3 أسئلة.
- **مراجعة بعد أسبوع:** نفّذ تمرينًا صغيرًا أو اشرح الفصل لشخص.
- **مراجعة بعد شهر:** ارجع إلى جدول المقارنة وأسئلة المقابلات، وحدّث ملاحظاتك.

- **بطاقات مفاهيم:** لكل مفهوم، بطاقة بها: التعريف، المعادلة، متى يُستخدم، متى يفشل.
- **دفتر أخطاء:** سجّل كل خطأ واجهته (شكل، NaN، OOM، تسريب بيانات) وكيف حللته. هذا الدفتر يصبح مرجعك الشخصي.

8. نصائح خاصة بالعربية

- **لا تخلط بين التطبيع والتشكيل.** حدد مبكرًا هل ستحذف التشكيل، وهل ستوحّد الهمزات، وهل ستتعامل مع التطويل. كل قرار يؤثر في التقييم.
- **اختبر Tokenizer على نصك الحقيقي.** متوسط عدد الرموز لكل كلمة (Fertility) قد يختلف كثيرًا بين نموذج وآخر.
- **انتبه إلى اللهجات.** نموذج مدرّب على الفصحى قد يفشل في اللهجة الخليجية أو المصرية أو المغاربية.
- **لا تعتمد على ترجمة آلية للتقييم.** راجع عينات عربية يدويًا، خصوصًا في السلامة والمصطلحات القانونية والطبية.
- **افصل اتجاه النص عن اتجاه الكود.** الواجهة عربية RTL، لكن الكود والمعادلات والأوامر تبقى LTR.

9. ماذا تفعل إذا شعرت بالضياع؟

1. **عد إلى الخريطة الذهنية** في بداية الفصل.
2. **اقرأ الملخص وأسئلة المقابلات** لتعرف الحد الأدنى المطلوب.
3. **قلّص السؤال:** بدل «كيف يعمل المحوّل؟» اسأل «كيف تحسب كلمة واحدة انتباهها لكلمة أخرى؟».
4. **ارجع إلى الفصل السابق** إذا كانت المتطلبات غير واضحة.
5. **اسأل سؤالًا محددًا:** «لماذا نضرب في $\sqrt{d_k}$؟» أفضل من «لم أفهم الانتباه».

6. **خذ استراحة.** بعض المفاهيم تحتاج نومًا وممارسة، لا مزيدًا من القراءة في اللحظة نفسها.

10. قائمة تحقق قبل الانتقال إلى الجزء التالي

- [ ] أستطيع شرح الفكرة الأساسية دون النظر إلى النص.
- [ ] نفّذت المثال المحلول أو نسخة منه.
- [ ] أجبت عن معظم أسئلة المقابلات.
- [ ] نفّذت التمرين أو كتبت خطة تنفيذه.
- [ ] أعرف متى يفشل هذا الأسلوب وما بدائله.
- [ ] قرأت مرجعًا أصليًا واحدًا على الأقل.
- [ ] سجّلت ما بقي غامضًا وخطة العودة إليه.

المقدمة

الرموز والاصطلاحات

> **اتفاق واحد على الرموز يوفّر ساعات من الالتباس.** ارجع إلى هذه الصفحة كلما ظهر رمز غير مألوف.

1. اصطلاحات النص

الاصطلاح	المعنى
الفصل	وحدة تعليمية واحدة ضمن جزء.
الجزء	مجموعة فصول مترابطة تنتهي بمشروع.
القسم	قسم داخل الفصل، مرقّم من 1 إلى 11.
المشروع	تطبيق ختامي يدمج مفاهيم الجزء.
الملحق	مرجع مساعد في نهاية الكتاب.
المستوى	مبتدئ / متوسط / متقدم / خبير (تقديري).
زمن القراءة	تقدير آلي بناءً على عدد الكلمات، وليس قياسًا دقيقًا.
التمرين	نشاط عملي في نهاية الفصل.
أسئلة المقابلات	أسئلة مراجعة وتقييم فهم، وليست بنكًا حصريًا.

2. مستويات الصعوبة

المستوى	ما يفترضه	ما يطلبه
مبتدئ	معرفة Python وأساسيات البرمجة	فهم الفكرة وتشغيل مثال
متوسط	أساسيات الجبر والاحتمالات	اشتقاق وتنفيذ وقياس
متقدم	معرفة شبكات عصبية ومحوّلات	قرارات تصميم ومقايضات

المستوى	ما يفترضه	ما يطلبه
خبير	خبرة تدريب/نشر أو بحث	تحليل حدود، أداء، ومخاطر

المستوى لا يعني «يمكن تخطيه». قد يكون فصل متقدم ضروريًا لفهم جزء لاحق حتى لو لم تكن خبيرًا بعد.

3. الرموز الرياضية الأساسية

3.1 المتجهات والمصفوفات

الرمز	المعنى	مثال
x	عدد قياسي (scalar)	$x = 3.14$
$\mathbf{x}$	متجه	$\mathbf{x} \in \mathbb{R}^d$
X	مصفوفة	$X \in \mathbb{R}^{n \times d}$
$X^\top$	منقول المصفوفة	$X^\top \in \mathbb{R}^{d \times n}$
X^{-1}	معكوس المصفوفة (إن وُجد)	$X X^{-1} = I$
$\odot$	ضرب عنصري (Hadamard)	$(a \odot b)_i = a_i b_i$
$\cdot$	ضرب نقطي أو قياسي	$\mathbf{a} \cdot \mathbf{b}$
$\|\mathbf{x}\|_2$	القاعدة الإقليدية	$\sqrt{\sum_i x_i^2}$
$\|X\|_F$	قاعدة فروبينيوس	$\sqrt{\sum_{ij} X_{ij}^2}$
$\mathrm{softmax}$	تحويل إلى توزيع احتمالي	$\mathrm{softmax}(\mathbf{z})_i = e^{z_i} / \sum_j e^{z_j}$
σ	دالة سيغمويد	$1/(1+e^{-x})$
$\tanh$	ظل زائدي	$(e^x - e^{-x})/(e^x + e^{-x})$
ReLU	دالة تفعيل	$\max(0, x)$

3.2 الأبعاد الشائعة

الرمز	المعنى
B	حجم الدفعة (batch size)
T	طول التسلسل (sequence length)
D أو d_{model}	بُعد التمثيل
H	عدد رؤوس الانتباه
$d_k = D/H$	بُعد كل رأس
V	حجم المفردات
L	عدد الطبقات
N	عدد المعاملات
C	ميزانية الحوسبة (FLOPs)
n_{ctx}	طول السياق

3.3 الاحتمالات والإحصاء

الرمز	المعنى
$P(x)$	احتمال الحدث أو التوزيع
$P(x \mid y)$	احتمال شرطي
$\mathbb{E}[x]$	التوقع
$\mathrm{Var}(x)$	التباين
$\mathrm{Cov}(x, y)$	التغاير
$D_{\mathrm{KL}}(P \| Q)$	تباعد كولباك-ليبلر
$\mathcal{N}(\mu, \sigma^2)$	توزيع طبيعي
$\mathcal{L}$	دالة الخسارة
η	معدل التعلم
$\nabla_\theta \mathcal{L}$	تدرّج الخسارة بالنسبة للمعاملات

3.4 التعلم والتحسين

الرمز	المعنى
θ	معاملات النموذج
θ_t	المعاملات عند الخطوة t
g_t	التدرّج عند الخطوة t
m_t	العزم الأول (متوسط التدرّج)
v_t	العزم الثاني (متوسط مربع التدرّج)
β_1, β_2	معاملات اضمحلال العزوم
ϵ	ثابت صغير لتفادي القسمة على صفر
λ	معامل تنظيم أو وزن عقوبة
τ	درجة الحرارة (temperature)
α	معامل LoRA أو وزن خسارة مساعدة
r	رتبة LoRA

3.5 المحوّلات

الرمز	المعنى
Q, K, V	الاستعلام، المفتاح، القيمة
W_Q, W_K, W_V	مصفوفات الإسقاط
W_O	مصفوفة الإخراج
A	مصفوفة أوزان الانتباه
d_k	بُعد المفتاح/الاستعلام
head_i	الرأس رقم i
FFN	شبكة التغذية الأمامية
LN	تطبيع الطبقة
RMSNorm	تطبيع الجذر التربيعي المتوسط

الرمز	المعنى
PE	الترميز الموضعي
RoPE	الترميز الموضعي الدوّار
m_t	قناع السببية عند الموضع t

4. اصطلاحات الأكواد

4.1 الأسماء

النمط	المعنى
`snake_case`	دوال ومتغيرات
`PascalCase`	كلاسات
`UPPER_CASE`	ثوابت
`_private`	عنصر داخلي
`model` ، `tokenizer` ، `optimizer`	كائنات مكتبة معروفة
`B, T, D`	أبعاد شائعة في التعليقات

4.2 أشكال الموترات (Tensor Shapes)

الشكل	المعنى
`(B, T)`	رموز أو تصنيفات
`(B, T, D)`	تمثيلات تسلسلية
`(B, H, T, d_k)`	تمثيلات متعددة الرؤوس
`(B, T, V)`	لوغاريتمات الاحتمالات
`(B, T, 1)`	قناع أو وزن لكل موضع
`(V, D)`	مصفوفة تضمين
`(D, D)`	مصفوفة إسقاط

4.3 اصطلاحات الكود

- `#>>` أو تعليق واضح: مخرج متوقع أو ملاحظة تعليمية.
- `...`: جزء محذوف للاختصار، وليس كودًا صالحًا.
- `TODO`: نقطة تحتاج إكمالًا، ولا تُترك في نسخة نهائية.
- `assert`: تحقق من شكل أو قيمة، ويُفضّل تركه في الكود التعليمي.
- `torch.no_grad()`: عند الاستدلال فقط.
- `model.train()` / `model.eval()`: لا تنسَ تبديل الوضع عند التقييم.

5. اصطلاحات المعادلات

- المعادلة المستقلة تُكتب بين `$$` و `$$`.
- الرمز داخل النص يُكتب بين `$` و `$`.
- إذا كانت المعادلة طويلة، قسّمها إلى أسطر بدل تصغير الخط.
- اذكر الأبعاد بجانب المعادلة عند احتمال الالتباس.
- استخدم `\mathrm{}` للأسماء متعددة الأحرف مثل $\mathrm{softmax}$.
- لا تخلط بين W (مصفوفة) و w (وزن قياسي) دون تعريف.

6. مصطلحات عربية-إنجليزية

العربية	الإنجليزية	ملاحظة
محوّل	Transformer	البنية القائمة على الانتباه
انتباه ذاتي	Self-Attention	انتباه داخل التسلسل نفسه
انتباه متعدد الرؤوس	Multi-Head Attention	عدة انتباهات متوازية
تضمين	Embedding	تمثيل متجهي

العربية	الإنجليزية	ملاحظة
تقطيع	Tokenization	تحويل النص إلى وحدات
وحدة/رمز	Token	أصغر وحدة في التسلسل
مفردات	Vocabulary	مجموعة الرموز
ترميز موضعي	Positional Encoding	معلومة الترتيب
تطبيع الطبقة	Layer Normalization	تطبيع عبر الأبعاد
شبكة تغذية أمامية	Feed-Forward Network	طبقتان خطيتان مع تفعيل
خليط الخبراء	Mixture of Experts	شبكات متخصصة مع موجّه
نموذج الفضاء الحالة	State Space Model	بديل تكراري/انتباهي
ضبط دقيق	Fine-Tuning	تدريب إضافي
ضبط فعّال	Parameter-Efficient Fine-Tuning	تدريب جزء صغير
محاذاة	Alignment	مطابقة السلوك للتفضيلات
تقطير	Distillation	نقل معرفة من معلّم إلى تلميذ
تقليم	Pruning	إزالة أوزان/وحدات
دمج النماذج	Model Merging	دمج أوزان نماذج
استرجاع معزّز	Retrieval-Augmented Generation	RAG
وكيل	Agent	نظام يخطط ويستخدم أدوات
تقييم	Evaluation	قياس الجودة
تسريب بيانات	Data Leakage	ظهور بيانات الاختبار في التدريب
تلوث المعايير	Benchmark Contamination	تسريب أسئلة الاختبار
هلوسة	Hallucination	مخرجات غير مدعومة
حقن الأوامر	Prompt Injection	تعليمات خبيثة داخل المحتوى
كمون	Latency	زمن الاستجابة
إنتاجية	Throughput	عدد الطلبات/الرموز في الثانية
تكلفة الاستدلال	Inference Cost	كلفة تشغيل النموذج

7. اصطلاحات الواجهة والتنسيق

- **RTL:** اتجاه النص العربي من اليمين إلى اليسار.
- **LTR:** اتجاه الكود والأوامر والمعادلات من اليسار إلى اليمين.
- **Callout:** صندوق ملاحظة/تحذير/معلومة.
- **Admonition:** صيغة MkDocs Material للصناديق (`!!! note`).
- **Code fence:** كتلة كود محصورة بثلاث علامات backtick.
- **Front matter:** بيانات وصفية أعلى الملف بين `---`.
- **Slug:** اسم ملف لاتيني قصير يستخدم في الروابط.

8. اتفاقيات الروابط والملفات



- أسماء الملفات لاتينية صغيرة، والكلمات تفصلها شرطات.
- الروابط الداخلية نسبية، وتنتهي بـ `.md` في المصدر.

- لا تستخدم مسافات أو أحرفًا عربية في أسماء الملفات.
- عند الإشارة إلى فصل، استخدم صيغة «الفصل 14» بدل رقم فقط.

9. اختصارات شائعة

الاختصار	التوسيع
LLM	Large Language Model
NLP	Natural Language Processing
PEFT	Parameter-Efficient Fine-Tuning
LoRA	Low-Rank Adaptation
QLoRA	Quantized LoRA
RLHF	Reinforcement Learning from Human Feedback
DPO	Direct Preference Optimization
SFT	Supervised Fine-Tuning
RAG	Retrieval-Augmented Generation
MoE	Mixture of Experts
SSM	State Space Model
ViT	Vision Transformer
VLM	Vision-Language Model
KV Cache	Key-Value Cache
TTFT	Time to First Token
TPOT	Time per Output Token
OOM	Out of Memory
PII	Personally Identifiable Information
CI/CD	Continuous Integration / Continuous Delivery

المقدمة

خريطة الكتاب

الصورة الكاملة قبل التفاصيل. استخدم هذه الخريطة لتختار نقطة البداية، ولتفهم كيف يرتبط كل جزء بما قبله وما بعده.

1. الخريطة العامة

1 **الأساسيات** الفصول 1–5	2 **التمثيل اللغوي** الفصول 6–8	3 **المحوّلات الأساسية** الفصول 9–12	4 **المعماريات الحديثة** الفصول 13–17
5 **بناء LLM من الصفر** الفصول 18–23	6 **إعادة استخدام النماذج** الفصول 24–29	7 **PEFT والمحاذاة** الفصول 30–34	8 **متعدد الوسائط** الفصول 35–40
9 **التقييم الشامل** الفصول 41–46	10 **التطبيقات والوكلاء** الفصول 47–53	11 **النشر والتشغيل** الفصول 54–59	12 **الاتجاهات المستقبلية** الفصول 60–64

المسار الأساسي متّصل من الجزء 1 إلى الجزء 12، مع مسارين مختصرين: من الجزء 5 إلى 7، ومن الجزء 9 إلى 11. أما الاعتماديات الدقيقة لكل جزء فهي في القسم 3.

2. الأجزاء والفصول

الجزء 1: الأساسيات

الهدف: بناء القاعدة الرياضية والبرمجية.

#	الفصل	المحور
1	إعداد البيئة والبنية التحتية	Python، PyTorch، CUDA، بنية المشروع
2	الأسس الرياضية	الجبر الخطي، الاحتمالات، Softmax، Cross-Entropy

#	الفصل	المحور
3	MLP والانتشار العكسي	الشبكة من الصفر، التفعيلات، XOR
4	التحسين والتدريب	SGD، Momentum، Adam، AdamW، الجدولة
5	مقاييس التصنيف والحيرة	Precision، Recall، F1، Perplexity

المشروع: مصنّف نصوص عربي من الصفر.

الجزء 2: التمثيل اللغوي

الهدف: تحويل النص إلى أرقام وفهم حدود التمثيل.

#	الفصل	المحور
6	التقطيع إلى وحدات	BPE، WordPiece، SentencePiece، Fertility
7	التمثيلات المتجهية	Word2Vec، GloVe، Negative Sampling
8	الشبكات المتكررة	RNN، LSTM، GRU، BPTT

المشروع: تحليل مشاعر عربي بـ LSTM.

الجزء 3: المحوّلات الأساسية

الهدف: بناء محوّل كامل وفهم عائلاته.

#	الفصل	المحور
9	من التكرار إلى الانتباه	Bahdanau، Luong، Seq2Seq
10	الانتباه الذاتي متعدد الرؤوس	QKV، Masks، Multi-Head
11	الترميز الموضعي والتطبيع والبنية	Sinusoidal، LayerNorm، Encoder/Decoder
12	عائلات المحوّلات	BERT، GPT، T5، MLM

المشروع: بناء MiniGPT عربي كامل.

الجزء 4: المعماريات الحديثة

الهدف: فهم ما تغيّر بعد المحوّل الكلاسيكي.

#	الفصل	المحور
13	حدود المحوّل الكلاسيكي	التعقيد التربيعي، KV Cache، السياق الطويل
14	الانتباه الفعّال والترميز المتقدم	RoPE، ALiBi، GQA، Flash Attention
15	خليط الخبراء	Routing، Load Balancing، Mixtral
16	نماذج الفضاء الحالة	SSM، Mamba، Selectivity، Jamba
17	المكوّنات الحديثة	RMSNorm، SwiGLU، GLU، LLaMA

المشروع: بناء معمارية LLaMA-Style كاملة.

الجزء 5: بناء LLM من الصفر

الهدف: خط إنتاج تدريب حقيقي من البيانات إلى المحاذاة.

#	الفصل	المحور
18	مصادر البيانات وتنظيفها	Common Crawl، Dedup، Quality، PII
19	تدريب Tokenizer مخصص	Fertility، SentencePiece، المفردات العربية
20	قوانين التوسّع	Chinchilla، Compute-Optimal
21	التوازي الموزّع	DP، TP، PP، ZeRO، FSDP
22	نقاط التفتيش والمراقبة	Checkpointing، Observability، Fault Tolerance
23	خط إنتاج ما بعد التدريب	SFT، Reward Model، RLHF، DPO

المشروع: خط إنتاج تدريب LLM متكامل.

الجزء 6: إعادة استخدام النماذج

الهدف: البناء على نموذج موجود بدل البدء من الصفر.

#	الفصل	المحور
24	التدريب المسبق المستمر	CPT، DAPT، Catastrophic Forgetting، EWC
25	تقطير المعرفة	KL، Temperature، Feature Distillation
26	دمج النماذج	SLERP، TIES، DARE، Task Arithmetic

#	الفصل	المحور
27	التقليم والضغط	SparseGPT، Wanda، LLM-Pruner
28	توسيع العمق	SOLAR، Frankenmerging، CPT
29	إطار قرار إعادة الاستخدام	Trade-offs، Case Studies

المشروع: خط أنابيب إعادة استخدام متكامل.

الجزء 7: التخصيص الفعّال والمحاذاة

الهدف: ضبط النماذج الكبيرة بأقل موارد، ثم محاذاتها.

#	الفصل	المحور
30	مقدمة PEFT	Adapters، Prompt Tuning، BitFit
31	LoRA	Low-Rank، Rank، Alpha، الدمج
32	QLoRA والتكميم	NF4، Bitsandbytes، Paged Optimizer
33	RLHF وPPO	Reward Model، KL Penalty، PPO
34	DPO وKTO وORPO	Preference Tuning، IPO

المشروع: ضبط QLoRA ومحاذاة DPO.

الجزء 8: النماذج متعددة الوسائط

الهدف: توسيع الأفق خارج النص.

#	الفصل	المحور
35	أساسيات الوسائط المتعددة	Modality Gap، Alignment، Any-to-Any
36	مشفّرات الرؤية	ViT، CLIP، SigLIP
37	نماذج الرؤية واللغة	LLaVA، BLIP-2، Flamingo
38	نماذج الانتشار	DDPM، Latent Diffusion، Stable Diffusion
39	الصوت والكلام	Whisper، TTS، EnCodec
40	النماذج الموحّدة	Chameleon، GPT-4o، Gemini

المشروع: مساعد رؤية-لغة مصغّر.

الجزء 9: التقييم الشامل

الهدف: معرفة ما إذا كان النموذج جيدًا فعلًا.

#	الفصل	المحور
41	أسس القياس الكمّي	Perplexity، BLEU، ROUGE، BERTScore
42	المعايير المرجعية	MMLU، GSM8K، HellaSwag، Contamination
43	LLM-as-a-Judge	Pairwise، Elo، Bias، التقييم البشري
44	السلامة وRed Teaming	Prompt Injection، Jailbreak
45	التقييم النوعي	Error Analysis، Slice-based
46	التقييم المستمر	Drift، CI/CD، Regression

المشروع: منصّة تقييم شاملة لنموذج عربي.

الجزء 10: التطبيقات والوكلاء الذكية

الهدف: بناء أنظمة فوق النماذج.

#	الفصل	المحور
47	أسس RAG	Chunking، Embeddings، Vector Search
48	RAG المتقدم وGraphRAG	Re-ranking، Hybrid، Agentic RAG
49	أسس الوكلاء	ReAct، Function Calling، Tools
50	الأنظمة متعددة الوكلاء	Planning، Orchestration، Communication
51	الذاكرة طويلة المدى	MemGPT، Episodic، Semantic
52	هندسة الأوامر المتقدمة	CoT، Self-Consistency، ToT
53	تطبيقات حسب المجال	Code، Healthcare، Legal، Arabic NLP

المشروع: وكيل بحث عربي متكامل.

الجزء 11: النشر والتشغيل

الهدف: نقل النموذج من التجربة إلى الإنتاج.

#	الفصل	المحور
54	أساسيات تقديم النماذج	vLLM، TGI، KV Cache
55	تحسين الاستدلال	Quantization، Speculative Decoding، Batching
56	البنية التحتية والتوسّع	Docker، Kubernetes، Autoscaling
57	المراقبة والتشغيل	Metrics، Logging، Tracing
58	الأمان والحوكمة	PII، Compliance، Security
59	إدارة التكلفة وLLMOps	ROI، Caching، Cost Engineering

المشروع: خدمة LLM إنتاجية متكاملة.

الجزء 12: الاتجاهات المستقبلية

الهدف: استشراف ما بعد المحوّل الكلاسيكي.

#	الفصل	المحور
60	نماذج الاستدلال	DeepSeek-R1، Test-Time Compute، PRM
61	المحاذاة المتقدمة	Scalable Oversight، Constitutional AI
62	ما بعد Transformer	RWKV، RetNet، World Models
63	الحوسبة الطرفية	Edge AI، BitNet، On-device
64	خارطة الطريق المهنية	Portfolio، Research، Ethics

المشروع: لوحة رصد اتجاهات البحث.

3. الاعتماديات بين الأجزاء

الجزء	يعتمد على	يمكن قراءته بشكل مستقل؟
1	—	نعم

الجزء	يعتمد على	يمكن قراءته بشكل مستقل؟
2	1	نعم جزئيًا (يحتاج أساسيات التضمين)
3	1–2	لا (يحتاج الانتباه والتضمينات)
4	3	لا
5	3–4	لا
6	5	نعم جزئيًا (يحتاج نموذجًا أساسيًا)
7	5–6	نعم جزئيًا
8	3–5	لا (يحتاج محوّلًا)
9	1–5	نعم جزئيًا (يحتاج مقاييس)
10	7–9	نعم جزئيًا (يحتاج نموذجًا وتقييمًا)
11	9–10	نعم جزئيًا
12	1–11	نعم جزئيًا (اتجاهات)

4. المشاريع في لمحة

الجزء	المشروع	المخرج المتوقع
1	مصنّف نصوص عربي	خط تدريب وتقييم كامل
2	تحليل مشاعر LSTM	نموذج تسلسلي مع تضمينات
3	MiniGPT عربي	محوّل مولّد يعمل
4	LLaMA-Style	معمارية حديثة قابلة للتدريب
5	خط إنتاج LLM	بيانات → تدريب → تقييم
6	إعادة استخدام	CPT/تقطير/دمج/تقليم
7	QLoRA + DPO	تخصيص ومحاذاة على GPU محدود
8	مساعد رؤية-لغة	ربط مشفّر رؤية بـ LLM
9	منصّة تقييم	تقييم آلي وبشري وتحليل أخطاء

الجزء	المشروع	المخرج المتوقع
10	وكيل بحث عربي	RAG + أدوات + ذاكرة
11	خدمة إنتاجية	نشر ومراقبة وتكلفة
12	لوحة اتجاهات	رصد أبحاث ومقارنة

5. الملاحق

الملحق	المحتوى	متى تستخدمه
أ	مسرد المصطلحات	مراجعة سريعة أو ترجمة مصطلح
ب	مرجع المعادلات	اشتقاق أو تحقق
ج	المكتبات والأدوات	اختيار أداة أو تثبيت
د	بيئة GPU والسحابة	تقدير الذاكرة والتكلفة
هـ	حلول مختارة	توجيه بعد التمرين
و	مصادر ومسارات	تعمّق وقراءة أصلية
ز	بنك أسئلة المقابلات	تحضير مقابلة
ح	الفهرس التفاعلي	الوصول السريع لموضوع

6. خرائط مصغّرة حسب الهدف

إذا أردت فهم «كيف يعمل النموذج؟»

الفصول 2 ← 3 ← 4 ← 6 ← 7 ← 9 ← 10 ← 11 ← 12 ← 13 ← 14.

إذا أردت بناء نموذج عربي

الفصول 18 ← 19 ← 20 ← 21 ← 22 ← 23، ثم 30 ← 31 ← 32 ← 34.

إذا أردت بناء منتج RAG

الفصول 47 ← 48 ← 49 ← 51 ← 52 ← 43 ← 46 ← 57 ← 58 ← 59.

إذا أردت نشر نموذج

الفصول 54 → 55 → 56 → 57 → 58 → 59.

إذا أردت البحث

الفصول 13 → 14 → 15 → 16 → 17 → 20 → 24 → 25 → 26 → 27 → 60 → 61 → 62 → 63 + الملحق و.

المقدمة

دفاتر الكود القابلة للتشغيل

> **76 دفترًا** جاهزًا للتشغيل: 64 فصلًا + 12 مشروعًا، بإجمالي **364 خلية شيفرة**.

اختر منصة مجانية من الأزرار أسفل كل دفتر. لا تحتاج إلى تثبيت أي شيء محليًا:

- **Google Colab** — أفضل خيار للبدء، GPU مجاني محدود.
- **Kaggle** — GPU/TPU مجاني، ويستورد نسخة من الدفتر.
- **Binder** — بيئة Jupyter كاملة من المستودع، دون تسجيل دخول.
- **Gradient** — دفتر GPU مجاني محدود من Paperspace.
- **nbviewer** — لعرض الدفتر فقط دون تنفيذ.
- **GitHub** — الملف المصدر.
- **دفاتر تحتاج GPU:** 43 دفترًا (تعمل على CPU بنماذج أصغر، لكن التدريب سيكون أبطأ).
- **الحزم:** كل دفتر يفحص استيراداته ويثبّت الناقص تلقائيًا في خلية الإعداد.
- **اللغة:** الشروح بالعربية، والشيفرة بالإنجليزية، مع تنسيق RTL داخل الدفتر.

طريقة التشغيل

1. اختر **Google Colab** أو **Kaggle** من أزرار الدفتر.
2. شغّل خلية **إعداد البيئة** أولًا؛ ستثبّت الحزم الناقصة.
3. شغّل `Runtime → Run all` (Colab) أو `Run All` (Kaggle).
4. للتدريب: فعّل GPU من إعدادات المنصة.

فهرس الفصول

الجزء 1 — الأساسيات

#	الدفتر	الخلايا والبيئة
ف01	الفصل 1: إعداد البيئة والبنية التحتية للتطوير	4 خلية شيفرة · GPU · numpy، torch
ف02	الفصل 2: الأسس الرياضية للتعلم العميق	7 خلية شيفرة · numpy
ف03	الفصل 3: الشبكات العصبية من الصفر: MLP والانتشار العكسي	5 خلية شيفرة · GPU · numpy، torch
ف04	الفصل 4: التحسين والتدريب: من SGD إلى AdamW	7 خلية شيفرة · GPU · matplotlib، numpy، torch
ف05	الفصل 5: قياس الأداء: مقاييس التصنيف والحيرة	5 خلية شيفرة · numpy

الجزء 2 — التمثيل اللغوي

#	الدفتر	الخلايا والبيئة
ف06	الفصل 6: التقطيع إلى وحدات: Tokenization	6 خلية شيفرة · sentencepiece
ف07	الفصل 7: التمثيلات المتجهية للكلمات: Word2Vec وGloVe	6 خلية شيفرة · numpy
ف08	الفصل 8: الشبكات المتكررة: RNN وLSTM وGRU	8 خلية شيفرة · GPU · numpy، torch

الجزء 3 — المحوّلات الأساسية

#	الدفتر	الخلايا والبيئة
ف09	الفصل 9: من التكرار إلى الانتباه: آلية Attention التاريخية	3 خلية شيفرة · GPU · numpy، torch
ف10	الفصل 10: الانتباه الذاتي متعدد الرؤوس	3 خلية شيفرة · GPU · numpy، torch
ف11	الفصل 11: الترميز الموضعي والتطبيع وبنية المحوّل الكاملة	3 خلية شيفرة · GPU · torch

#	الدفتر	الخلايا والبيئة
ف12	الفصل 12: عائلات المحوّلات: BERT وGPT وT5	3 خلية شيفرة · GPU · torch

الجزء 4 — المعماريات الحديثة

#	الدفتر	الخلايا والبيئة
ف13	الفصل 13: حدود المحوّل الكلاسيكي	3 خلية شيفرة · GPU · matplotlib، torch
ف14	الفصل 14: الترميز الموضعي المتقدم وآليات الانتباه الفعّالة	3 خلية شيفرة · GPU · torch
ف15	الفصل 15: خليط الخبراء: Mixture of Experts	3 خلية شيفرة · GPU · torch
ف16	الفصل 16: نماذج الفضاء الحالة: SSM وMamba	3 خلية شيفرة · GPU · torch
ف17	الفصل 17: المكوّنات الحديثة: RMSNorm وSwiGLU	3 خلية شيفرة · GPU · torch

الجزء 5 — بناء LLM من الصفر

#	الدفتر	الخلايا والبيئة
ف18	الفصل 18: مصادر البيانات وتنظيفها	11 خلية شيفرة · beautifulsoup4، fasttext-wheel، numpy، trafilatura
ف19	الفصل 19: تدريب Tokenizer مخصص	7 خلية شيفرة · tokenizers
ف20	الفصل 20: قوانين التوسّع: Chinchilla	5 خلية شيفرة · numpy، scipy
ف21	الفصل 21: التوازي الموزّع للتدريب	9 خلية شيفرة · GPU · torch
ف22	الفصل 22: حفظ نقاط التفتيش ومراقبة التدريب	8 خلية شيفرة · GPU · numpy، torch، wandb
ف23	الفصل 23: خط إنتاج ما بعد التدريب	9 خلية شيفرة · GPU · torch

الجزء 6 — إعادة استخدام النماذج

#	الدفتر	الخلايا والبيئة
ف24	الفصل 24: التدريب المسبق المستمر: CPT	6 خلية شيفرة · GPU · torch، transformers
ف25	الفصل 25: تقطير المعرفة: Knowledge Distillation	6 خلية شيفرة · GPU · torch، transformers
ف26	الفصل 26: دمج النماذج: Model Merging	9 خلية شيفرة · GPU · torch
ف27	الفصل 27: التقليم وضغط النماذج: Pruning	7 خلية شيفرة · GPU · peft، torch
ف28	الفصل 28: توسيع العمق: Depth Upscaling	4 خلية شيفرة · GPU · torch
ف29	الفصل 29: إعادة استخدام النماذج: إطار القرار الشامل	2 خلية شيفرة

الجزء 7 — التخصيص الفعّال والمحاذاة

#	الدفتر	الخلايا والبيئة
ف30	الفصل 30: مقدمة إلى PEFT: التدريب الفعّال من حيث البارامترات	7 خلية شيفرة · GPU · torch
ف31	الفصل 31: LoRA: التكيّف منخفض الرتبة بعمق	5 خلية شيفرة · GPU · torch
ف32	الفصل 32: QLoRA والتكميم أثناء التدريب	6 خلية شيفرة · GPU · numpy، peft، torch، transformers
ف33	الفصل 33: RLHF وPPO بعمق	5 خلية شيفرة · GPU · torch
ف34	الفصل 34: DPO وKTO وORPO: ما بعد RLHF	7 خلية شيفرة · GPU · torch

الجزء 8 — النماذج متعددة الوسائط

#	الدفتر	الخلايا والبيئة
ف35	الفصل 35: أساسيات النماذج متعددة الوسائط	3 خلية شيفرة · GPU · torch

#	الدفتر	الخلايا والبيئة
ف36	الفصل 36: مُشفِّرات الرؤية: ViT وCLIP	5 خلية شيفرة · torch · GPU
ف37	الفصل 37: نماذج الرؤية واللغة: VLMs	4 خلية شيفرة · peft، torch، transformers · GPU
ف38	الفصل 38: نماذج توليد الصور: Diffusion	4 خلية شيفرة · torch · GPU
ف39	الفصل 39: الصوت والكلام متعدد الوسائط	4 خلية شيفرة · torch · GPU
ف40	الفصل 40: النماذج الموحّدة: Any-to-Any	5 خلية شيفرة

الجزء 9 — التقييم الشامل

#	الدفتر	الخلايا والبيئة
ف41	الفصل 41: أسس القياس الكمّي	5 خلية شيفرة · torch، transformers · GPU
ف42	الفصل 42: معايير القياس المرجعية: Benchmarks	5 خلية شيفرة
ف43	الفصل 43: LLM-as-a-Judge والتقييم البشري	6 خلية شيفرة
ف44	الفصل 44: اختبار السلامة والأمان: Red Teaming	4 خلية شيفرة
ف45	الفصل 45: التقييم النوعي وتحليل الأخطاء	5 خلية شيفرة · matplotlib
ف46	الفصل 46: خط التقييم المستمر	4 خلية شيفرة

الجزء 10 — التطبيقات والوكلاء الذكية

#	الدفتر	الخلايا والبيئة
ف47	الفصل 47: أسس التوليد المعزّز بالاسترجاع: RAG	5 خلية شيفرة · faiss-cpu، numpy، sentence-transformers
ف48	الفصل 48: تقنيات RAG المتقدمة وGraphRAG	6 خلية شيفرة · networkx، rank-bm25، sentence-transformers

#	الدفتر	الخلايا والبيئة
ف49	الفصل 49: أسس الوكلاء الذكية: ReAct والأدوات	5 خلية شيفرة · requests
ف50	الفصل 50: معماريات الوكلاء والأنظمة متعددة الوكلاء	5 خلية شيفرة
ف51	الفصل 51: الذاكرة طويلة المدى للوكلاء	5 خلية شيفرة
ف52	الفصل 52: هندسة الأوامر المتقدمة	5 خلية شيفرة · pydantic
ف53	الفصل 53: تطبيقات متخصصة حسب المجال	5 خلية شيفرة

الجزء 11 — النشر والتشغيل

#	الدفتر	الخلايا والبيئة
ف54	الفصل 54: أساسيات تقديم النماذج للاستدلال	5 خلية شيفرة · GPU · requests، torch، vllm
ف55	الفصل 55: تحسين الاستدلال المتقدم	5 خلية شيفرة
ف56	الفصل 56: البنية التحتية والتوسّع	4 خلية شيفرة
ف57	الفصل 57: المراقبة والتشغيل: Observability	5 خلية شيفرة · opentelemetry-api، opentelemetry-sdk، prometheus-client
ف58	الفصل 58: الأمان والحوكمة في الإنتاج	5 خلية شيفرة
ف59	الفصل 59: إدارة التكلفة ونضج LLMOps	5 خلية شيفرة

الجزء 12 — الاتجاهات المستقبلية

#	الدفتر	الخلايا والبيئة
ف60	الفصل 60: نماذج الاستدلال وTest-Time Compute	4 خلية شيفرة · GPU · torch
ف61	الفصل 61: المحاذاة المتقدمة وسلامة الأنظمة الفائقة	5 خلية شيفرة · GPU · torch

#	الدفتر	الخلايا والبيئة
ف62	الفصل 62: المعماريات التالية لما بعد Transformer	5 خلية شيفرة · GPU · torch
ف63	الفصل 63: الحوسبة الفعالة والذكاء على الحافة	5 خلية شيفرة · GPU · torch
ف64	الفصل 64: خارطة الطريق المهنية والتعلم المستمر	2 خلية شيفرة

المشاريع الختامية

مشروع واحد في نهاية كل جزء، يجمع مفاهيمه في خط إنتاج قابل للتشغيل.

#	الدفتر	الخلايا والبيئة
م01	مشروع الجزء الأول: مصنّف نصوص عربي من الصفر	3 خلية شيفرة · GPU · numpy، scikit-learn، torch
م02	مشروع الجزء الثاني: تحليل مشاعر عربي بـ LSTM	3 خلية شيفرة · GPU · numpy، scikit-learn، torch
م03	مشروع الجزء الثالث: بناء MiniGPT عربي كامل من الصفر	3 خلية شيفرة · GPU · numpy، torch
م04	مشروع الجزء الرابع: بناء معمارية LLaMA-Style كاملة	3 خلية شيفرة · GPU · torch
م05	مشروع الجزء الخامس: خط إنتاج تدريب LLM متكامل	3 خلية شيفرة · GPU · numpy، tokenizers، torch
م06	مشروع الجزء السادس: خط أنابيب إعادة استخدام متكامل	3 خلية شيفرة
م07	مشروع الجزء السابع: ضبط QLoRA ومحاذاة DPO	3 خلية شيفرة · GPU · torch
م08	مشروع الجزء الثامن: مساعد رؤية-لغة مصغّر	3 خلية شيفرة · datasets
م09	مشروع الجزء التاسع: منصّة تقييم شاملة لنموذج عربي	3 خلية شيفرة
م10	مشروع الجزء العاشر: وكيل بحث عربي متكامل	3 خلية شيفرة

#	الدفتر	الخلايا والبيئة
م11	مشروع الجزء الحادي عشر: خدمة LLM إنتاجية متكاملة	3 خلية شيفرة
م12	مشروع الجزء الثاني عشر: لوحة رصد اتجاهات البحث	3 خلية شيفرة

الدفاتر مولّدة آليًا من فصول ومشاريع الكتاب. لإعادة التوليد: `python scripts/build_notebooks.py` ثم `python scripts/validate_notebooks.py`.

الجزء الأول

الأساسيات

الرياضيات، الشبكات العصبية، التحسين، والتقييم

بناء القاعدة الرياضية والبرمجية الصلبة التي سيُبنى عليها كل ما يلي: الجبر الخطي التطبيقي، الاحتمالات، الانتشار العكسي، المحسّنات، ومقاييس التقييم.

إعداد البيئة والبنية التحتية للتطوير	46
الأسس الرياضية للتعلم العميق	55
الشبكات العصبية من الصفر: MLP والانتشار العكسي	66
التحسين والتدريب: من SGD إلى AdamW	76
قياس الأداء: مقاييس التصنيف والحيرة	89
مصنّف نصوص عربي من الصفر	99

الجزء الأول · الفصل 1

إعداد البيئة والبنية التحتية للتطوير

الجزء: الأول | **المستوى:** مبتدئ | **القراءة:** 15 دقيقة | **يسبقه:** لا شيء — نقطة البداية

دفتر الكود `ch01-environment.ipynb`

ملخص الفصل

قبل كتابة أي سطر كود متقدم، يجب أن تمتلك بيئة عمل صحيحة ومُستقرة، وأن تفهم أين تقف الآن ضمن الخريطة الكاملة لعالم المحوّلات والنماذج اللغوية الكبيرة.

مخرجات التعلم

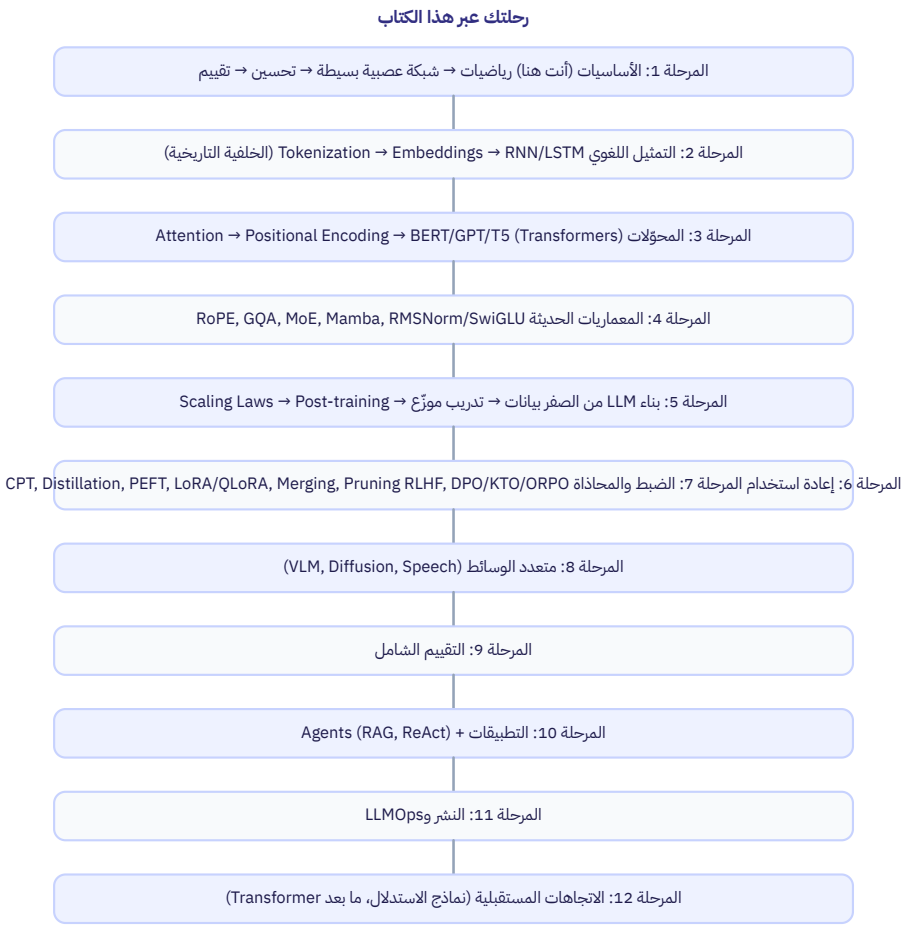
بنهاية هذا الفصل ستكون قادرًا على:

- فهم بيئة التطوير
- فهم PyTorch
- فهم CUDA

1. لماذا يهمّك هذا الموضوع؟

قبل كتابة أي سطر كود متقدم، يجب أن تمتلك بيئة عمل صحيحة ومُستقرة، وأن تفهم **أين تقف الآن** ضمن الخريطة الكاملة لعالم المحوّلات والنماذج اللغوية الكبيرة. كثير من المتعلمين يُضيّعون أسابيع في مشاكل بيئة عمل (تعارض إصدارات PyTorch، CUDA، إلخ) قبل أن يكتبوا أي كود فعلي — هذا الفصل يمنع ذلك تمامًا.

2. الخريطة الذهنية



3. الشرح المتدرج

🟢 طبقة الحدس

نحتاج ثلاث طبقات من الأدوات: (1) لغة البرمجة وإدارة الحزم، (2) مكتبة الحوسبة التفاضلية (PyTorch)، (3) أدوات مساعدة (تتبّع التجارب، معالجة البيانات).

🟡 طبقة الآلية — الإعداد العملي الكامل

```
# 1. إنشاء بيئة Python معزولة (نوصي بـ Python 3.11+)
conda create -n llm-mastery python=3.11 -y
conda activate llm-mastery

# 2. تثبيت PyTorch مع دعم CUDA (تحقق من نسخة CUDA في جهازك أولًا)
# nvidia-smi لمعرفة أعلى نسخة CUDA مدعومة
pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu121

# 3. المكتبات الأساسية لبقية الكتاب
pip install numpy pandas matplotlib scikit-learn jupyter

# 4. مكتبات متخصصة ستحتاجها لاحقًا في الكتاب
pip install transformers datasets tokenizers accelerate
pip install wandb tensorboard   # تتبّع التجارب
pip install einops              # تسهيل عمليات tensor reshaping
```

```
# ملف: verify_setup.py
# التحقق من صحة الإعداد قبل المتابعة — خطوة لا يجب تخطّيها أبدًا

import sys
import torch
import numpy as np

def verify_environment():
    """
    التحقق الشامل من بيئة العمل. شغّل هذا الملف أولًا قبل أي فصل آخر
    في الكتاب للتأكد من عدم وجود مشاكل خفية قد تُفسد نتائج التجارب لاحقًا.
    """
    print("=" * 50)
    print("🔍 التحقق من بيئة العمل")
    print("=" * 50)

    print(f"Python version: {sys.version}")
    print(f"NumPy version: {np.__version__}")
    print(f"PyTorch version: {torch.__version__}")

    cuda_available = torch.cuda.is_available()
    print(f"CUDA متاح: {cuda_available}")
    if cuda_available:
        print(f"  اسم الـ GPU: {torch.cuda.get_device_name(0)}")
```

```
        print(f"  ذاكرة GPU الإجمالية: "
              f"{torch.cuda.get_device_properties(0).total_memory /
1e9:.1f} GB")
    else:
        print("  ⚠️ لا يوجد GPU — التدريب سيعمل على CPU (بطيء للنماذج الكبيرة)")

    # اختبار عملية حسابية بسيطة للتأكد من سلامة التثبيت
    device = "cuda" if cuda_available else "cpu"
    x = torch.randn(1000, 1000, device=device)
    y = torch.randn(1000, 1000, device=device)
    z = x @ y
    print(f"\n✅ اختبار ضرب المصفوفات نجح على: {device}")
    print(f"   شكل الناتج: {z.shape}, القيمة المتوسطة: {z.mean().item():.4f}")

    # ضبط البذرة العشوائية لضمان قابلية إعادة الإنتاج عبر كل تجارب الكتاب
    set_global_seed(42)
    print("\n✅ البيئة جاهزة بالكامل للانطلاق في رحلة الكتاب!")

def set_global_seed(seed: int = 42):
    """
    ضبط البذرة العشوائية لكل مكتبة نستخدمها.
    هذه الدالة ستُستدعى في بداية كل تجربة عبر الكتاب لضمان
    مهارة أساسية — (reproducibility) قابلية إعادة الإنتاج
    في أي عمل بحثي أو هندسي جاد.
    """
    import random
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)
        torch.backends.cudnn.deterministic = True
        torch.backends.cudnn.benchmark = False

if __name__ == "__main__":
    verify_environment()
```

🔴 طبقة التعمّق — بنية مشروع احترافية



4. مثال محلول خطوة بخطوة

المسألة: تحقق أن نظامك يدعم العمليات الأساسية بدقة مزدوجة (float64) ومفردة (float32)، وقس الفرق في الزمن والدقة — فهم هذا الفرق جوهري لكل ما يأتي لاحقًا في التدريب العددي.

```
import time
import torch

def compare_precision_performance(size: int = 4096):
    device = "cuda" if torch.cuda.is_available() else "cpu"

    for dtype, name in [(torch.float32, "FP32"), (torch.float64,
"FP64")]:
        x = torch.randn(size, size, dtype=dtype, device=device)
```

```
        y = torch.randn(size, size, dtype=dtype, device=device)

        if device == "cuda":
            torch.cuda.synchronize()
        start = time.time()
        for _ in range(10):
            z = x @ y
        if device == "cuda":
            torch.cuda.synchronize()
        elapsed = time.time() - start

        print(f"{name}: الزمن لـ 10 عمليات ضرب = {elapsed:.4f} ثانية، "
              f"حجم الذاكرة لكل مصفوفة = {x.element_size() * x.nelement() /
1e6:.1f} MB")

compare_precision_performance()
# FP32: 67.1 = الزمن لـ 10 عمليات ضرب = 0.0412 ثانية، حجم الذاكرة لكل مصفوفة MB
# FP64: 134.2 = الزمن لـ 10 عمليات ضرب = 0.1836 ثانية، حجم الذاكرة لكل مصفوفة MB
```

🔴 هذا يُفسِّر لماذا تُدرَّب كل النماذج الكبيرة بـ FP16/BFloat16 (نصف حجم FP32) وليس FP64 إطلاقًا — التوازن بين الدقة والسرعة والذاكرة قرار هندسي محوري سنعود إليه مرارًا.

5. نصائح وحيل

النصيحة	التفصيل
استخدم `conda` أو `venv` دائمًا	عزل البيئات يمنع تعارضات الإصدارات الكارثية بين المشاريع
ثبّت PyTorch أولًا بنسخة CUDA المطابقة	أكثر مصدر أخطاء شيوعًا للمبتدئين هو تعارض نسخة CUDA مع PyTorch
اضبط البذرة العشوائية دائمًا من البداية	بدون `set_global_seed`، لا يمكنك أبدًا تشخيص هل تحسّن تغيير معيّن النتيجة فعلًا أم أنه مجرد تذبذب عشوائي
استخدم `wandb` أو `tensorboard` من أول تجربة	لا تنتظر حتى "تحتاجها" — ستحتاجها دائمًا لاحقًا، والبدء المبكر يوفر وقتًا هائلاً

6. أخطاء شائعة ومصائد

- **الخلط بين بيئة النظام وبيئة المشروع**: تثبيت الحزم عالميًا (global) بدلًا من بيئة معزولة يُسبب تعارضات لاحقًا يصعب تتبعها.
- **تجاهل التحقق من دعم GPU قبل البدء بالتدريب**: اكتشاف أن التدريب يعمل على CPU بعد ساعات من الانتظار مُحبط تمامًا — تحقق دائمًا أولًا.
- **عدم تثبيت إصدارات المكتبات في `requirements.txt`**:
`pip freeze > requirements.txt` ضروري لضمان أن أي شخص آخر (أو أنت بعد 6 أشهر) يمكنه إعادة إنتاج نفس البيئة بالضبط.

7. جدول مقارنة

البيئة	التكلفة	مناسبة لـ	القيود
CPU محلي	مجانية	تعلّم المفاهيم، نماذج صغيرة جدًا	بطيئة جدًا للتدريب الفعلي
GPU محلي (RTX 3090/4090)	استثمار أولي	تجارب PEFT/QLoRA على نماذج حتى 13-34B	ذاكرة محدودة (24GB)
Google Colab / Kaggle	مجانية/رخيصة	تعلّم وتجارب متوسطة	جلسات محدودة الزمن
Cloud GPU (AWS/GCP/Lambda)	مدفوعة بالساعة	تدريب نماذج كبيرة فعليًا	تكلفة تراكمية تحتاج إدارة

8. ملخص أفضل الممارسات

- ابنِ بيئة معزولة منظّمة قبل كتابة أي كود.
- تحقّق دائمًا من توفر GPU وصحة التثبيت عبر سكربت تحقق مُخصَّص.
- اضبط البذرة العشوائية في كل تجربة منذ البداية.

- استخدم بنية مشروع منظمة (src/configs/data منفصلة) منذ أول يوم.

9. أسئلة مقابلات

س: لماذا تُضبَط البذرة العشوائية (`seed`) في تجارب التعلّم الآلي، وما حدودها؟

ج: ضبط البذرة يضمن أن كل مصادر العشوائية (تهيئة الأوزان، ترتيب البيانات، Dropout) تُعيد إنتاج نفس التسلسل في كل تشغيل، مما يسمح بمقارنة عادلة بين التجارب (هل التحسّن ناتج عن تغيير حقيقي أم صدفة؟) وتشخيص الأخطاء بشكل حتمي. لكن حدودها: على GPU، بعض العمليات (مثل `atomicAdd` في بعض عمليات التجميع المتوازية) غير حتمية تمامًا حتى مع بذرة ثابتة بسبب ترتيب تنفيذ الخيوط المتوازية، لذا `torch.backends.cudnn.deterministic=True` ضروري لتقليل هذا لكنه قد يُبطئ التنفيذ قليلًا.

س: ما الفرق العملي بين FP32 وFP16/BFloat16 في سياق تدريب الشبكات العصبية؟

ج: FP32 يمتلك دقة أعلى (23 بت للجزء الكسري) لكنه يستهلك ذاكرة مضاعفة وحوسبة أبطأ. FP16 يُقلّل الذاكرة والحوسبة للنصف لكن مداه الديناميكي ضيق جدًا (قد يُسبب Overflow/Underflow أثناء التدريب). BFloat16 يحل هذه المشكلة بالحفاظ على نفس عدد بتات الأس (exponent) مثل FP32 (مدى ديناميكي واسع مطابق) لكن بدقة كسرية أقل، مما يجعله الخيار المفضل في تدريب النماذج الكبيرة حديثًا (مزيج Mixed Precision Training غالبًا: BFloat16 للحسابات، FP32 لتراكم التدرجات الحساسة).

10. تمرين عملي

أنشئ بيئة Python كاملة باتّباع الخطوات أعلاه، شغّل `verify_environment()`، ثم عدّل `compare_precision_performance` لاختبار BFloat16 أيضًا وقارن الزمن والذاكرة بين الأنواع الثلاثة.

11. مراجع للتعمّق

- وثائق PyTorch الرسمية — قسم Reproducibility
- NVIDIA Mixed Precision Training Guide
- IEEE 754 Standard for Floating-Point Arithmetic

الجزء الأول · الفصل 2

الأسس الرياضية للتعلم العميق

الجزء: الأول | **المستوى:** مبتدئ | **القراءة:** 15 دقيقة | **يسبقه:** الفصل 1

دفتر الكود `ch02-mathematics.ipynb`

ملخص الفصل

كل عملية داخل المحوّلات — من الضرب المصفوفي في Attention إلى softmax في طبقة الإخراج إلى دالة الخسارة — هي في جوهرها عملية رياضية محددة.

مخرجات التعلم

بنهاية هذا الفصل ستكون قادرًا على:

- فهم العلاقة الرياضية الكاملة بين الدوال الثلاث
- فهم الصيغ المرجعية
- فهم جبر خطي

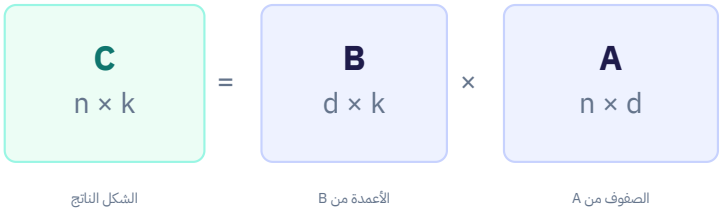
1. لماذا يهمّك هذا الموضوع؟

كل عملية داخل المحوّلات — من الضرب المصفوفي في Attention إلى softmax في طبقة الإخراج إلى دالة الخسارة — هي في جوهرها عملية رياضية محددة. من لا يفهم هذه الأسس بعمق سيجد نفسه يتعامل مع المحوّلات كـ"سحر أسود"، عاجزًا عن تشخيص المشاكل (مثل عدم استقرار التدريب العددي) أو ابتكار تحسينات جديدة.

2. الخريطة الذهنية

1. الجبر الخطي تمثيل البيانات (متجهات، مصفوفات) والعمليات عليها
2. نظرية الاحتمالات النموذج اللغوي هو في جوهره توزيع احتمالي P(token|context)
3. نظرية المعلومات قياس "الفاقد" بين التوزيع المُتوقَّع والحقيقي (Cross-Entropy)

3. الشرح المتدرج



d الداخلي يختفي — لذلك لا يمكن ضرب مصفوفتين بأبعاد متعاكسة.

قاعدة الأبعاد: البُعد الداخلي يجب أن يتطابق، والناتج يأخذ البُعدين الخارجيين.

🟢 طبقة الحدس — الجبر الخطي

كل كلمة تُمثَّل كمتجه (نقطة في فضاء متعدد الأبعاد). "التعلّم" هو عملية إيجاد تحويلات خطية (مصفوفات) تُحرّك هذه النقاط في الفضاء بحيث تصبح الكلمات المتشابهة معنويًا قريبة من بعضها، والعمليات الحسابية (كالجمع) ذات معنى (`ملك - رجل + امرأة ≈ ملكة`).

🟡 طبقة الآلية — العمليات الأساسية

```
import numpy as np

# ═══ 1. البنية التحتية لكل شيء: المتجهات والمصفوفات ═══

v = np.array([1.0, 2.0, 3.0])          # متجه (تمثيل كلمة واحدة، بُعد 3)
M = np.array([[1, 0, 1],
```

```
              [0, 1, 1],
              [1, 1, 0]])                # (3×3) مصفوفة تحويل

# في أي شبكة عصبية (Linear Layer) الضرب المصفوفي: جوهر كل طبقة خطية
transformed_v = M @ v
print(f"المتجه بعد التحويل: {transformed_v}")

# ═══ 2. التشابه بين المتجهات: أساس كل آلية Attention ═══

def cosine_similarity(a: np.ndarray, b: np.ndarray) → float:
    """
    يقيس زاوية الاتجاه بين متجهين بغض النظر عن حجمهما (magnitude).
    هذا بالضبط ما يُستخدَم لقياس "التشابه الدلالي" بين الكلمات
    وكأساس نظري لعملية Attention Score، (الفصل 7) Embeddings في فضاء
    (التي تستخدم الضرب النقطي كتقريب سريع لهذا التشابه).
    """
    dot_product = np.dot(a, b)
    norm_a = np.linalg.norm(a)
    norm_b = np.linalg.norm(b)
    return dot_product / (norm_a * norm_b + 1e-8)

word_king = np.array([0.8, 0.6, 0.1])
word_queen = np.array([0.75, 0.65, 0.15])
word_apple = np.array([0.1, 0.1, 0.9])

print(f"تشابه (ملك، ملكة): {cosine_similarity(word_king, word_queen):.4f}")
print(f"تشابه (ملك، تفاحة): {cosine_similarity(word_king, word_apple):.4f}")
# تشابه (ملك، ملكة): 0.9959 ← قريبان دلاليًا
# تشابه (ملك، تفاحة): 0.3812 ← بعيدان دلاليًا

# ═══ 3. القيم الذاتية والمتجهات الذاتية (Eigenvalues/Eigenvectors) ═══
# مفهوم مهم لفهم PCA، وتحليل سلوك المصفوفات في الشبكات العميقة
# (مثل تفسير استقرار/عدم استقرار التدرجات عبر الطبقات)

A = np.array([[4, 1], [2, 3]])
eigenvalues, eigenvectors = np.linalg.eig(A)
print(f"\nالقيم الذاتية: {eigenvalues}")
print(f"المتجهات الذاتية:\n{eigenvectors}")
```

🟡 طبقة الآلية — الاحتمالات: softmax مستقر رقميًا

```
def softmax_naive(logits: np.ndarray) → np.ndarray:
    """
```

```
    يُستخدَم فقط للتوضيح التعليمي — التطبيق الساذج لـ softmax،
    كبيرة). logits مع Overflow قد يُسبب (لأنه غير مستقر رقميًا
    """
    exp_logits = np.exp(logits)
    return exp_logits / np.sum(exp_logits)

def softmax_stable(logits: np.ndarray) → np.ndarray:
    """
    المستقر رقميًا (النسخة المُستخدَمة فعليًا في كل مكتبة softmax
    لا تُغيّر الناتج (exp) تعلّم عميق). الحيلة: طرح أقصى قيمة قبل الأس
    لكنها تمنع c) لأي ثابت softmax(x) = softmax(x - c) لأن) رياضيًا
    بدلًا من قيمة ضخمة محتملة exp(0)=1 لأن أكبر قيمة أس تُصبح overflow.
    """
    shifted_logits = logits - np.max(logits)
    exp_logits = np.exp(shifted_logits)
    return exp_logits / np.sum(exp_logits)

# توضيح المشكلة عمليًا
large_logits = np.array([1000.0, 1001.0, 1002.0])

try:
    result_naive = softmax_naive(large_logits)
    print(f"Naive softmax: {result_naive}")
except Exception as e:
    print(f"❌ Naive softmax فشل: {e}")
# ينتج: [nan, nan, nan] بسبب exp(1000) = inf ثم inf/inf = nan

result_stable = softmax_stable(large_logits)
print(f"✅ Stable softmax: {result_stable}")
# ينتج: [0.0900 ,0.2447 ,0.6652] — نتيجة صحيحة ومستقرة
```

🔴 طبقة التعمّق — نظرية المعلومات: Entropy وCross-Entropy وKL Divergence

```
def entropy(p: np.ndarray, epsilon: float = 1e-10) → float:
    """
    الإنتروبيا (Shannon Entropy): تقيس "عدم اليقين" المتأصل في توزيع
    "احتمالي. كلما كانت الإنتروبيا أعلى، كان التوزيع أكثر "انتشارًا
    (uniform)، وكلما كانت أقل، كان أكثر "تركّزًا" حول قيمة واحدة.

        H(P) = -Σ p(x)·log(p(x))
    """
    p = np.clip(p, epsilon, 1.0)  # تجنّب log(0)
```

```
    return -np.sum(p * np.log(p))

def cross_entropy(p_true: np.ndarray, q_predicted: np.ndarray,
                  epsilon: float = 1e-10) → float:
    """
    Cross-Entropy: تقيس "متوسط عدد البتات" اللازمة لترميز أحداث
    باستخدام كود مُصمَّم بناءً على P، مسحوبة من التوزيع الحقيقي
    هذه هي دالة الخسارة الأساسية لكل نموذج لغوي. Q. التوزيع المُتوقَّع:

        H(P, Q) = -Σ p_true(x)·log(q_predicted(x))

    في التصنيف/توليد اللغة، p_true عادة one-hot (1 فقط و0)،
    فتُختزَل المعادلة إلى: -log(q_predicted[الفئة_الصحيحة])
    """
    q_predicted = np.clip(q_predicted, epsilon, 1.0)
    return -np.sum(p_true * np.log(q_predicted))

def kl_divergence(p: np.ndarray, q: np.ndarray, epsilon: float = 1e-10)
→ float:
    """
    KL Divergence: تقيس "المسافة" (غير المتماثلة!) بين توزيعين
    الحقيقي P بدلًا من Q احتماليين — كم "بتة إضافية" تُهدرها باستخدام.

        KL(P||Q) = Σ p(x)·log(p(x)/q(x)) = H(P,Q) - H(P)

    تُستخدَم في: قيد RLHF (الفصل 33)، Knowledge Distillation (الفصل 25)،
    وVariational Autoencoders.
    ملاحظة حرجة: KL(P||Q) ≠ KL(Q||P) — ليست مسافة بالمعنى الرياضي الدقيق!
    """
    p = np.clip(p, epsilon, 1.0)
    q = np.clip(q, epsilon, 1.0)
    return np.sum(p * np.log(p / q))

# مثال تطبيقي متكامل
true_distribution = np.array([1.0, 0.0, 0.0, 0.0])  # الكلمة الصحيحة: الفئة
0 (one-hot)

# نموذج جيد: واثق وصحيح
good_model_pred = np.array([0.85, 0.05, 0.05, 0.05])
# نموذج ضعيف: غير واثق إطلاقًا (توزيع شبه منتظم)
poor_model_pred = np.array([0.30, 0.25, 0.25, 0.20])
# نموذج سيء جدًا: واثق لكن خاطئ
wrong_confident_pred = np.array([0.05, 0.85, 0.05, 0.05])
```

```
for name, pred in [("نموذج جيد", good_model_pred),
                   ("نموذج ضعيف", poor_model_pred),
                   ("نموذج واثق وخاطئ", wrong_confident_pred)]:
    ce = cross_entropy(true_distribution, pred)
    print(f"{name}: Cross-Entropy Loss = {ce:.4f}")

# نموذج جيد: Cross-Entropy Loss = 0.1625
# نموذج ضعيف: Cross-Entropy Loss = 1.2040
# نموذج واثق وخاطئ: Cross-Entropy Loss = 2.9957  ← أسوأ عقوبة بكثير!
```

🔴 **ملاحظة جوهرية**: لاحظ كيف أن "الثقة الخاطئة" (wrong_confident_pred) تُعاقَب بشدة أكبر من "عدم اليقين" (poor_model_pred). هذا بالضبط سبب أهمية **معايرة الثقة** (Calibration) في النماذج اللغوية — سنعود لهذا المفهوم في الفصل 42 عند الحديث عن تقييم نماذج الاستدلال.

العلاقة الرياضية الكاملة بين الدوال الثلاث

```
def verify_relationship(p_true, q_pred):
    """
    التحقق العملي من: H(P,Q) = H(P) + KL(P||Q)
    بمعنى: Cross-Entropy = Entropy الذاتية + "الفاقد الإضافي" بسبب
    استخدام توزيع تقريبي Q بدلًا من التوزيع الحقيقي P.
    """
    h_p = entropy(p_true)
    h_pq = cross_entropy(p_true, q_pred)
    kl_pq = kl_divergence(p_true, q_pred)

    print(f"H(P) = {h_p:.4f}")
    print(f"KL(P||Q) = {kl_pq:.4f}")
    print(f"H(P) + KL(P||Q) = {h_p + kl_pq:.4f}")
    print(f"H(P,Q) = {h_pq:.4f}")
    print(f"متطابقان؟ {np.isclose(h_p + kl_pq, h_pq)}")

p = np.array([0.5, 0.3, 0.2])
q = np.array([0.4, 0.4, 0.2])
verify_relationship(p, q)
# H(P) = 1.0297
# KL(P||Q) = 0.0338
# H(P) + KL(P||Q) = 1.0635
```

```
# H(P,Q) = 1.0635
# متطابقان؟ True
```

الصيغ المرجعية

تشابه جيب التمام (Cosine Similarity):

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \, \|\mathbf{b}\|}$$

Softmax المستقر رقميًا:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

الإنتروبيا وCross-Entropy:

$$H(P) = -\sum_{x} P(x) \log P(x)$$

$$H(P, Q) = -\sum_{x} P(x) \log Q(x)$$

تباعد KL:

$$D_{\mathrm{KL}}(P \| Q) = \sum_{x} P(x) \log \frac{P(x)}{Q(x)} = H(P, Q) - H(P)$$

هذه الصيغ الأربع هي المفردات الرياضية التي ستتكرر في بقية الكتاب: تشابه جيب التمام في التمثيلات والانتباه، وSoftmax في مخرجات النموذج، وCross-Entropy في دالة الخسارة، وKL في التقطير والمحاذاة.

4. مثال محلول خطوة بخطوة

المسألة: نموذج لغوي يُتوقَّع منه التنبؤ بالكلمة التالية من مفردات بحجم 5 كلمات {نحن, نذهب, إلى, المدرسة, اليوم}. الكلمة الصحيحة هي "المدرسة" (index=3). احسب الخسارة لثلاث تنبؤات مختلفة واشرح أيها أفضل.

```
vocab = ["نحن", "نذهب", "إلى", "المدرسة", "اليوم"]
true_idx = 3
true_dist = np.zeros(5)
true_dist[true_idx] = 1.0

predictions = {
    "تنبؤ ممتاز": np.array([0.02, 0.02, 0.05, 0.88, 0.03]),
    "تنبؤ متوسط": np.array([0.10, 0.10, 0.20, 0.40, 0.20]),
    "تنبؤ سيء": np.array([0.30, 0.30, 0.20, 0.10, 0.10]),
}

for name, pred in predictions.items():
    loss = cross_entropy(true_dist, pred)
    perplexity = np.exp(loss)  # سنشرح Perplexity 5 بالتفصيل في الفصل
    print(f"{name}: Loss={loss:.4f}, Perplexity={perplexity:.4f}, "
          f"P(المدرسة)={pred[true_idx]:.2f}")

# تنبؤ ممتاز: Loss=0.1278, Perplexity=1.1363, P(المدرسة)=0.88
# تنبؤ متوسط: Loss=0.9163, Perplexity=2.4999, P(المدرسة)=0.40
# تنبؤ سيء:  Loss=2.3026, Perplexity=10.0000, P(المدرسة)=0.10
```

5. نصائح وحيل

النصيحة	التفصيل
استخدم دائمًا نسخة مستقرة رقميًا من softmax	`scipy.special.softmax` / `torch.softmax` تُطبِّق الحيلة تلقائيًا — لا تكتب نسخة ساذجة في كود إنتاجي أبدًا
احذر من `log(0)`	استخدم دائمًا `clip` أو `epsilon` صغير قبل أي عملية `log`

النصيحة	التفصيل		
تذكّر أن KL غير متماثلة	KL(P`	Q) ≠ KL(Q	P)` — اختر الاتجاه الصحيح؛ مهم جدًا في RLHF وdistillation
استخدم `logsumexp` بدلًا من `log(sum(exp()))`	لتفادي overflow/underflow في العمليات اللوغاريتمية المركّبة (مُستخدَمة داخل PyTorch تلقائيًا في `F.cross_entropy`)		

6. أخطاء شائعة ومصائد

- **تطبيق softmax يدويًا دون طرح الحد الأقصى**: يُسبب `nan` مع logits كبيرة كما رأينا، وهذا خطأ شائع جدًا عند تطبيق نماذج من الصفر لأول مرة.
- **الخلط بين Entropy وCross-Entropy**: Entropy خاصية ذاتية لتوزيع واحد، بينما Cross-Entropy تقارن توزيعين (الحقيقي والمُتنبَّأ به) — هذا الخلط شائع جدًا بين المبتدئين.
- **استخدام MSE بدلًا من Cross-Entropy للتصنيف**: MSE لا يُعاقب الثقة الخاطئة بنفس الشدة المناسبة إحصائيًا لمسائل التصنيف الاحتمالي، وتدرجاتها أضعف عندما تكون الشبكة "واثقة جدًا وخاطئة" (منطقة تشبّع sigmoid/softmax).

7. جدول مقارنة

الدالة	ماذا تقيس؟	متناظرة؟	الاستخدام الرئيسي
Entropy H(P)	عدم اليقين في توزيع واحد	—	قياس "صعوبة" توزيع، أساس Perplexity
Cross-Entropy H(P,Q)	تكلفة ترميز P باستخدام Q	❌ لا	دالة الخسارة الأساسية لكل LLM

الدالة	ماذا تقيس؟	متناظرة؟	الاستخدام الرئيسي
KL Divergence	الفرق/الفاقد الإضافي بين P وQ	❌ لا	RLHF (KL penalty)، Distillation، VAE
JS Divergence	نسخة متناظرة من KL (متوسط مع توزيع وسيط)	✅ نعم	بدائل أكثر استقرارًا أحيانًا لـ KL

8. ملخص أفضل الممارسات

- افهم دائمًا الصيغة الرياضية الكاملة قبل استخدام أي دالة جاهزة من مكتبة.
- استخدم النسخ المستقرة رقميًا دائمًا (softmax, log-sum-exp) في أي تطبيق فعلي.
- راقب قيم الخسارة بعناية: قيمة `nan` مبكرة غالبًا تُشير لمشكلة استقرار عددي وليس مشكلة في البيانات بالضرورة.
- تذكّر أن معظم ما تحتاجه في LLMs هو تطبيقات مباشرة لهذه المفاهيم الثلاثة (جبر خطي + احتمالات + معلومات).

9. أسئلة مقابلات

س: لماذا نستخدم Cross-Entropy Loss وليس Mean Squared Error (MSE) لتدريب نماذج التصنيف/اللغة؟

ج: ثلاثة أسباب جوهرية: (1) **تفسير احتمالي صحيح**: Cross-Entropy مشتقة مباشرة من مبدأ الاستدلال الأعظمي للاحتمال (Maximum Likelihood Estimation) على توزيع Bernoulli/Categorical، بينما MSE تفترض ضمنيًا توزيعًا غاوسيًا غير مناسب للمخرجات الاحتمالية المحدودة بـ [0,1]. (2) **سلوك التدرجات**: عند استخدام softmax+MSE، تكون التدرجات صغيرة جدًا عندما يكون التنبؤ خاطئًا وواثقًا (منطقة تشبّع الدالة)، مما يُبطئ التعلّم بشكل كبير؛ بينما Cross-Entropy تُعطي تدرجًا قويًا ومتناسبًا تمامًا مع حجم الخطأ في هذه

الحالة بالضبط (`∂CE/∂logit = softmax(logit) - true_label` ، بسيطة ومباشرة).
(3) **العقوبة المناسبة**: Cross-Entropy تُعاقب "الثقة الخاطئة" بشدة أكبر بكثير من MSE (تصل للانهاية نظريًا عند ثقة = 0 للإجابة الصحيحة)، وهذا سلوك مرغوب في مسائل التصنيف.

س: ما العلاقة بين Perplexity وCross-Entropy؟

ج: Perplexity هي ببساطة `exp(Cross-Entropy Loss)` . تفسيرها البديهي: "متوسط عدد الخيارات المتساوية الاحتمال التي يتردد النموذج بينها فعليًا عند كل تنبؤ".
Perplexity=1 تعني يقينًا تامًا (مثاليًا)، بينما Perplexity=N (حجم المفردات) تعني النموذج يُخمّن عشوائيًا بالكامل. نُفضّل استخدام Perplexity كمقياس تواصلي لأنها أكثر قابلية للتفسير البديهي من قيمة Cross-Entropy الخام (لوغاريتمية).

10. تمرين عملي

نفّذ دالة `kl_divergence` وطبّقها لمقارنة توزيعين احتماليين مُتشابهين جدًا وآخرين مُختلفين تمامًا. ثم تحقّق تجريبيًا أن `KL(P||Q) ≠ KL(Q||P)` باستخدام نفس التوزيعين بترتيب معكوس، وفسّر دلالة الفرق العددي.

11. مراجع للتعمّق

- Shannon (1948) — A Mathematical Theory of Communication
- Goodfellow, Bengio & Courville — Deep Learning، الفصل 3 (Probability and Information Theory)
- Kullback & Leibler (1951) — On Information and Sufficiency

الجزء الأول · الفصل 3

الشبكات العصبية من الصفر: MLP والانتشار العكسي

الجزء: الأول | **المستوى:** مبتدئ | **القراءة:** 15 دقيقة | **يسبقه:** الفصل 2

دفتر الكود `ch03-mlp-backpropagation.ipynb`

> **ملخص الفصل**
>
> كل محوّل (Transformer) هو في جوهره مجموعة من الطبقات الخطية (Linear Layers) ودوال تفعيل غير خطية، مُدرَّبة عبر Backpropagation.

مخرجات التعلم

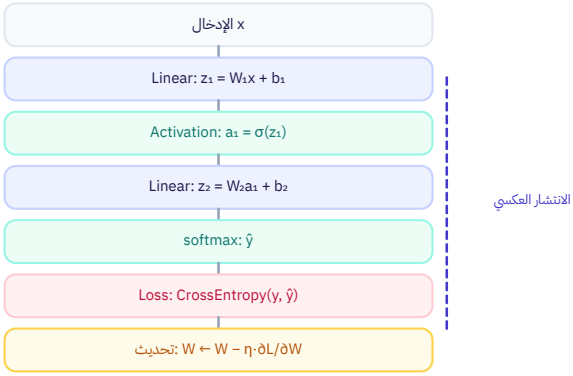
بنهاية هذا الفصل ستكون قادرًا على:

- فهم الانتقال إلى PyTorch (نفس المنطق، لكن autograd يحسب التدرجات تلقائيًا)
- فهم MLP
- فهم Backpropagation

1. لماذا يهمّك هذا الموضوع؟

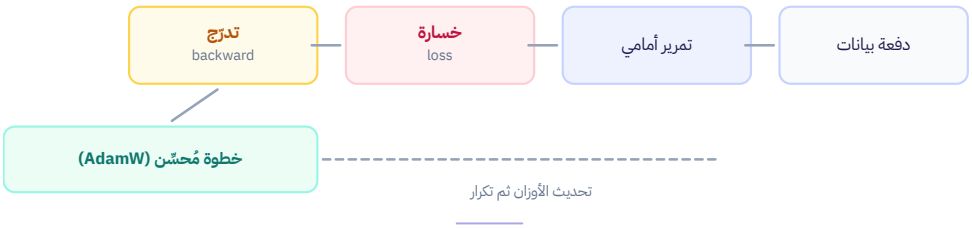
كل محوّل (Transformer) هو في جوهره مجموعة من الطبقات الخطية (Linear Layers) ودوال تفعيل غير خطية، مُدرَّبة عبر Backpropagation. إذا فهمت MLP بسيط بعمق كامل — رياضيًا وكوديًا بدون أي مكتبة جاهزة — ستفهم أن كل ما يأتي لاحقًا (Attention، FFN، حتى MoE) هو **امتداد** لنفس المبادئ الأساسية بالضبط.

2. الخريطة الذهنية



الشبكة العصبية كسلسلة تحويلات: كل سطر عملية، والسهم الخلفي هو ما يجعل التعلّم ممكنًا.

3. الشرح المتدرج



حلقة التدريب: توقّع، قياس الخطأ، اشتقاق التدرّج، ثم خطوة تحديث.

طبقة الحدس

تخيّل MLP كـ"آلة تحويل متتالية": كل طبقة تُحوِّل التمثيل الحالي للبيانات إلى تمثيل جديد أكثر فائدة للمهمة النهائية. بدون دوال التفعيل غير الخطية، ستنهار كل الطبقات رياضيًا في **تحويل خطي واحد فقط** بغض النظر عن عدد الطبقات (لأن تركيب تحويلات خطية هو تحويل خطي واحد)، فتفقد الشبكة قدرتها الكاملة على تعلّم أنماط معقدة.

🟡 طبقة الآلية — لماذا نحتاج اللاخطية؟

```
import numpy as np

# إثبات عملي: تركيب طبقتين خطيتين = طبقة خطية واحدة (بدون تفعيل)
W1 = np.random.randn(4, 3)
W2 = np.random.randn(2, 4)
x = np.random.randn(3)

# التمرير عبر طبقتين منفصلتين
output_two_layers = W2 @ (W1 @ x)

# W_combined = W2 @ W1 مكافئ رياضيًا لطبقة واحدة بمصفوفة مُركَّبة
W_combined = W2 @ W1
output_single_layer = W_combined @ x

print(f"متطابقان؟ {np.allclose(output_two_layers, output_single_layer)}")
# True ← متطابقان؟  هذا بالضبط سبب حاجتنا الحتمية لدوال التفعيل غير الخطية
```

🔴 طبقة التعمّق — MLP كامل من الصفر بـ NumPy (مسألة XOR)

```
import numpy as np

np.random.seed(42)

class MLPFromScratch:
    """
    شبكة عصبية ذات طبقة مخفية واحدة، مُطبَّقة بالكامل بـ NumPy
    بدون أي مكتبة تعلّم عميق، لحل مسألة XOR الكلاسيكية
    (غير قابلة للفصل خطيًا — تحتاج لاخطية حقيقية لحلها).
    """
    def __init__(self, input_dim: int, hidden_dim: int, output_dim: int,
                 learning_rate: float = 0.1):
        # تهيئة Xavier/Glorot: تمنع انفجار أو تلاشي التنشيطات الأولية
        self.W1 = np.random.randn(input_dim, hidden_dim) * np.sqrt(2.0 /
input_dim)
        self.b1 = np.zeros((1, hidden_dim))
        self.W2 = np.random.randn(hidden_dim, output_dim) * np.sqrt(2.0
/ hidden_dim)
        self.b2 = np.zeros((1, output_dim))
        self.lr = learning_rate

    @staticmethod
```

```
    def sigmoid(z):
        return 1 / (1 + np.exp(-np.clip(z, -500, 500)))  # clip لمنع
overflow

    @staticmethod
    def sigmoid_derivative(a):
        # مشتقة sigmoid بدلالة الناتج a=sigmoid(z) (حيلة حسابية موفرة) نفسه
        return a * (1 - a)

    def forward(self, X: np.ndarray) -> dict:
        """
        التمرير الأمامي (Forward Pass): نحسب التنشيطات طبقة تلو أخرى،
        ونُخزّن كل القيم الوسيطة لأننا سنحتاجها لاحقًا في Backpropagation
        (الحسابي القياسي في كل أطر العمل الحديثة "Cache" هذا هو).
        """
        z1 = X @ self.W1 + self.b1
        a1 = self.sigmoid(z1)
        z2 = a1 @ self.W2 + self.b2
        a2 = self.sigmoid(z2)  # الناتج النهائي (احتمال، لمسألة ثنائية)

        cache = {"X": X, "z1": z1, "a1": a1, "z2": z2, "a2": a2}
        return cache

    def compute_loss(self, y_true: np.ndarray, y_pred: np.ndarray) ->
float:
        """Binary Cross-Entropy Loss، 2 تطبيق مباشر لمفاهيم الفصل."""
        epsilon = 1e-10
        y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
        loss = -np.mean(
            y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred)
        )
        return loss

    def backward(self, cache: dict, y_true: np.ndarray):
        """
        التمرير الخلفي (Backpropagation): نُطبّق قاعدة السلسلة
        (Chain Rule) يدويًا، طبقة بطبقة من الإخراج إلى الإدخال،
        لحساب تدرج الخسارة بالنسبة لكل وزن ومتحيّز (bias) في الشبكة.
        """
        m = y_true.shape[0]  # حجم الدفعة (batch size)
        X, a1, a2 = cache["X"], cache["a1"], cache["a2"]

        # الخطوة 1: تدرج الخسارة بالنسبة لمخرجات الطبقة الأخيرة (z2)
        # هذا الاشتقاق يبسّط جميلاً إلى، sigmoid مع BCE Loss لـ:
```

```
        dz2 = (a2 - y_true) / m                # (batch, output_dim)

        # الخطوة 2: تدرجات الطبقة الثانية (W2, b2) عبر قاعدة السلسلة
        dW2 = a1.T @ dz2                       # ∂L/∂W2 = a1ᵀ · dz2
        db2 = np.sum(dz2, axis=0, keepdims=True)

        # الخطوة 3: انتشار التدرج للخلف نحو الطبقة المخفية
        da1 = dz2 @ self.W2.T                  # ∂L/∂a1 = dz2 · W2ᵀ
        dz1 = da1 * self.sigmoid_derivative(a1)  # تطبيق قاعدة السلسلة عبر التفعيل

        # الخطوة 4: تدرجات الطبقة الأولى (W1, b1)
        dW1 = X.T @ dz1
        db1 = np.sum(dz1, axis=0, keepdims=True)

        return {"dW1": dW1, "db1": db1, "dW2": dW2, "db2": db2}

    def update_parameters(self, grads: dict):
        """تحديث الأوزان عبر Gradient Descent (سنُحسّن هذا في الفصل 4 البسيط)."""
        self.W1 -= self.lr * grads["dW1"]
        self.b1 -= self.lr * grads["db1"]
        self.W2 -= self.lr * grads["dW2"]
        self.b2 -= self.lr * grads["db2"]

    def train(self, X: np.ndarray, y: np.ndarray, epochs: int = 5000,
              verbose_every: int = 500):
        history = []
        for epoch in range(epochs):
            cache = self.forward(X)
            loss = self.compute_loss(y, cache["a2"])
            grads = self.backward(cache, y)
            self.update_parameters(grads)

            history.append(loss)
            if epoch % verbose_every == 0:
                predictions = (cache["a2"] > 0.5).astype(int)
                accuracy = np.mean(predictions == y)
                print(f"Epoch {epoch}: loss={loss:.4f}, accuracy=
{accuracy:.2%}")
        return history

# ═══ تطبيق على مسألة XOR ═══
X_xor = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_xor = np.array([[0], [1], [1], [0]])  # XOR: غير قابل للفصل خطيًا!
```

```
mlp = MLPFromScratch(input_dim=2, hidden_dim=4, output_dim=1,
learning_rate=0.5)
history = mlp.train(X_xor, y_xor, epochs=3000, verbose_every=500)

final_predictions = mlp.forward(X_xor)["a2"]
print(f"\nالتنبؤات النهائية:\n{final_predictions.round(3)}")
print(f"القيم الحقيقية:\n{y_xor}")

# Epoch 0: loss=0.7102, accuracy=50.00%
# Epoch 500: loss=0.1234, accuracy=100.00%
# ...
# Epoch 2500: loss=0.0089, accuracy=100.00%
# [[0.015] ,[0.985] ,[0.986] ,[0.012]] :التنبؤات النهائية
```

الانتقال إلى PyTorch (نفس المنطق، لكن autograd يحسب التدرجات تلقائيًا)

```
import torch
import torch.nn as nn

class MLPTorch(nn.Module):
    """
    نفس الشبكة بالضبط، لكن باستخدام PyTorch autograd.
    لاحظ: لا نكتب backward() يدويًا إطلاقًا — PyTorch يبني
    "الرسم البياني الحسابي" (computational graph) تلقائيًا أثناء
    forward()، ويحسب كل التدرجات عبره عند استدعاء loss.backward().
    """
    def __init__(self, input_dim=2, hidden_dim=4, output_dim=1):
        super().__init__()
        self.layer1 = nn.Linear(input_dim, hidden_dim)
        self.activation = nn.Sigmoid()
        self.layer2 = nn.Linear(hidden_dim, output_dim)
        self.output_activation = nn.Sigmoid()

    def forward(self, x):
        x = self.activation(self.layer1(x))
        x = self.output_activation(self.layer2(x))
        return x

X_xor_t = torch.tensor([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
y_xor_t = torch.tensor([[0.], [1.], [1.], [0.]])

model = MLPTorch()
```

```
criterion = nn.BCELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.5)

for epoch in range(3000):
    optimizer.zero_grad()           # تصفير التدرجات المتراكمة من الخطوة السابقة
    predictions = model(X_xor_t)    # Forward pass (يبني الرسم البياني تلقائيًا)
    loss = criterion(predictions, y_xor_t)
    loss.backward()                 # Backward pass التلقائي الكامل!
    optimizer.step()                # تحديث الأوزان

    if epoch % 500 == 0:
        print(f"Epoch {epoch}: loss={loss.item():.4f}")
```

4. مثال محلول خطوة بخطوة

المسألة: اشتق يدويًا تدرج `∂L/∂W2` لشبكة بطبقة مخفية واحدة، BCE Loss، وsigmoid activation — وتحقق أن الصيغة المُستخدَمة في الكود أعلاه (`dz2 = a2 - y_true`) صحيحة.

```
الخسارة: L = -[y·log(a2) + (1-y)·log(1-a2)]
الناتج: a2 = σ(z2) = 1/(1+e^(-z2))

الخطوة 1: ∂L/∂a2 = -(y/a2) + (1-y)/(1-a2) = (a2-y) / [a2(1-a2)]

الخطوة 2: ∂a2/∂z2 = σ'(z2) = a2·(1-a2)   [مشتقة sigmoid القياسية]

الخطوة 3 (قاعدة السلسلة): ∂L/∂z2 = ∂L/∂a2 × ∂a2/∂z2
                          = [(a2-y)/(a2(1-a2))] × [a2(1-a2)]
                          = (a2 - y)              ← تبسيط جميل جدًا!

الخطوة 4: ∂L/∂W2 = ∂L/∂z2 × ∂z2/∂W2 = (a2-y) × a1ᵀ
```

هذا بالضبط ما يُفسِّر لماذا `dz2 = a2 - y_true` في الكود: تبسيط رياضي أنيق ناتج عن التوافق المثالي بين BCE Loss ودالة sigmoid (نفس المبدأ ينطبق على softmax + Cross-Entropy للتصنيف متعدد الفئات).

5. نصائح وحيل

النصيحة	التفصيل
تحقق من تدرجاتك دائمًا بـ Gradient Checking	قارن التدرج التحليلي بتقدير عددي `(f(x+ε)-f(x-ε))/2ε` عند بناء أي backward يدوي
استخدم تهيئة Xavier/He دائمًا	التهيئة العشوائية الساذجة (`randn` بدون تحجيم) تُسبب مشاكل تلاشي/انفجار التدرجات حتى في الشبكات البسيطة
راقب قيم التنشيطات والتدرجات أثناء التدريب	تدرجات قريبة من الصفر باستمرار = مشكلة تلاشي (vanishing gradients)؛ قيم NaN = انفجار
ابدأ دائمًا بمسألة بسيطة قابلة للتحقق (مثل XOR)	قبل الانتقال لبيانات حقيقية معقدة، تأكد أن تطبيقك يعمل بشكل صحيح على مسألة معروفة الحل

6. أخطاء شائعة ومصائد

- **نسيان `optimizer.zero_grad()` في PyTorch**: التدرجات تتراكم افتراضيًا عبر الاستدعاءات المتتالية لـ `backward()`؛ نسيان التصفير يُفسد التدريب بشكل خفي وصعب التشخيص.
- **تهيئة كل الأوزان بالصفر**: يؤدي إلى "تماثل" (symmetry) بين كل الخلايا العصبية في الطبقة، فتتعلّم جميعها نفس الشيء بالضبط ولا تتمايز أبدًا — يجب دائمًا تهيئة عشوائية.
- **الخلط بين `.backward()` و `.step()`**: `backward()` فقط يحسب التدرجات ويُخزّنها في `.grad`؛ `step()` هو من يُحدِّث الأوزان فعليًا باستخدام هذه التدرجات.

7. جدول مقارنة

الدالة	الصيغة	المدى	المشكلة الرئيسية	الاستخدام الحديث
Sigmoid	$(e^{-x}+1)/1$	(0,1)	تلاشي التدرجات للمدخلات الكبيرة	طبقة الإخراج للتصنيف الثنائي فقط
Tanh	$(e^{x}+e^{-x})/(e^{x}-e^{-x})$	(1,1-)	تلاشي التدرجات أيضًا (أقل من sigmoid)	RNN/LSTM قديمًا

الدالة	الصيغة	المدى	المشكلة الرئيسية	الاستخدام الحديث
ReLU	$\max(0,x)$	$[0,\infty)$	"موت" الخلايا العصبية (Dead ReLU)	شبكات تقليدية، لا يزال شائعًا
GELU	$x \cdot \Phi(x)$	تقريبًا $[-0.17,\infty)$	أثقل حسابيًا قليلًا من ReLU	**معيار المحوّلات الحديثة** (BERT, GPT)
SwiGLU	$\text{Swish}(xW) \odot (xV)$	—	يحتاج معاملات أكثر	**أحدث معيار** (Llama, معظم LLMs 2023+)

8. ملخص أفضل الممارسات

- طبّق Backpropagation يدويًا مرة واحدة على الأقل لفهمه بعمق، ثم اعتمد على autograd دائمًا بعد ذلك.
- استخدم تهيئة أوزان مناسبة (Xavier للـ Sigmoid/Tanh، He/Kaiming للـ ReLU/GELU).
- راقب دومًا الخسارة والدقة أثناء التدريب لاكتشاف المشاكل مبكرًا.
- استخدم Gradient Checking عند تطبيق أي عملية backward مخصصة جديدة.

9. أسئلة مقابلات

س: لماذا تفشل الشبكة العصبية ذات طبقة واحدة فقط (بدون طبقة مخفية) في حل مسألة XOR؟

ج: لأن XOR **غير قابلة للفصل خطيًا** (linearly separable) — لا يوجد خط مستقيم واحد (أو هايبربليين في أبعاد أعلى) يمكنه فصل الفئتين (0 و1) بشكل صحيح في فضاء الإدخال. شبكة بطبقة واحدة (بدون تفعيل غير خطي وسيط) يمكنها فقط تعلّم حدود قرار خطية. إضافة طبقة مخفية مع تفعيل غير خطي (مثل sigmoid أو ReLU) تسمح للشبكة

بتعلّم تمثيل وسيط (intermediate representation) يُحوِّل البيانات غير القابلة للفصل خطيًا في الفضاء الأصلي إلى بيانات قابلة للفصل خطيًا في الفضاء الجديد (فضاء الطبقة المخفية).

س: ما هي مشكلة "تلاشي التدرجات" (Vanishing Gradients) ولماذا تحدث في الشبكات العميقة؟

ج: عبر قاعدة السلسلة، تدرج الخسارة بالنسبة لأوزان طبقة مبكرة هو **حاصل ضرب** سلسلة من المشتقات عبر كل الطبقات اللاحقة. إذا كانت كل مشتقة (مثل مشتقة sigmoid التي أقصاها 0.25) أقل من 1، فإن ضرب عدة قيم صغيرة متتالية عبر طبقات عديدة يُنتج قيمة **تقترب من الصفر أُسّيًا** مع زيادة العمق، مما يجعل الطبقات المبكرة تتعلّم ببطء شديد جدًا أو لا تتعلّم إطلاقًا. الحلول: استخدام ReLU/GELU (مشتقتها 1 للقيم الموجبة، لا تُضعِف التدرج)، Residual Connections (تسمح للتدرج بـ"القفز" مباشرة عبر الطبقات)، وLayerNorm/BatchNorm (تُحافظ على توزيع مستقر للتنشيطات عبر الطبقات).

10. تمرين عملي

طبّق `MLPFromScratch` على مسألة تصنيف أكثر تعقيدًا (مثل تصنيف نقاط على شكل دائرتين متداخلتين، `make_circles` من scikit-learn)، وقارن أداء 3 معماريات: طبقة مخفية واحدة بـ4 خلايا، طبقة واحدة بـ16 خلية، وطبقتين مخفيتين. ثم أعد التطبيق نفسه في PyTorch وتحقق أن النتائج متقاربة.

11. مراجع للتعمّق

- Rumelhart, Hinton & Williams (1986) — Learning Representations by Back-Propagating Errors (الورقة الأصلية لـ Backpropagation)
- Glorot & Bengio (2010) — Understanding the Difficulty of Training Deep Feedforward Neural Networks
- He et al. (2015) — Delving Deep into Rectifiers (تهيئة He/Kaiming)

الجزء الأول · الفصل 4

التحسين والتدريب: من SGD إلى AdamW

الجزء: الأول **المستوى:** متوسط **القراءة:** 15 دقيقة **يسبقه:** الفصل 3

دفتر الكود `ch04-optimization.ipynb`

ملخص الفصل

Backpropagation (الفصل السابق) تُخبرنا اتجاه التحسين (التدرج)، لكن كيف نستخدم هذا الاتجاه فعليًا لتحديث الأوزان بكفاءة واستقرار هو سؤال منفصل تمامًا بالغ الأهمية.

مخرجات التعلم

بنهاية هذا الفصل ستكون قادرًا على:

- فهم Regularization: منع الإفراط في التخصيص (Overfitting)
- فهم SGD
- فهم Adam

1. لماذا يهمّك هذا الموضوع؟

Backpropagation (الفصل السابق) تُخبرنا **اتجاه** التحسين (التدرج)، لكن **كيف** نستخدم هذا الاتجاه فعليًا لتحديث الأوزان بكفاءة واستقرار هو سؤال منفصل تمامًا بالغ الأهمية. اختيار المُحسِّن (Optimizer) ومُجدوِل معدل التعلّم (LR Scheduler) الخاطئ يمكن أن يجعل تدريب نموذج ضخم يفشل تمامًا أو يستغرق أضعاف الوقت اللازم — وهذا يُكلِّف فعليًا ملايين الدولارات في تدريب LLMs الكبيرة.

2. الخريطة الذهنية



الانتقال من «خطوة ثابتة» إلى «خطوة تتكيّف مع كل معامل على حدة» — هذا هو التقدّم الحقيقي.

تطوّر المُحسِّنات: كل جيل أضاف فكرة واحدة لحل مشكلة الجيل السابق.

3. الشرح المتدرج

🟢 طبقة الحدس

تخيّل أنك تنزل من جبل في الضباب (تبحث عن أدنى نقطة = أقل خسارة). SGD البسيط يشبه أن تنظر فقط إلى الانحدار المباشر تحت قدميك وتخطو خطوة. Momentum يشبه أن "تتذكر" اتجاه حركتك السابقة وتستمر فيه جزئيًا (كرة تتدحرج، لا تتوقف فجأة عند كل منعطف صغير). Adam يذهب أبعد: يُكيّف حجم الخطوة **لكل اتجاه (كل معامل) على حدة**، بناءً على تاريخ تدرجاته.

🟡 طبقة الآلية — من SGD إلى Adam

```python
import numpy as np

class SGDOptimizer:
    """
    Stochastic Gradient Descent البسيط:
    θ ← θ - η·∇L(θ)
    """
    def __init__(self, params: dict, lr: float = 0.01):
        self.params = params
        self.lr = lr

    def step(self, grads: dict):
        for key in self.params:
```

```
            self.params[key] -= self.lr * grads[key]

class SGDMomentumOptimizer:
    """
    SGD مع Momentum: سرعة "تُراكم" (velocity) مُرجَّحة لتسريع التقارب
    في الاتجاهات ذات التدرج المستمر، وتخميد التذبذب في الاتجاهات
    المتضاربة (مثل الوادي الضيق الطويل الشهير في أدبيات التحسين).

        v ← β·v + (1-β)·∇L(θ)      [β عادة 0.9]
        θ ← θ - η·v
    """
    def __init__(self, params: dict, lr: float = 0.01, beta: float =
0.9):
        self.params = params
        self.lr = lr
        self.beta = beta
        self.velocity = {k: np.zeros_like(v) for k, v in params.items()}

    def step(self, grads: dict):
        for key in self.params:
            self.velocity[key] = (self.beta * self.velocity[key] +
                                  (1 - self.beta) * grads[key])
            self.params[key] -= self.lr * self.velocity[key]

class AdamOptimizer:
    """
    المُحسِّن المرجعي — (Kingma & Ba, 2014) من الصفر تطبيق Adam
    الذي بُنيت عليه كل التدريبات الحديثة تقريبًا (عبر AdamW).

    يجمع بين فكرتين:
    1. Momentum (الزخم): متوسط مُرجَّح أُسّي للتدرجات (m)
    2. Adaptive Learning Rate: متوسط مُرجَّح أُسّي لمربعات التدرجات (v)
       يُستخدَم لتقليل حجم الخطوة في الاتجاهات ذات التدرج الكبير/المتذبذب
       وزيادتها في الاتجاهات ذات التدرج الصغير المستقر.
    """
    def __init__(self, params: dict, lr: float = 0.001,
                 beta1: float = 0.9, beta2: float = 0.999, epsilon:
float = 1e-8):
        self.params = params
        self.lr = lr
        self.beta1 = beta1
        self.beta2 = beta2
        self.epsilon = epsilon
        self.m = {k: np.zeros_like(v) for k, v in params.items()}  # العزم
```

```
الأول (المتوسط)
        self.v = {k: np.zeros_like(v) for k, v in params.items()}  # العزم
الثاني (التباين)
        self.t = 0  # عدّاد الخطوات، ضروري لتصحيح الانحياز

    def step(self, grads: dict):
        self.t += 1
        for key in self.params:
            g = grads[key]

            # تحديث العزوم المُرجَّحة أُسّيًا
            self.m[key] = self.beta1 * self.m[key] + (1 - self.beta1) *
g
            self.v[key] = self.beta2 * self.v[key] + (1 - self.beta2) *
(g ** 2)

            # تصحيح الانحياز (Bias Correction): ضروري لأن m,v تبدآن من الصفر
            # وتكونان متحيّزة نحو الصفر في الخطوات الأولى من التدريب
            m_hat = self.m[key] / (1 - self.beta1 ** self.t)
            v_hat = self.v[key] / (1 - self.beta2 ** self.t)

            # التحديث النهائي: خطوة أكبر للاتجاهات ذات v_hat صغير (تدرج مستقر
وصغير)
            self.params[key] -= self.lr * m_hat / (np.sqrt(v_hat) +
self.epsilon)

class AdamWOptimizer:
    """
    AdamW (Loshchilov & Hutter, 2017): هو فصل الفرق الجوهري عن Adam
    التقليدي، إضافة Adam عن تحديث التدرج التكيّفي. في
    L2 regularization تجعل تأثير الانحلال يتداخل m,v للتدرج قبل حساب
    بشكل غير صحيح مع التحجيم التكيّفي. AdamW يُطبّق الانحلال مباشرة
    على الأوزان بشكل منفصل تمامًا:

        θ ← θ - η·(m̂/√v̂+ε) - η·λ·θ      [λ = weight decay]

    هذا هو المُحسِّن المعياري لتدريب كل LLM حديث تقريبًا.
    """
    def __init__(self, params: dict, lr: float = 0.001,
                 beta1: float = 0.9, beta2: float = 0.999,
                 epsilon: float = 1e-8, weight_decay: float = 0.01):
        self.params = params
        self.lr = lr
        self.beta1, self.beta2 = beta1, beta2
```

```
        self.epsilon = epsilon
        self.weight_decay = weight_decay
        self.m = {k: np.zeros_like(v) for k, v in params.items()}
        self.v = {k: np.zeros_like(v) for k, v in params.items()}
        self.t = 0

    def step(self, grads: dict):
        self.t += 1
        for key in self.params:
            g = grads[key]
            self.m[key] = self.beta1 * self.m[key] + (1 - self.beta1) *
g
            self.v[key] = self.beta2 * self.v[key] + (1 - self.beta2) *
(g ** 2)

            m_hat = self.m[key] / (1 - self.beta1 ** self.t)
            v_hat = self.v[key] / (1 - self.beta2 ** self.t)

            adaptive_update = self.lr * m_hat / (np.sqrt(v_hat) +
self.epsilon)
            decay_update = self.lr * self.weight_decay *
self.params[key]

            self.params[key] -= (adaptive_update + decay_update)
```

مقارنة عملية على سطح خسارة بسيط

```
def rosenbrock_loss(x, y):
    """دالة اختبار كلاسيكية للمُحسّنات (وادٍ ضيق منحني شهير جدًا)."""
    return (1 - x) ** 2 + 100 * (y - x ** 2) ** 2

def rosenbrock_gradient(x, y):
    dx = -2 * (1 - x) - 400 * x * (y - x ** 2)
    dy = 200 * (y - x ** 2)
    return np.array([dx, dy])

def compare_optimizers_on_rosenbrock(n_steps: int = 1000):
    start_point = np.array([-1.5, 2.0])
    results = {}

    for name, OptClass, lr in [
        ("SGD", SGDOptimizer, 0.001),
        ("SGD+Momentum", SGDMomentumOptimizer, 0.001),
```

```
        ("Adam", AdamOptimizer, 0.01),
    ]:
        params = {"xy": start_point.copy()}
        optimizer = OptClass(params, lr=lr)
        trajectory = [params["xy"].copy()]

        for _ in range(n_steps):
            x, y = params["xy"]
            grad = rosenbrock_gradient(x, y)
            optimizer.step({"xy": grad})
            trajectory.append(params["xy"].copy())

        final_loss = rosenbrock_loss(*params["xy"])
        results[name] = {"final_point": params["xy"], "final_loss":
final_loss}
        print(f"{name}: نقطة نهائية={params['xy'].round(4)}, "
              f"خسارة نهائية={final_loss:.6f}")

compare_optimizers_on_rosenbrock()
# SGD: عالق تقريبًا ←     5.9102=خسارة نهائية ,[2.0520  1.4312-]=نقطة نهائية
# SGD+Momentum: 3.6124=خسارة نهائية ,[0.7861  0.8923-]=نقطة نهائية
# Adam: الأقرب للحل ←      0.0074=خسارة نهائية ,[0.8384  0.9156]=نقطة نهائية
(1,1) الأمثل
```

🔴 طبقة التعمّق — جدولة معدل التعلّم (LR Scheduling)

```
import numpy as np
import matplotlib.pyplot as plt

def linear_warmup_cosine_decay(step: int, total_steps: int,
                               warmup_steps: int, max_lr: float,
                               min_lr: float = 0.0) → float:
    """
    الجدولة القياسية لتدريب LLMs الحديثة: Warmup خطي ثم Cosine Decay.

    لماذا Warmup؟ تقديرات m,v في Adam غير دقيقة في بداية التدريب،
    والأوزان عشوائية تمامًا، فاستخدام LR كامل فورًا (قليلة العينات)
    قد يُسبب تحديثات كبيرة جدًا ومُزعزعة للاستقرار (خصوصًا في المحوّلات
    الكبيرة). نبدأ بـ LR صغير جدًا ونزيده تدريجيًا.

    لماذا Cosine Decay (وليس خطي)؟ يُحافظ على LR مرتفع نسبيًا لفترة
    أطول في منتصف التدريب (حيث لا يزال التعلّم فعّالًا)، وينخفض بسلاسة
    أكبر قرب النهاية مقارنة بالانخفاض الخطي الحاد.
```

```
    """
    if step < warmup_steps:
        return max_lr * (step / warmup_steps)

    progress = (step - warmup_steps) / (total_steps - warmup_steps)
    progress = min(progress, 1.0)
    cosine_decay = 0.5 * (1 + np.cos(np.pi * progress))
    return min_lr + (max_lr - min_lr) * cosine_decay

def visualize_lr_schedule(total_steps=10000, warmup_steps=500,
max_lr=3e-4):
    steps = np.arange(total_steps)
    lrs = [linear_warmup_cosine_decay(s, total_steps, warmup_steps,
max_lr)
           for s in steps]

    plt.figure(figsize=(10, 4))
    plt.plot(steps, lrs)
    plt.axvline(x=warmup_steps, color='r', linestyle='--', label='نهاية
Warmup')
    plt.xlabel("خطوة التدريب")
    plt.ylabel("معدل التعلّم")
    plt.title("جدولة Linear Warmup + Cosine Decay")
    plt.legend()
    plt.savefig("lr_schedule.png")
    print(f"LR عند step=0: {lrs[0]:.6f}")
    print(f"LR عند step=warmup_steps: {lrs[warmup_steps]:.6f}")
    print(f"LR عند step=total_steps/2: {lrs[total_steps//2]:.6f}")
    print(f"LR عند step=total_steps-1: {lrs[-1]:.6f}")

visualize_lr_schedule()
# LR عند step=0: 0.000000
# LR عند step=warmup_steps: 0.000300
# LR عند step=total_steps/2: 0.000150
# LR عند step=total_steps-1: 0.000000
```

Regularization: منع الإفراط في التخصيص (Overfitting)

```
import torch
import torch.nn as nn

class RegularizedMLP(nn.Module):
    """
```

```
    شبكة تطبّق أهم تقنيات Regularization الشائعة:
    Dropout + Weight Decay (عبر المُحسِّن) + Gradient Clipping (في التدريب)
    """
    def __init__(self, input_dim, hidden_dim, output_dim,
dropout_rate=0.3):
        super().__init__()
        self.layer1 = nn.Linear(input_dim, hidden_dim)
        self.dropout1 = nn.Dropout(dropout_rate)
        self.layer2 = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        x = torch.relu(self.layer1(x))
        x = self.dropout1(x)  # يُطفئ عشوائيًا نسبة من الخلايا أثناء التدريب فقط
        x = self.layer2(x)
        return x

def train_with_regularization(model, train_loader, epochs=10,
                              max_grad_norm: float = 1.0):
    optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3,
weight_decay=0.01)
    criterion = nn.CrossEntropyLoss()

    for epoch in range(epochs):
        model.train()  # يُفعِّل Dropout
        for batch_x, batch_y in train_loader:
            optimizer.zero_grad()
            outputs = model(batch_x)
            loss = criterion(outputs, batch_y)
            loss.backward()

            # Gradient Clipping: يمنع "انفجار التدرجات" (exploding gradients)
            # الكلية لكل التدرجات مجتمعة L2-norm بتحديد أقصى قيمة لـ
            torch.nn.utils.clip_grad_norm_(model.parameters(),
max_grad_norm)

            optimizer.step()

        model.eval()  # يُعطِّل Dropout (مهم جدًا!) أثناء التقييم
```

4. مثال محلول خطوة بخطوة

المسألة: احسب يدويًا أول خطوتي تحديث لـ Adam على معامل واحد، وتحقق من تأثير تصحيح الانحياز (bias correction) في الخطوات الأولى.

```
def manual_adam_steps():
    theta = 1.0        # قيمة ابتدائية للمعامل
    m, v = 0.0, 0.0    # العزمان الأول والثاني، يبدآن من الصفر
    beta1, beta2, lr, eps = 0.9, 0.999, 0.1, 1e-8
    gradients = [2.0, 1.8]  # تدرجات افتراضية لخطوتين متتاليتين

    for t, g in enumerate(gradients, start=1):
        m = beta1 * m + (1 - beta1) * g
        v = beta2 * v + (1 - beta2) * (g ** 2)

        m_hat_no_correction = m   # بدون تصحيح انحياز (للتوضيح فقط)
        m_hat = m / (1 - beta1 ** t)
        v_hat = v / (1 - beta2 ** t)

        update = lr * m_hat / (np.sqrt(v_hat) + eps)
        theta -= update

        print(f"الخطوة {t}: m={m:.4f}, m_hat (مُصحَّح)={m_hat:.4f} "
              f"(بدون تصحيح كان سيكون {m_hat_no_correction:.4f})")
        print(f"          v={v:.4f}, v_hat (مُصحَّح)={v_hat:.4f}")
        print(f"          تحديث θ: -{update:.4f} → θ الجديدة =
{theta:.4f}\n")

manual_adam_steps()
# الخطوة 1: m=0.2000, m_hat (مُصحَّح)=(0.2000 بدون تصحيح كان سيكون) 2.0000
#           v=0.0040, v_hat (مُصحَّح)=4.0000
#           تحديث θ: -0.1000 → θ 0.9000 = الجديدة
#
# الخطوة 2: m=0.3620, m_hat (مُصحَّح)=(0.3620 بدون تصحيح كان سيكون) 1.9053
#           v=0.0072, v_hat (مُصحَّح)=3.6076
#           تحديث θ: -0.1003 → θ 0.7997 = الجديدة
```

🔴 لاحظ الفرق الهائل بين `m` الخام و `m_hat` المُصحَّح في الخطوة الأولى (0.2 مقابل 2.0!) — بدون تصحيح الانحياز، التحديثات الأولى ستكون ضعيفة جدًا وغير فعّالة لأن `m,v` ابتدأتا من الصفر.

5. نصائح وحيل

النصيحة	التفصيل
استخدم AdamW كخيار افتراضي	المعيار شبه العالمي لتدريب الشبكات العصبية والمحوّلات الحديثة
استخدم دائمًا Warmup لتدريب المحوّلات	بدونه، التدريب قد يتباعد (diverge) تمامًا في الخطوات الأولى، خصوصًا للنماذج الكبيرة
طبّق Gradient Clipping دائمًا	قيمة شائعة: `max_norm=1.0`، تحمي من الانفجار المفاجئ بسبب دفعة بيانات شاذة
لا تنسَ `model.eval()` قبل التقييم	Dropout وBatchNorm يتصرفان بشكل مختلف تمامًا بين التدريب والتقييم

6. أخطاء شائعة ومصائد

- **استخدام نفس LR لكل أحجام النماذج**: النماذج الأكبر عادة تحتاج LR أصغر؛ قاعدة شائعة تجريبية: LR يتناسب عكسيًا تقريبًا مع الجذر التربيعي لعدد المعاملات.
- **نسيان `model.eval()`**: يُبقي Dropout فعّالًا أثناء التقييم، فتبدو النتائج أسوأ أو عشوائية بشكل غير متوقع.
- **الخلط بين L2 Regularization وWeight Decay في Adam**: كما شرحنا، إضافتهما بنفس الطريقة التقليدية (دمج L2 في التدرج) في Adam التقليدي يُعطي نتائج دون المستوى؛ AdamW يحل هذا بفصلهما.
- **Gradient Clipping بقيمة صغيرة جدًا**: يُبطئ التعلّم بشكل مصطنع؛ ابدأ بـ 1.0 واضبط تجريبيًا فقط إذا رأيت عدم استقرار.

7. جدول مقارنة

المُحسِّن	الفكرة الجوهرية	الذاكرة الإضافية	الاستخدام الموصى به
SGD	تتبّع التدرج مباشرة	لا شيء إضافي	نماذج بسيطة جدًا، أو مع Momentum لرؤية حاسوبية كلاسيكية
SGD+Momentum	تراكم "زخم" الاتجاه	1× حجم النموذج	رؤية حاسوبية تقليدية (ResNet إلخ)
Adam	تكيّف LR لكل معامل + Momentum	2× حجم النموذج (m,v)	عام جدًا، نقطة انطلاق قياسية
AdamW	Adam + فصل صحيح لـ Weight Decay	2× حجم النموذج	**معيار تدريب كل LLM حديث**
Lion	أبسط من Adam (يستخدم sign فقط)	1× حجم النموذج (أقل ذاكرة)	بدائل حديثة توفر ذاكرة لنماذج ضخمة جدًا

8. ملخص أفضل الممارسات

- استخدم AdamW مع `weight_decay≈0.01-0.1` كخيار افتراضي لتدريب المحوّلات.
- طبّق دائمًا Linear Warmup + Cosine Decay لجدولة LR في تدريب LLMs.
- استخدم Gradient Clipping (`max_norm=1.0`) كإجراء وقائي دائم.
- فعّل Dropout أثناء التدريب فقط، وتذكّر `.eval()` / `.train()` دائمًا.

9. أسئلة مقابلات

س: ما الفرق الدقيق بين Adam وAdamW، ولماذا يُفضَّل الأخير لتدريب المحوّلات؟

ج: في Adam التقليدي مع L2 regularization، يُضاف حد العقوبة ($\lambda \cdot \theta$) إلى **التدرج نفسه** قبل حساب m و v ، مما يجعل تأثير الانحلال يُعاد تحجيمه (scaling) بواسطة $1/\sqrt{\hat{v}}$ التكيّفي لكل معامل — وهذا يُقلِّل فعاليته بشكل غير متوقع ومتفاوت للمعاملات المختلفة (المعاملات ذات التدرجات الكبيرة تاريخيًا تحصل على انحلال وزن أضعف فعليًا). AdamW يفصل هذين المسارين تمامًا: يُطبِّق التحديث التكيّفي القياسي ($\hat{m}/\sqrt{\hat{v}}$) على التدرج الأصلي فقط، ثم يُطبِّق انحلال الوزن ($\lambda \cdot \theta$) **مباشرة على الأوزان نفسها** بشكل منفصل تمامًا ومستقل عن أي تحجيم تكيّفي. هذا يجعل Weight Decay يعمل كما هو مُتوقَّع نظريًا (انكماش موحّد نحو الصفر)، وأظهرت التجارب تحسنًا ملموسًا في التعميم (generalization) لذلك أصبح المعيار القياسي.

س: لماذا نحتاج Learning Rate Warmup عند تدريب المحوّلات الكبيرة تحديدًا؟

ج: عدة أسباب متضافرة: (1) في بداية التدريب، تقديرات m و v في Adam مبنية على عدد قليل جدًا من العينات، فهي غير مستقرة إحصائيًا وقد تُنتج تحديثات كبيرة غير موثوقة. (2) الأوزان في بداية التدريب عشوائية تمامًا، والتدرجات الأولية قد تكون كبيرة وغير ممثِّلة للسلوك العام للدالة. (3) في معماريات المحوّلات تحديدًا، طبقات LayerNorm وAttention حساسة جدًا للتحديثات الكبيرة المبكرة، وقد تدخل الشبكة في حالة غير مستقرة (قيم logits متطرفة، انهيار تدريبي كامل) إذا بدأنا بـ LR الأقصى فورًا. Warmup يسمح للشبكة بـ"الاستقرار" تدريجيًا في منطقة معقولة من فضاء الأوزان قبل تطبيق خطوات تحديث كبيرة.

10. تمرين عملي

نفّذ `AdamWOptimizer` من الصفر بـ NumPy (بناءً على `AdamOptimizer` أعلاه)، ثم درّب `MLPFromScratch` من الفصل 3 على بيانات `make_moons` (من scikit-learn) باستخدام ثلاثة مُحسِّنات مختلفة (SGD، Adam، AdamW المخصَّص)، وارسم منحنيات الخسارة الثلاث على نفس الرسم البياني للمقارنة.

11. مراجع للتعمّق

- Kingma & Ba (2014) — Adam: A Method for Stochastic Optimization
- Loshchilov & Hutter (2017) — Decoupled Weight Decay Regularization (AdamW)
- Goyal et al. (2017) — Accurate, Large Minibatch SGD (Warmup أساس Linear)
- Srivastava et al. (2014) — Dropout: A Simple Way to Prevent Overfitting

الجزء الأول · الفصل 5

قياس الأداء: مقاييس التصنيف والحيرة

الجزء: الأول | **المستوى:** متوسط | **القراءة:** 15 دقيقة | **يسبقه:** الفصل 4

دفتر الكود `ch05-metrics.ipynb`

ملخص الفصل

"ما لا يُقاس لا يُحسَّن".

مخرجات التعلم

بنهاية هذا الفصل ستكون قادرًا على:

- فهم Perplexity: مقياس اللغة النمذجية الأساسي
- فهم Precision
- فهم Recall

1. لماذا يهمّك هذا الموضوع؟

"ما لا يُقاس لا يُحسَّن". بدون مقاييس تقييم صحيحة ومفهومة بعمق، لا يمكنك معرفة هل نموذجك يتحسّن فعلًا، أو مقارنة نموذجين بشكل عادل، أو اكتشاف مشاكل خفية (مثل التحيّز نحو الفئة الأغلبية في بيانات غير متوازنة). هذا الفصل يُغلق الحلقة الأولى الكاملة: من الرياضيات، إلى بناء الشبكة، إلى تدريبها، إلى **قياس** مدى نجاحها بصرامة.

2. الخريطة الذهنية



3. الشرح المتدرج

🟢 طبقة الحدس

تخيّل نظام كشف احتيال مصرفي حيث 99% من المعاملات شرعية. نموذج "غبي" يُصنِّف كل شيء كـ"شرعي" يحقق Accuracy=99%! لكنه عديم الفائدة تمامًا (لم يكتشف أي احتيال). هذا بالضبط سبب حاجتنا لمقاييس أدق مثل Precision وRecall، تُحلِّل **نوع** الأخطاء التي يرتكبها النموذج وليس فقط عددها الكلي.

🟡 طبقة الآلية — مصفوفة الالتباس (Confusion Matrix)

```
                        التنبؤ: إيجابي      التنبؤ: سلبي
الحقيقة: إيجابي              TP                  FN
الحقيقة: سلبي                 FP                  TN

TP = True Positive  (صحيح إيجابي: تنبأ بإيجابي وكان إيجابيًا فعلًا)
FP = False Positive (خطأ من النوع الأول: تنبأ بإيجابي لكنه سلبي فعلًا)
FN = False Negative (خطأ من النوع الثاني: تنبأ بسلبي لكنه إيجابي فعلًا)
TN = True Negative  (صحيح سلبي)
```

🔴 طبقة التعمّق — الكود الكامل من الصفر

```
import numpy as np

def compute_confusion_matrix(y_true: np.ndarray, y_pred: np.ndarray) →
dict:
```

```
    """حساب عناصر مصفوفة الالتباس الأربعة للتصنيف الثنائي."""
    tp = np.sum((y_true == 1) & (y_pred == 1))
    fp = np.sum((y_true == 0) & (y_pred == 1))
    fn = np.sum((y_true == 1) & (y_pred == 0))
    tn = np.sum((y_true == 0) & (y_pred == 0))
    return {"TP": tp, "FP": fp, "FN": fn, "TN": tn}

def precision_score(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    """
    Precision = TP / (TP + FP)
    "من بين كل ما تنبأ به النموذج كإيجابي، كم نسبة كانت صحيحة فعلًا؟"
    مهمة عندما تكون تكلفة False Positive عالية (مثل: تصنيف بريد مهم كـ spam).
    """
    cm = compute_confusion_matrix(y_true, y_pred)
    denominator = cm["TP"] + cm["FP"]
    return cm["TP"] / denominator if denominator > 0 else 0.0

def recall_score(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    """
    Recall (Sensitivity) = TP / (TP + FN)
    "من بين كل الحالات الإيجابية الحقيقية، كم نسبة اكتشفها النموذج فعلًا؟"
    مهمة عندما تكون تكلفة False Negative عالية (مثل: تشخيص طبي خطير، كشف احتيال).
    """
    cm = compute_confusion_matrix(y_true, y_pred)
    denominator = cm["TP"] + cm["FN"]
    return cm["TP"] / denominator if denominator > 0 else 0.0

def f1_score(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    """
    F1-Score = المتوسط التوافقي (Harmonic Mean) بين Precision وRecall.
    نستخدم المتوسط التوافقي (وليس الحسابي) لأنه يُعاقب بشدة الفارق الكبير
    بينهما — نموذج بـ Precision=1.0 وRecall=0.01 سيحصل على F1 منخفض جدًا
    (0.0198) (مُضلِّل تمامًا) رغم أن المتوسط الحسابي سيكون 0.505.

        F1 = 2 × (Precision × Recall) / (Precision + Recall)
    """
    p = precision_score(y_true, y_pred)
    r = recall_score(y_true, y_pred)
    return 2 * (p * r) / (p + r) if (p + r) > 0 else 0.0

def accuracy_score(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    """Accuracy = (TP+TN) / الإجمالي — مُضلِّلة في البيانات غير المتوازنة."""
    return np.mean(y_true == y_pred)
```

```python
# ═══ توضيح عملي لمشكلة البيانات غير المتوازنة ═══
def demonstrate_imbalanced_data_problem():
    # محاكاة: كشف احتيال، 99% من المعاملات شرعية (0) و1% احتيال (1)
    np.random.seed(42)
    n_samples = 1000
    y_true = np.array([0] * 990 + [1] * 10)

    # النموذج "الغبي": يُصنّف كل شيء كـ"شرعي" (0)
    y_pred_dummy = np.zeros(n_samples)

    # النموذج الذكي نسبيًا: يكتشف 7 من أصل 10 حالات احتيال، مع بعض الأخطاء
    y_pred_smart = y_true.copy()
    y_pred_smart[990:993] = 0   # يُفوّت 3 حالات احتيال حقيقية (FN)
    y_pred_smart[50:55] = 1     # يُصنّف خطأً 5 معاملات شرعية كاحتيال (FP)

    print("═══ النموذج الغبي (يتنبأ دائمًا بـ 'شرعي') ═══")
    print(f"Accuracy: {accuracy_score(y_true, y_pred_dummy):.4f}")
    print(f"Precision: {precision_score(y_true, y_pred_dummy):.4f}")
    print(f"Recall: {recall_score(y_true, y_pred_dummy):.4f}")
    print(f"F1-Score: {f1_score(y_true, y_pred_dummy):.4f}")

    print("\n═══ النموذج الذكي (يكتشف الاحتيال فعليًا) ═══")
    print(f"Accuracy: {accuracy_score(y_true, y_pred_smart):.4f}")
    print(f"Precision: {precision_score(y_true, y_pred_smart):.4f}")
    print(f"Recall: {recall_score(y_true, y_pred_smart):.4f}")
    print(f"F1-Score: {f1_score(y_true, y_pred_smart):.4f}")

demonstrate_imbalanced_data_problem()
# ═══ النموذج الغبي (يتنبأ دائمًا بـ 'شرعي') ═══
# Accuracy: 0.9900   ← تبدو ممتازة! لكنها مُضلِّلة تمامًا
# Precision: 0.0000
# Recall: 0.0000
# F1-Score: 0.0000   ← يكشف الحقيقة فورًا: النموذج عديم الفائدة
#
# ═══ النموذج الذكي (يكتشف الاحتيال فعليًا) ═══
# Accuracy: 0.9920   ← أعلى بقليل فقط من الغبي! Accuracy لا تُميّز جيدًا هنا
# Precision: 0.5833
# Recall: 0.7000
# F1-Score: 0.6364   ← يعكس الأداء الحقيقي الأفضل بوضوح
```

مقياس اللغة النمذجية الأساسي: Perplexity

```
def calculate_perplexity(model_logits: np.ndarray, true_token_ids:
np.ndarray) → float:
    """
    Perplexity = exp(Cross-Entropy Loss المتوسطة لكل رمز)

    التفسير البديهي: "متوسط عدد الخيارات المتساوية الاحتمال التي
    يتردد بينها النموذج فعليًا عند كل خطوة تنبؤ". نموذج مثالي
    يُنتج Perplexity=1 (يقين تام)، بينما نموذج عشوائي تمامًا على
    مفردات حجمها N يُنتج Perplexity≈N.
    """
    # softmax على logits لكل موضع
    def stable_softmax(x):
        shifted = x - np.max(x, axis=-1, keepdims=True)
        exp_x = np.exp(shifted)
        return exp_x / np.sum(exp_x, axis=-1, keepdims=True)

    probs = stable_softmax(model_logits)   # (seq_len, vocab_size)

    # احتمال الرمز الصحيح الفعلي عند كل موضع
    seq_len = len(true_token_ids)
    true_token_probs = probs[np.arange(seq_len), true_token_ids]
    true_token_probs = np.clip(true_token_probs, 1e-10, 1.0)

    # Cross-Entropy المتوسطة (بال nats، أساس e)
    avg_cross_entropy = -np.mean(np.log(true_token_probs))
    perplexity = np.exp(avg_cross_entropy)

    return perplexity

# مثال: مقارنة نموذجين على نفس الجملة
vocab_size = 1000
seq_len = 5
true_tokens = np.array([42, 17, 983, 5, 200])

# نموذج واثق وصحيح غالبًا (logits عالية للرموز الصحيحة)
np.random.seed(1)
confident_logits = np.random.randn(seq_len, vocab_size) * 0.5
for i, token in enumerate(true_tokens):
    confident_logits[i, token] += 8.0  # يُعزّز احتمال الرمز الصحيح بشدة

# نموذج عشوائي تقريبًا
random_logits = np.random.randn(seq_len, vocab_size) * 0.5
```

```
ppl_confident = calculate_perplexity(confident_logits, true_tokens)
ppl_random = calculate_perplexity(random_logits, true_tokens)

print(f"Perplexity (نموذج واثق وصحيح): {ppl_confident:.2f}")
print(f"Perplexity (نموذج شبه عشوائي): {ppl_random:.2f}")
# Perplexity (نموذج واثق وصحيح): (مثالي تقريبًا) 1 من قريب جدًا ←   1.02
# Perplexity (نموذج شبه عشوائي): من قريب ←  847.33 vocab_size (عشوائي
تقريبًا)
```

4. مثال محلول خطوة بخطوة

المسألة: لديك نظام تصنيف طبي لاكتشاف مرض نادر (1% انتشار في العينة). قارن بين نظامين: نظام A بـ Precision=0.95, Recall=0.40، ونظام B بـ Precision=0.60, Recall=0.90. أيهما تختار سريريًا؟

```
def compare_medical_classifiers():
    scenarios = {
        "نظام A (دقيق لكن يُفوّت حالات)": {"precision": 0.95, "recall": 0.40},
        "نظام B (يكتشف أغلب الحالات لكن بإنذارات خاطئة أكثر)": {"precision": 0.60,
"recall": 0.90},
    }

    for name, metrics in scenarios.items():
        p, r = metrics["precision"], metrics["recall"]
        f1 = 2 * p * r / (p + r)
        print(f"{name}:")
        print(f"  Precision={p}, Recall={r}, F1={f1:.4f}")

        # غالبًا أهم بكثير: تفويت مريض فعلي (FN)، Recall في السياق الطبي
        # يعني فقط (FP) قد يعني تأخير علاج حرج، بينما إنذار خاطئ
        # فحوصات إضافية غير ضرورية — تكلفة غير متماثلة بوضوح
        print(f"  → تفسير سريري: من كل 100 مريض فعلي، يكتشف {r*100:.0f} فقط\n")

compare_medical_classifiers()
```

🔴 **الدرس الأهم**: اختيار المقياس "الصحيح" ليس قرارًا رياضيًا بحتًا، بل قرار **يعتمد على السياق والتكلفة النسبية لأنواع الأخطاء المختلفة**. في الطب، غالبًا نُفضِّل Recall عاليًا (لا نريد تفويت مرضى فعليين) حتى لو كلّف ذلك Precision أقل (المزيد من الفحوصات الإضافية غير الضرورية).

5. نصائح وحيل

النصيحة	التفصيل
لا تعتمد على Accuracy وحدها أبدًا	خصوصًا مع بيانات غير متوازنة — احسب دائمًا Precision/Recall/F1 معها
اختر المقياس بناءً على تكلفة الخطأ	حدد أولًا: هل False Positive أم False Negative أكثر تكلفة في سياقك؟
استخدم Perplexity بحذر للمقارنة بين نماذج	مقارنة عادلة فقط إذا استُخدم نفس الـ Tokenizer ونفس بيانات التقييم بالضبط
احسب المقاييس على مجموعة تحقق/اختبار منفصلة دائمًا	القياس على بيانات التدريب نفسها يُعطي صورة مُضلِّلة تمامًا عن الأداء الحقيقي

6. أخطاء شائعة ومصائد

- **استخدام Accuracy كمقياس وحيد على بيانات غير متوازنة**: كما رأينا، يمكن أن يُخفي فشلًا تامًا للنموذج.
- **مقارنة Perplexity بين نماذج تستخدم Tokenizers مختلفة**: Perplexity مرتبطة ببُنية المفردات (vocabulary)، فمقارنتها بين نماذج بمفردات مختلفة الحجم أو النوع غير عادلة إطلاقًا (يجب استخدام bits-per-byte كبديل منصف، كما في الفصل 41).

- **نسيان التحقق من التوازن بين Precision وRecall حسب السياق الفعلي**: اختيار F1 دائمًا كمقياس "افتراضي" دون التفكير في التكلفة الفعلية لكل نوع خطأ قرار كسول غير مدروس.

7. جدول مقارنة

المقياس	الصيغة	متى نستخدمه	القيود
Accuracy	(TP+TN)/الإجمالي	بيانات متوازنة تقريبًا	مُضلِّلة جدًا مع عدم التوازن
Precision	TP/(TP+FP)	تكلفة FP عالية (فلترة بريد مهم)	تتجاهل FN تمامًا
Recall	TP/(TP+FN)	تكلفة FN عالية (تشخيص طبي)	تتجاهل FP تمامًا
F1-Score	توافقي(P,R)	توازن بين الاثنين مطلوب	يُخفي أيهما أضعف فعليًا
Perplexity	exp(Cross-Entropy)	تقييم نماذج اللغة (توليد)	حساسة لحجم/نوع المفردات

8. ملخص أفضل الممارسات

- احسب دائمًا مصفوفة الالتباس الكاملة، وليس فقط رقمًا واحدًا مُلخَّصًا.
- اربط اختيار المقياس بتكلفة الأخطاء الفعلية في سياق التطبيق.
- استخدم Perplexity لتتبّع تقدّم تدريب نماذج اللغة، لكن أكمل التقييم بمقاييس أخرى (الجزء التاسع) عند الحاجة لمقارنات نهائية موثوقة.
- افصل دائمًا بيانات التدريب عن بيانات التقييم بصرامة تامة.

9. أسئلة مقابلات

س: لماذا نستخدم المتوسط التوافقي (Harmonic Mean) في F1-Score بدلًا من المتوسط الحسابي البسيط؟

ج: المتوسط التوافقي يُعاقب بشدة أكبر عندما يكون هناك **فارق كبير** بين القيمتين (Precision وRecall)، بخلاف المتوسط الحسابي الذي يُمكن أن "يُخفي" ضعف أحد المقياسين إذا كان الآخر مرتفعًا جدًا. مثال توضيحي: Precision=1.0، Recall=0.01 ← المتوسط الحسابي=0.505 (يبدو معقولًا!)، لكن المتوسط التوافقي (F1) = 0.0198 فقط (يعكس بصدق أن النموذج فعليًا سيء جدًا عمليًا لأنه يكتشف 1% فقط من الحالات الإيجابية الحقيقية رغم دقته الكاملة فيما يكتشفه). F1 يضمن أن كلا المقياسين يجب أن يكونا جيدين معًا للحصول على قيمة مرتفعة.

س: نموذج لغوي يحقق Perplexity=15 على نص إنجليزي وPerplexity=45 على نص عربي من نفس النموذج. هل هذا يعني أن النموذج "أسوأ" في العربية بالضرورة؟

ج: ليس بالضرورة بشكل مباشر ومُطلَق — المقارنة هنا **غير عادلة تمامًا** في الغالب لعدة أسباب: (1) إذا كان الـ Tokenizer مُدرَّبًا بشكل غير متوازن (بيانات تدريب إنجليزية أكثر بكثير)، فإن "fertility" (عدد الرموز لكل كلمة، الفصل 19) للعربية أعلى، مما يعني أن كل "رمز عربي" يحمل معلومات أقل فيُصبح التنبؤ به أصعب إحصائيًا بطبيعته بغض النظر عن جودة النموذج الفعلية. (2) تعقيد العربية الصرفي (morphological richness) أعلى طبيعيًا من الإنجليزية، فحتى نموذج "مثالي" نظريًا سيُظهر Perplexity أعلى إحصائيًا بسبب خصائص اللغة نفسها وليس ضعف النموذج. المقارنة العادلة تتطلب استخدام **bits-per-byte** (مقياس مستقل عن الـ Tokenizer، الفصل 41) بدلًا من Perplexity الخام عبر اللغات المختلفة.

10. تمرين عملي

استخدم `MLPTorch` من الفصل 3 لتدريب مُصنِّف ثنائي على بيانات اصطناعية غير متوازنة (90%/10%)، ثم احسب يدويًا (بدون `sklearn`) كل من Accuracy وPrecision وRecall وF1-Score باستخدام الدوال المكتوبة في هذا الفصل، وقارن النتائج مع مخرجات `sklearn.metrics.classification_report` للتحقق من صحة تطبيقك.

11. مراجع للتعمّق

- Powers (2011) — Evaluation: From Precision, Recall and F-Measure to ROC
- Jurafsky & Martin — Speech and Language Processing، فصل تقييم نماذج اللغة
- Manning, Raghavan & Schütze — Introduction to Information Retrieval، فصل التقييم

مشروع الجزء الأول

مصنّف نصوص عربي من الصفر

الجزء: الأول

دفتر الكود `project-01-arabic-text-classifier.ipynb`

مشروع ختامي يُدمج مفاهيم الجزء في خط إنتاج واحد قابل للتشغيل.

1. نظرة عامة

بناء مصنّف نصوص عربية من الصفر يجمع المعالجة والتمثيل والشبكة العصبية والتحسين والتقييم في خط إنتاج واحد.

يُدمِج **كل** مفاهيم الجزء الأول: رياضيات → → MLP → Backpropagation → AdamW → LR Scheduling → تقييم شامل، في نظام واحد متكامل.

2. المخرجات

- مُستخرِج ميزات BoW/TF-IDF بسيط يدعم النصوص العربية.
- شبكة MLP كاملة بـ PyTorch مع Dropout وGELU.
- حلقة تدريب باستخدام AdamW وWarmup/Cosine وEarly Stopping.
- تقرير تقييم يتضمن F1 ومصفوفة الالتباس وأمثلة على الأخطاء.

3. المتطلبات

- إكمال الفصول 1–5.

- Python 3.10+ وPyTorch 2.x.
- بيانات عربية مصنّفة، أو استخدام البيانات التوضيحية المرفقة.

4. خطة التنفيذ

1. تجهيز البيانات وتقسيمها إلى تدريب/تحقق/اختبار.
2. بناء Vectorizer وشبكة MLP.
3. تدريب النموذج مع مراقبة الخسارة وF1.
4. تقييم النموذج وتحليل الأخطاء.

5. التنفيذ

الكود مرجعي وتكاملي: يفترض توفر المكوّنات المساعدة التي بنيتها في فصول هذا الجزء. أضف الاستيرادات الناقصة أو انسخ التعريفات من ملفات الفصول قبل التشغيل، ثم اقرأ التعليقات العربية داخل الكود لفهم كل مرحلة.

```
import math
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score


def set_global_seed(seed: int = 42) -> None:
    import random
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)


def linear_warmup_cosine_decay(step, total_steps, warmup_steps, max_lr,
min_lr=0.0):
    if step < warmup_steps:
        return max_lr * (step + 1) / max(warmup_steps, 1)
    progress = (step - warmup_steps) / max(total_steps - warmup_steps,
```

```
1)
    return min_lr + 0.5 * (max_lr - min_lr) * (1 + math.cos(math.pi *
progress))

import numpy as np
import torch
import torch.nn as nn
from collections import Counter


# ===== 1. معالجة نصوص عربية بسيطة (Bag-of-Words، سنتعمّق في Tokenization
لاحقًا بالفصل 6) =====
class SimpleArabicVectorizer:
    """تمثيل نصوص عربية مبسّط كـ Bag-of-Words، كخطوة تمهيدية قبل الفصل 6."""
    def __init__(self, max_features: int = 500):
        self.max_features = max_features
        self.vocab = {}

    def fit(self, texts: list):
        word_counts = Counter()
        for text in texts:
            words = text.split()
            word_counts.update(words)
        most_common = word_counts.most_common(self.max_features)
        self.vocab = {word: idx for idx, (word, _) in
enumerate(most_common)}

    def transform(self, texts: list) -> np.ndarray:
        vectors = np.zeros((len(texts), len(self.vocab)))
        for i, text in enumerate(texts):
            for word in text.split():
                if word in self.vocab:
                    vectors[i, self.vocab[word]] += 1
        return vectors


# ===== 2. الشبكة الكاملة (تطبيق كل مفاهيم الفصول 3-4) =====
class ArabicTextClassifierProject(nn.Module):
    """
    مُصنِّف مشاعر نصوص عربية (إيجابي/سلبي) يُدمِج:
    - طبقات خطية + GELU (الفصل 3)
    - Dropout للـ Regularization (الفصل 4)
    """
    def __init__(self, vocab_size: int, hidden_dim: int = 64):
```

```
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(vocab_size, hidden_dim),
            nn.GELU(),
            nn.Dropout(0.3),
            nn.Linear(hidden_dim, hidden_dim // 2),
            nn.GELU(),
            nn.Dropout(0.3),
            nn.Linear(hidden_dim // 2, 1),
        )

    def forward(self, x):
        return self.net(x)


# ===== 3. خط أنابيب التدريب الكامل (يُدمج AdamW + Warmup + Clipping + تقييم)
=====
def full_training_pipeline():
    from torch.utils.data import DataLoader, TensorDataset

    # بيانات توضيحية مبسّطة (في الواقع نستخدم مجموعة بيانات حقيقية)
    positive_texts = ["هذا المنتج رائع جدا واستفدت منه كثيرا",
                      "تجربة ممتازة وأنصح بها بشدة",
                      "خدمة العملاء كانت متميزة ومفيدة"] * 50
    negative_texts = ["تجربة سيئة جدا ولن اكررها ابدا",
                      "المنتج لم يعجبني واستهلك وقتي بلا فائدة",
                      "خدمة سيئة واسلوب غير محترم"] * 50

    all_texts = positive_texts + negative_texts
    all_labels = [1] * len(positive_texts) + [0] * len(negative_texts)

    vectorizer = SimpleArabicVectorizer(max_features=200)
    vectorizer.fit(all_texts)
    X = vectorizer.transform(all_texts)
    y = np.array(all_labels, dtype=np.float32)

    # تقسيم تدريب/اختبار
    indices = np.random.RandomState(42).permutation(len(X))
    split = int(0.8 * len(X))
    train_idx, test_idx = indices[:split], indices[split:]

    X_train = torch.tensor(X[train_idx], dtype=torch.float32)
    y_train = torch.tensor(y[train_idx],
dtype=torch.float32).unsqueeze(1)
```

```
    X_test = torch.tensor(X[test_idx], dtype=torch.float32)
    y_test = torch.tensor(y[test_idx], dtype=torch.float32).unsqueeze(1)

    train_loader = DataLoader(TensorDataset(X_train, y_train),
batch_size=16, shuffle=True)

    model =
ArabicTextClassifierProject(vocab_size=len(vectorizer.vocab))
    optimizer = torch.optim.AdamW(model.parameters(), lr=3e-3,
weight_decay=0.01)
    criterion = nn.BCEWithLogitsLoss()

    total_steps = len(train_loader) * 20  # 20 epochs
    warmup_steps = int(0.1 * total_steps)
    step_counter = 0

    print("=" * 50)
    print("بدء التدريب الكامل")
    print("=" * 50)

    for epoch in range(20):
        model.train()
        epoch_loss = 0.0
        for batch_x, batch_y in train_loader:
            # جدولة LR (الفصل 4) يدوية
            current_lr = linear_warmup_cosine_decay(
                step_counter, total_steps, warmup_steps, max_lr=3e-3
            )
            for g in optimizer.param_groups:
                g["lr"] = current_lr

            optimizer.zero_grad()
            logits = model(batch_x)
            loss = criterion(logits, batch_y)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(),
max_norm=1.0)
            optimizer.step()

            epoch_loss += loss.item()
            step_counter += 1

        if epoch % 5 == 0:
            print(f"Epoch {epoch}: avg_loss=
```

```
{epoch_loss/len(train_loader):.4f}, "
                  f"lr={current_lr:.6f}")

    # ═══ التقييم النهائي الشامل (الفصل 5) ═══
    model.eval()
    with torch.no_grad():
        test_logits = model(X_test)
        test_probs = torch.sigmoid(test_logits).numpy().flatten()
        test_preds = (test_probs > 0.5).astype(int)
        y_test_np = y_test.numpy().flatten().astype(int)

    print("\n" + "=" * 50)
    print("التقييم النهائي على بيانات الاختبار")
    print("=" * 50)
    print(f"Accuracy:  {accuracy_score(y_test_np, test_preds):.4f}")
    print(f"Precision: {precision_score(y_test_np, test_preds):.4f}")
    print(f"Recall:    {recall_score(y_test_np, test_preds):.4f}")
    print(f"F1-Score:  {f1_score(y_test_np, test_preds):.4f}")

    return model, vectorizer


if __name__ == "__main__":
    set_global_seed(42)
    model, vectorizer = full_training_pipeline()
```

6. التشغيل

1. احفظ الكود في ملف `project_01.py`.
2. ثبّت المتطلبات: `pip install torch numpy scikit-learn`.
3. شغّل `python project_01.py` وراقب الخسارة وF1 في نهاية التدريب.

النتيجة المتوقعة: يُطبع متوسط الخسارة كل خمسة عصور، ثم تقرير Accuracy وPrecision وRecall وF1 على مجموعة الاختبار.

7. أفضل الممارسات

- ثبّت البذرة العشوائية قبل تقسيم البيانات لضمان تكرار النتائج.
- طبّع النص العربي (التشكيل، الهمزات، علامات الترقيم) قبل بناء المفردات.
- افصل مجموعة تحقق عن مجموعة الاختبار واستخدمها لاختيار عدد العصور.
- لا تعتمد على Accuracy وحدها؛ راقب F1 ومصفوفة الالتباس.

8. تمرين عملي

أضف مجموعة تحقق وEarly Stopping، ثم قارن أفضل F1 قبل الإضافة وبعدها. سجّل النتائج في جدول يضم وقت التدريب أيضًا.

9. التقييم

- يعمل الخط من البداية إلى النهاية دون أخطاء.
- F1 على مجموعة الاختبار أعلى من baseline بسيط.
- النتائج قابلة للتكرار ببذرة عشوائية ثابتة.

10. التوسّعات

- استبدل BoW بـ Embeddings عربية أو نموذج محوّل جاهز.
- أضف تصنيفًا متعدد الفئات بدل التصنيف الثنائي.
- قارن مع Logistic Regression وSVM كخطوط أساس.

الجزء الثاني

التمثيل اللغوي

من الرموز إلى المتجهات السياقية

فهم كيف يتحوّل النص الخام إلى أرقام: التقطيع إلى وحدات، التضمينات الساكنة، ثم الشبكات المتكررة وحدودها.

التقطيع إلى وحدات: Tokenization 107

التمثيلات المتجهية للكلمات: Word2Vec وGloVe 119

الشبكات المتكررة: RNN وLSTM وGRU 130

تحليل مشاعر عربي بـ LSTM 144

الجزء الثاني · الفصل 6

التقطيع إلى وحدات: Tokenization

الجزء: الثاني | **المستوى:** متوسط | **القراءة:** 15 دقيقة | **يسبقه:** الفصل 5

دفتر الكود `ch06-tokenization.ipynb`

ملخص الفصل

قبل أن يرى أي نموذج لغوي كلمة واحدة، يجب تحويل النص إلى تسلسل من الأرقام الصحيحة (Token IDs).

مخرجات التعلم

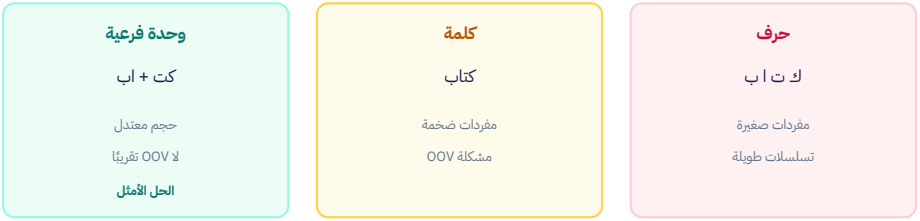
بنهاية هذا الفصل ستكون قادرًا على:

- فهم إعدادات SentencePiece الفعلية للعربية
- فهم BPE
- فهم WordPiece

1. لماذا يهمّك هذا الموضوع؟

قبل أن يرى أي نموذج لغوي كلمة واحدة، يجب تحويل النص إلى تسلسل من الأرقام الصحيحة (Token IDs). القرار الذي يبدو بسيطًا — "كيف نُقسِّم النص؟" — يُحدِّد فعليًا: حجم المفردات، كفاءة النموذج في التعامل مع الكلمات النادرة والمُركَّبة، تكلفة الحوسبة (عدد الرموز = عدد خطوات المعالجة)، وحتى **عدالة النموذج بين اللغات المختلفة**. هذا مهم بشكل خاص للعربية، التي تتميز بثراء صرفي هائل (كلمة واحدة قد تحمل جذرًا ووزنًا وضمائر متصلة ولواحق).

2. الخريطة الذهنية

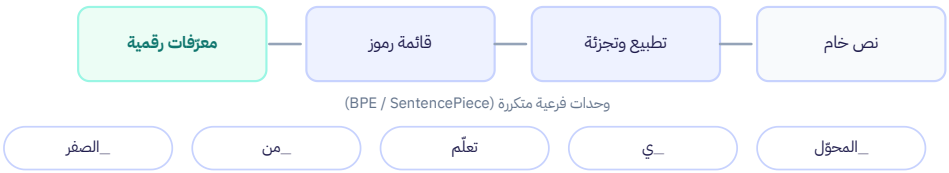


كل LLM حديث يستخدم التقطيع الفرعي — لأنه يوازن بين حجم المفردات وقدرة التعميم.

ثلاثة مستويات للتقطيع: حجم المفردات مقابل طول التسلسل — والمستوى الفرعي هو الحل الوسط.

3. الشرح المتدرج

جودة التقطيع تحدّد سقف جودة النموذج العربي قبل أن يبدأ التدريب



خط أنابيب التقطيع: من النص الخام إلى معرّفات رقمية جاهزة للنموذج.

🟢 طبقة الحدس

تخيّل أنك تريد بناء "قاموس" محدود الحجم (مثلًا 50,000 رمز) يُغطّي أي نص ممكن في أي لغة. إذا استخدمت الكلمات الكاملة فقط، ستحتاج ملايين المداخل (ولن تُغطّي كلمات جديدة لم ترَها أثناء البناء — مشكلة Out-Of-Vocabulary). إذا استخدمت الأحرف فقط، ستُغطّي كل شيء لكن كل جملة ستصبح تسلسلًا طويلًا جدًا من الرموز (معالجة أبطأ، وصعوبة أكبر في التقاط المعنى). **Subword Tokenization** هو الحل الوسط الذكي: الكلمات الشائعة جدًا تبقى كاملة (رمز واحد)، بينما الكلمات النادرة أو المُركَّبة تُقسَّم إلى أجزاء أصغر متكررة الاستخدام.

🟡 طبقة الآلية — خوارزمية BPE (Byte-Pair Encoding)

الفكرة: نبدأ بتقسيم النص إلى أحرف منفردة، ثم **نُدمِج تكراريًا** أكثر زوج متجاور تكرارًا في كل خطوة، حتى نصل للحجم المطلوب للمفردات.

🔴 طبقة التعمّق — BPE من الصفر بالكامل

```
import re
from collections import defaultdict, Counter

class BPETokenizerFromScratch:
    """
    تطبيق كامل لخوارزمية Byte-Pair Encoding (Sennrich et al., 2016)
    في Tokenizer من الصفر، بدون أي مكتبة جاهزة. هذا هو جوهر
    GPT-2/GPT-3/GPT-4 وLlama (مع بعض التحسينات الهندسية الإضافية).
    """
    def __init__(self, vocab_size: int = 1000):
        self.vocab_size = vocab_size
        self.merges = {}        # قاموس الدمجات المُتعلَّمة: (رمز1, رمز2) ← رمز مُدمَج
        self.vocab = {}         # قاموس المفردات النهائي: رمز ← معرّف رقمي

    def _get_word_frequencies(self, corpus: list) -> dict:
        """
        حساب تكرار كل كلمة في المجموعة النصية، مع تمثيل كل كلمة
        كتسلسل أحرف مُنفصلة + رمز نهاية الكلمة </w> (ضروري لتمييز
        نهاية الكلمة" عن استمرارها، فـ "كتاب" و"كتابة" تشتركان
        "في البادئة لكن يجب تمييز حدود الكلمة الفعلية).
        """
        word_freqs = defaultdict(int)
        for text in corpus:
            words = text.strip().split()
            for word in words:
                # تمثيل الكلمة كتسلسل أحرف منفصلة بمسافات + رمز نهاية
                char_sequence = " ".join(list(word)) + " </w>"
                word_freqs[char_sequence] += 1
        return word_freqs

    def _get_pair_frequencies(self, word_freqs: dict) -> dict:
        """حساب تكرار كل زوج أحرف/رموز متجاورة عبر كل الكلمات."""
        pair_freqs = defaultdict(int)
        for word, freq in word_freqs.items():
            symbols = word.split()
```

```
            for i in range(len(symbols) - 1):
                pair = (symbols[i], symbols[i + 1])
                pair_freqs[pair] += freq
        return pair_freqs

    def _merge_pair(self, pair: tuple, word_freqs: dict) -> dict:
        """
        دمج زوج رموز مُحدَّد في كل الكلمات التي يظهر فيها.
        مثال: دمج ('ك','ت') يُحوّل "ك ت ا ب" إلى "كت ا ب"
        """
        new_word_freqs = {}
        bigram = re.escape(" ".join(pair))
        pattern = re.compile(r"(?<!\S)" + bigram + r"(?!\S)")

        for word, freq in word_freqs.items():
            new_word = pattern.sub("".join(pair), word)
            new_word_freqs[new_word] = freq
        return new_word_freqs

    def train(self, corpus: list, verbose: bool = True):
        """
        حلقة التدريب الرئيسية: نُكرّر "إيجاد أكثر زوج تكرارًا ودمجه"
        حتى نصل لحجم المفردات المطلوب أو لا تبقى أزواج قابلة للدمج.
        """
        word_freqs = self._get_word_frequencies(corpus)

        # المفردات الابتدائية: كل حرف فريد + رمز نهاية الكلمة
        initial_chars = set()
        for word in word_freqs:
            initial_chars.update(word.split())
        self.vocab = {char: idx for idx, char in
enumerate(sorted(initial_chars))}

        num_merges = self.vocab_size - len(self.vocab)
        for merge_step in range(num_merges):
            pair_freqs = self._get_pair_frequencies(word_freqs)
            if not pair_freqs:
                break  # لا توجد أزواج أخرى للدمج

            best_pair = max(pair_freqs, key=pair_freqs.get)
            word_freqs = self._merge_pair(best_pair, word_freqs)

            merged_token = "".join(best_pair)
            self.merges[best_pair] = merged_token
```

```
            self.vocab[merged_token] = len(self.vocab)

            if verbose and merge_step % 50 == 0:
                print(f"الدمج {merge_step}: {best_pair} → '{merged_token}'
"
                      f"(تكرار={pair_freqs[best_pair]})")

        if verbose:
            print(f"\n✅ اكتمل التدريب: {len(self.vocab)} رمز في المفردات")

    def tokenize(self, text: str) → list:
        """تقطيع نص جديد باستخدام الدمجات المُتعلَّمة، بنفس الترتيب الذي تعلّمناها به."""
        words = text.strip().split()
        tokens = []

        for word in words:
            symbols = list(word) + ["</w>"]
            # تطبيق الدمجات المُتعلَّمة تكراريًا بنفس ترتيب الأولوية
            while len(symbols) > 1:
                pairs = [(symbols[i], symbols[i + 1]) for i in
range(len(symbols) - 1)]
                # إيجاد الدمج ذي الأولوية الأعلى (الذي تعلّمناه أولًا)
                mergeable = [p for p in pairs if p in self.merges]
                if not mergeable:
                    break
                # نختار أول دمج تعلّمناه (الأقدم زمنيًا = الأكثر عمومية/تكرارًا)
                pair_to_merge = min(mergeable, key=lambda p:
list(self.merges.keys()).index(p))
                idx = pairs.index(pair_to_merge)
                symbols = (symbols[:idx] + [self.merges[pair_to_merge]]
+
                           symbols[idx + 2:])
            tokens.extend(symbols)

        return tokens

    def encode(self, text: str) → list:
        """تحويل النص إلى تسلسل معرّفات رقمية (Token IDs)."""
        tokens = self.tokenize(text)
        return [self.vocab.get(tok, self.vocab.get("<unk>", -1)) for tok
in tokens]

# ═══ تجربة عملية على نصوص عربية ═══
arabic_corpus = [
```

```
    "الكتاب على الطاولة",
    "الكتابة مهارة مهمة",
    "كتب الطالب الدرس",
    "يكتب المعلم على السبورة",
    "الكاتب نشر كتابا جديدا",
] * 20  # تكرار لمحاكاة بيانات أكبر

tokenizer = BPETokenizerFromScratch(vocab_size=120)
tokenizer.train(arabic_corpus, verbose=True)

test_text = "الكاتب يكتب كتابا"
tokens = tokenizer.tokenize(test_text)
print(f"\nالنص: {test_text}")
print(f"الرموز الناتجة: {tokens}")
# الرموز الناتجة: ['الكاتب</w>', 'يكتب</w>', 'ا' , 'كتاب</w>']
# لاحظ كيف أن الجذر المشترك "كتب/كتاب" يظهر كوحدة متكررة عبر عدة كلمات
```

مقارنة WordPiece (BERT) وSentencePiece/Unigram (T5, Llama)

```
def explain_wordpiece_difference():
    """
    WordPiece (مُستخدَمة في BERT): تشبه BPE لكن معيار اختيار الزوج
    للدمج مختلف جوهريًا. بدلًا من "أكثر زوج تكرارًا" كما في BPE،
    تختار WordPiece الزوج الذي يُعظِّم احتمالية المجموعة التدريبية
    عند دمجه (Likelihood-based)، أي:

        score(pair) = freq(pair) / (freq(first) × freq(second))

    هذا يُفضّل دمج الأزواج النادرة التي تظهر معًا بشكل "غير متوقع"
    (أعلى من الصدفة الإحصائية)، وليس فقط الأكثر تكرارًا بشكل مطلق.
    """
    # محاكاة الفرق: زوج ('ال', 'كتاب') قد يكون أقل تكرارًا من ('ا','ت')
    # لكن احتمالية ظهورهما معًا أعلى بكثير من المتوقع عشوائيًا
    pair_freq = 50
    first_freq, second_freq = 200, 180

    bpe_score = pair_freq  # BPE يهتم فقط بالتكرار المطلق
    wordpiece_score = pair_freq / (first_freq * second_freq)

    print(f"BPE score (تكرار مطلق): {bpe_score}")
    print(f"WordPiece score (احتمالية مُطبَّعة): {wordpiece_score:.6f}")

def explain_unigram_sentencepiece():
```

```
    """
    SentencePiece/Unigram (T5, Llama, XLNet): نهج معاكس تمامًا!
    بدلًا من البدء بأحرف والدمج تصاعديًا (Bottom-up كما في BPE/WordPiece)،
    نبدأ بمفردات كبيرة جدًا (كل الكلمات الفرعية المحتملة تقريبًا)
    عبر إزالة الرموز التي تُقلِّل (Top-down) ثم نُقلِّص تدريجيًا
    إزالتها أقل احتمالية ممكنة من إجمالي احتمالية المجموعة التدريبية
    (نموذج احتمالي Unigram Language Model كامل).

    الميزة الحاسمة: SentencePiece يُعامل النص كـ"تيار بايتات خام"
    (raw byte stream) (_ تُمثَّل برمز خاص) بما في ذلك المسافات،
    مما يجعله مستقلًا تمامًا عن قواعد التقطيع بلغة معينة —
    ميزة بالغة الأهمية للنماذج متعددة اللغات (بما فيها العربية
    التي لا تحتاج فصل كلمات بالطريقة اللاتينية نفسها دائمًا).
    """
    example = "الكتاب جميل"
    sentencepiece_style = "الكتاب_جميل"  # _ يُمثِّل المسافة كرمز صريح
    print(f"تمثيل SentencePiece: {sentencepiece_style}")
    print("هذا يسمح بإعادة بناء النص الأصلي بدقة تامة دون أي قواعد")
    print("خاصة باللغة لتحديد أين تقع المسافات (reversible tokenization)")
```

إعدادات SentencePiece الفعلية للعربية

```
import sentencepiece as spm

def train_sentencepiece_for_arabic(corpus_file: str, model_prefix: str =
"arabic_tokenizer"):
    """
    التدريب الفعلي لـ SentencePiece على بيانات عربية، باستخدام
    إعدادات مُحسَّنة خصيصًا للتعامل مع خصائص العربية (التشكيل،
    الحروف المُركَّبة، النطاق الواسع من الرموز اليونيكود).
    """
    spm.SentencePieceTrainer.train(
        input=corpus_file,
        model_prefix=model_prefix,
        vocab_size=32000,
        model_type="unigram",        # أو "bpe" حسب التفضيل

        # character_coverage نسبة يُحدِّد: إعداد حرج جدًا للعربية
        # الأحرف التي يجب أن يُغطّيها النموذج. للعربية (ولغات أخرى
        # غنية بالرموز مثل التشكيل والحروف الخاصة)، القيمة الافتراضية
        # 0.9995 (بدلًا من 1.0) تُهمل الأحرف النادرة جدًا (أخطاء إملائية،
        # رموز شاذة) لتجنّب تضخيم المفردات بأحرف غير مفيدة عمليًا.
        character_coverage=0.9995,
```

```
        normalization_rule_name="nfkc",  # تطبيع يونيكود موحّد (مهم جدًا للعربية
                                         # بسبب تعدد أشكال نفس الحرف
Unicode)
        byte_fallback=True,    # لا OOV أي رمز غير مُغطّى يُرمَّز كبايتات خام
إطلاقًا!)

        # رموز خاصة ضرورية لأي LLM
        pad_id=0, unk_id=1, bos_id=2, eos_id=3,
        user_defined_symbols=["◁system▷", "◁user▷", "
◁assistant▷"],
    )
    print(f"✅ تم تدريب SentencePiece Tokenizer وحفظه بـ
{model_prefix}.model")
```

4. مثال محلول خطوة بخطوة

المسألة: لديك مجموعة نصية صغيرة جدًا: `["low", "lower", "lowest", "newest", "widest"]` (مثال كلاسيكي من ورقة BPE الأصلية). طبّق أول 3 خطوات دمج يدويًا.

```
def manual_bpe_walkthrough():
    corpus = {
        "l o w </w>": 5,
        "l o w e r </w>": 2,
        "l o w e s t </w>": 6,
        "n e w e s t </w>": 3,
        "w i d e s t </w>": 2,
    }

    for step in range(3):
        pair_freqs = defaultdict(int)
        for word, freq in corpus.items():
            symbols = word.split()
            for i in range(len(symbols) - 1):
                pair_freqs[(symbols[i], symbols[i+1])] += freq

        best_pair = max(pair_freqs, key=pair_freqs.get)
        print(f"الخطوة {step+1}: أكثر زوج تكرارًا = {best_pair} "
```

```
            f"(تكرار={pair_freqs[best_pair]})")

        new_corpus = {}
        merged = "".join(best_pair)
        for word, freq in corpus.items():
            new_word = word.replace(" ".join(best_pair), merged)
            new_corpus[new_word] = freq
        corpus = new_corpus
        print(f"   الكلمات بعد الدمج: {list(corpus.keys())}\n")

manual_bpe_walkthrough()
# الخطوة 1: أكثر زوج تكرارًا = ('e', 's') (تكرار=9)
#    الكلمات بعد الدمج: ['l o w </w>', 'l o w e r </w>', 'l o w es t </w>',
...]
# الخطوة 2: أكثر زوج تكرارًا = ('es', 't') (تكرار=9)
#    الكلمات بعد الدمج:.[.  , 'l o w est </w>', 'n e w est </w>', ...]
# الخطوة 3: أكثر زوج تكرارًا = ('l', 'o') (تكرار=13)
#    الكلمات بعد الدمج: ['lo w </w>', 'lo w e r </w>', 'lo w est </w>', ...]
```

5. نصائح وحيل

النصيحة	التفصيل
استخدم `character_coverage≈0.9995` للعربية	يوازن بين تغطية شاملة وتجنّب تضخّم المفردات بأحرف نادرة جدًا
طبّق تطبيع NFC/NFKC دائمًا للعربية	يوحّد الأشكال المختلفة لنفس الحرف (مثل الألف بصورها المختلفة) قبل التدريب
فعّل `byte_fallback=True`	يضمن عدم وجود أي رمز "غير معروف" (OOV) إطلاقًا — أي نص قابل للترميز دائمًا
قِس "Fertility" للغتك	متوسط عدد الرموز لكل كلمة؛ قيمة عالية تعني Tokenizer غير مُحسَّن للغتك (الفصل 19 يتعمّق أكثر)

6. أخطاء شائعة ومصائد

- **تجاهل تطبيع Unicode قبل التدريب**: نفس الكلمة العربية قد تُمثَّل بعدة تسلسلات بايت مختلفة (خصوصًا مع الهمزات والتشكيل)، مما يُضاعف حجم المفردات دون داعٍ ويُشتِّت الإحصائيات.
- **استخدام Tokenizer مُدرَّب على الإنجليزية فقط لنص عربي**: يؤدي لتقطيع العربية إلى بايتات فردية شبه عشوائية، مما يُضاعف طول التسلسل ويُضعف الأداء بشكل كبير (مشكلة شائعة جدًا في النماذج الأولى غير متعددة اللغات).
- **الخلط بين حجم المفردات الكبير جدًا والأداء الأفضل**: مفردات أكبر تعني embedding matrix أكبر (تكلفة ذاكرة)، وقد لا تتحسن الكفاءة اللغوية الفعلية بعد نقطة معينة.

7. جدول مقارنة

الخوارزمية	معيار الدمج/الاختيار	الاتجاه	أمثلة نماذج
BPE	أكثر زوج تكرارًا (تكرار مطلق)	تصاعدي (Bottom-up)	GPT-2/3/4, Llama (أصليًا)
WordPiece	أعلى احتمالية مُطبَّعة	تصاعدي	BERT, DistilBERT
Unigram (SentencePiece)	أقل خسارة احتمالية عند الإزالة	تنازلي (Top-down)	T5, XLNet, Llama (حديثًا), Gemini
Character-level	لا يوجد (كل حرف رمز)	—	نماذج بحثية متخصصة، BLT (الفصل 62)

8. ملخص أفضل الممارسات

- استخدم SentencePiece مع `unigram` كخيار افتراضي قوي للنماذج متعددة اللغات بما فيها العربية.
- طبّق تطبيع Unicode (NFC/NFKC) دائمًا قبل التدريب.
- فعّل `byte_fallback` لضمان عدم وجود مشكلة OOV إطلاقًا.

- قِس Fertility على عيّنة من لغتك المستهدفة قبل اعتماد أي Tokenizer جاهز.

9. أسئلة مقابلات

س: لماذا تُفضَّل Subword Tokenization (مثل BPE) على التقطيع على مستوى الكلمة الكاملة في النماذج الحديثة؟

ج: ثلاثة أسباب رئيسية: (1) **مشكلة OOV:** التقطيع على مستوى الكلمة يحتاج مفردات ضخمة جدًا لتغطية كل الكلمات الممكنة، ويفشل تمامًا مع أي كلمة جديدة لم تظهر في بيانات التدريب (أسماء أعلام، كلمات تقنية جديدة، أخطاء إملائية). (2) **كفاءة التمثيل**: Subword Tokenization تسمح بتمثيل الكلمات المُركَّبة (شائعة جدًا في العربية والألمانية) عبر إعادة استخدام أجزاء شائعة، فتُقلِّل حجم المفردات المطلوب مع الحفاظ على تغطية شاملة. (3) **التعميم المورفولوجي**: كلمات مُشتقّة من نفس الجذر (مثل "كتب"، "كاتب"، "مكتوب"، "كتابة") تُقسَّم غالبًا بشكل يُحافظ على الجذر المشترك كوحدة واحدة، مما يساعد النموذج على تعلّم العلاقات الصرفية بكفاءة أعلى من التعامل مع كل كلمة كوحدة منفصلة تمامًا.

س: ما الفرق الجوهري بين نهج BPE/WordPiece (Bottom-up) ونهج Unigram (Top-down)؟

ج: BPE وWordPiece يبدآن من أصغر وحدة (الأحرف المنفردة) ويدمجان تصاعديًا بناءً على معيار محلي (تكرار الزوج في BPE، أو احتمالية مُطبَّعة في WordPiece) في كل خطوة، وهذا قرار **جشع (greedy)** لا يراجع نفسه لاحقًا. أما Unigram فيبدأ من مفردات كبيرة جدًا (كل الكلمات الفرعية المحتملة تقريبًا) ثم يُقلِّصها تدريجيًا عبر حذف الرموز التي يُسبِّب حذفها أقل انخفاض في احتمالية (likelihood) المجموعة التدريبية الكاملة بموجب نموذج احتمالي Unigram صريح. هذا النهج "الشمولي" (holistic) يسمح لـ Unigram بإعادة تقييم جودة كل رمز في سياق المفردات الكاملة، وغالبًا ما يُنتج تقطيعًا أكثر مرونة (يمكنه توليد عدة تقطيعات محتملة لنفس الكلمة مع احتمالاتها، مفيد لتقنيات مثل Subword Regularization في التدريب).

10. تمرين عملي

درّب `BPETokenizerFromScratch` بحجم مفردات 300 على نص عربي حقيقي (فقرة من ويكيبيديا العربية، 500+ كلمة)، ثم قارن متوسط عدد الرموز لكل كلمة (Fertility) مع تطبيق نفس الفقرة عبر `tiktoken` (Tokenizer الخاص بـ GPT-4) وسجّل الفرق — سيُظهر لك هذا عمليًا كيف أن Tokenizers غير المُحسَّنة للعربية تُنتج تسلسلات أطول بكثير.

11. مراجع للتعمّق

- Sennrich, Haddow & Birch (2016) — Neural Machine Translation of Rare Words with Subword Units (BPE)
- Schuster & Nakajima (2012) — Japanese and Korean Voice Search (WordPiece)
- Kudo (2018) — Subword Regularization + Kudo & Richardson (2018) — SentencePiece

الجزء الثاني · الفصل 7

التمثيلات المتجهية للكلمات: Word2Vec وGloVe

الجزء: الثاني | **المستوى:** متوسط | **القراءة:** 15 دقيقة | **يسبقه:** الفصل 6

دفتر الكود `ch07-word-embeddings.ipynb`

> **ملخص الفصل**
>
> بعد Tokenization، لدينا تسلسل من الأعداد الصحيحة (Token IDs) — لكن هذه الأعداد عشوائية ولا معنى دلاليًا لها (الرمز 42 ليس "أقرب" للرمز 43 من الرمز 9000 بأي معنى رياضي مفيد).

مخرجات التعلم

بنهاية هذا الفصل ستكون قادرًا على:

- فهم GloVe — نهج إحصائي بديل
- فهم Static مقابل Contextual Embeddings — التوضيح الحاسم
- فهم Word2Vec

1. لماذا يهمّك هذا الموضوع؟

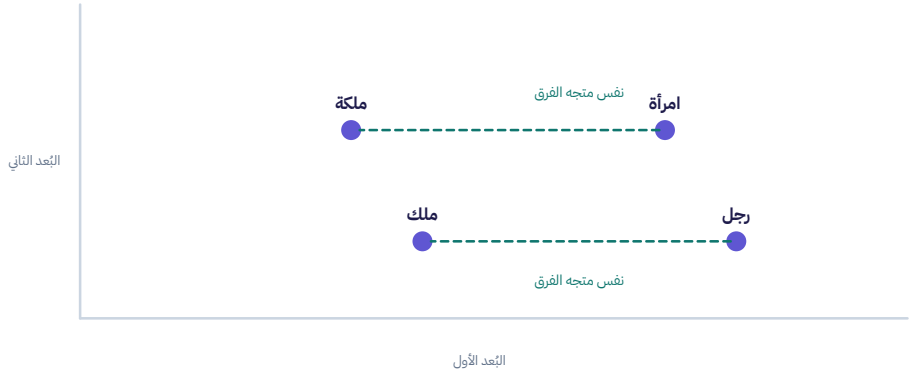
بعد Tokenization، لدينا تسلسل من الأعداد الصحيحة (Token IDs) — لكن هذه الأعداد **عشوائية ولا معنى دلاليًا لها** (الرمز 42 ليس "أقرب" للرمز 43 من الرمز 9000 بأي معنى رياضي مفيد). Word Embeddings تحلّ هذه المشكلة جذريًا: نُمثِّل كل رمز كمتجه كثيف

(Dense Vector) في فضاء مستمر، بحيث **الكلمات ذات المعنى المتشابه تكون متجهاتها قريبة من بعضها**. هذا هو حجر الأساس الذي بُنيت عليه كل الثورة الحديثة في معالجة اللغات الطبيعية.

2. الخريطة الذهنية



3. الشرح المتدرج



التضمينات ترتّب المعنى هندسيًا: الفرق بين «رجل» و«ملك» يشبه الفرق بين «امرأة» و«ملكة».

🟢 طبقة الحدس — الفرضية التوزيعية (Distributional Hypothesis)

المبدأ الأساسي وراء كل Word Embeddings: **"الكلمة تُعرَف من خلال رفقتها"** (You shall know a word by the company it keeps — J.R. Firth، 1957). كلمتان تظهران غالبًا في سياقات متشابهة (محاطتان بنفس الكلمات المجاورة) من المرجّح أن تحملا

معنى متشابهًا. Word2Vec يُحوِّل هذه الفرضية اللغوية إلى **مهمة تنبؤية قابلة للتدريب**: درّب شبكة عصبية بسيطة للتنبؤ بالكلمات المجاورة انطلاقًا من كلمة معينة (أو العكس)، والتمثيلات الوسيطة المُتعلَّمة أثناء هذا التدريب تصبح هي "الـ Embeddings" المفيدة.

🟡 طبقة الآلية — معماريتا Word2Vec

```
CBOW (Continuous Bag of Words)          Skip-gram
"تنبأ بالسياق من الكلمة"                  "تنبأ بالكلمة من السياق"

السياق: [القطة, تحب, ___, اللذيذ]          الكلمة: الحليب
                → تنبأ بـ [القطة, تحب, اللذيذ] →          تنبأ بـ "الحليب"

مناسب لـ: الكلمات النادرة              مناسب لـ: مجموعات بيانات كبيرة،
يُعطي تمثيلات أدق لكلمات نادرة                  كلمات شائعة، تدريب أسرع
```

🔴 طبقة التعمّق — Skip-gram مع Negative Sampling من الصفر

```python
import numpy as np
from collections import Counter
import random

class Word2VecSkipGram:
    """
    تطبيق Skip-gram مع Negative Sampling (Mikolov et al., 2013)
    هذا هو التطبيق الأصلي الذي أحدث ثورة في NumPy. من الصفر بـ
    تمثيل الكلمات قبل ظهور المحوّلات.
    """
    def __init__(self, vocab_size: int, embedding_dim: int = 100,
                 learning_rate: float = 0.025):
        self.vocab_size = vocab_size
        self.embedding_dim = embedding_dim
        self.lr = learning_rate

        # مصفوفتا تضمين منفصلتان: واحدة للكلمة "المركزية" وأخرى للسياق
        # (النهائية Embeddings ونستخدم عادة مصفوفة الكلمة المركزية فقط كـ)،هذا تفصيل تنفيذي مهم: نحتفظ بتمثيلين لكل كلمة أثناء التدريب
        self.W_center = np.random.uniform(-0.5, 0.5, (vocab_size,
embedding_dim)) / embedding_dim
        self.W_context = np.zeros((vocab_size, embedding_dim))

    @staticmethod
```

```
    def sigmoid(x):
        return 1 / (1 + np.exp(-np.clip(x, -500, 500)))

    def train_pair(self, center_idx: int, context_idx: int,
                   negative_indices: list):
        """
        خطوة تدريب واحدة: زوج (كلمة مركزية، كلمة سياق حقيقية)
        + (عدة عينات سلبية (كلمات عشوائية ليست في السياق الفعلي).

        الهدف: نُعظِّم P(context_idx|center_idx) = σ(v_center · v_context)
        لكل عينة سلبية — بدلًا من P(negative|center_idx) ونُصغِّر
        حساب softmax كامل عبر كل المفردات (مُكلِف جدًا حسابيًا
        لمفردات بحجم عشرات الآلاف أو أكثر).
        """
        v_center = self.W_center[center_idx]

        # ═══ العينة الإيجابية (السياق الحقيقي) ═══
        v_context_pos = self.W_context[context_idx]
        score_pos = self.sigmoid(np.dot(v_center, v_context_pos))
        # نريد score_pos قريبًا من 1 (تصنيف "إيجابي" صحيح)
        grad_pos = score_pos - 1

        grad_center_total = grad_pos * v_context_pos
        self.W_context[context_idx] -= self.lr * grad_pos * v_center

        # ═══ العينات السلبية (كلمات عشوائية، نريد score قريبًا من 0) ═══
        for neg_idx in negative_indices:
            v_context_neg = self.W_context[neg_idx]
            score_neg = self.sigmoid(np.dot(v_center, v_context_neg))
            grad_neg = score_neg - 0  # نريد قريبًا من 0

            grad_center_total += grad_neg * v_context_neg
            self.W_context[neg_idx] -= self.lr * grad_neg * v_center

        # تحديث متجه الكلمة المركزية بتجميع كل التدرجات
        self.W_center[center_idx] -= self.lr * grad_center_total

        # حساب الخسارة للمراقبة (Negative Log-Likelihood)
        loss = -np.log(score_pos + 1e-10)
        for neg_idx in negative_indices:
            score_neg = self.sigmoid(np.dot(v_center,
self.W_context[neg_idx]))
            loss -= np.log(1 - score_neg + 1e-10)
        return loss
```

```
class NegativeSampler:
    """
    اختيار عينات سلبية بناءً على توزيع تكرار الكلمات، لكن مع
    الأس 3/4 — حيلة مهمة جدًا من الورقة الأصلية (smoothing) "تنعيم"
    لمنع الكلمات شائعة جدًا (مثل أدوات الربط) من الهيمنة الكاملة
    على العينات السلبية، مع إعطاء الكلمات النادرة فرصة أكبر نسبيًا
    من توزيعها الخام.
    """
    def __init__(self, word_frequencies: dict, power: float = 0.75):
        words = list(word_frequencies.keys())
        freqs = np.array([word_frequencies[w] for w in words],
dtype=np.float64)

        # التنعيم: رفع كل تكرار للأس 0.75 ثم إعادة التطبيع
        smoothed_freqs = freqs ** power
        self.probabilities = smoothed_freqs / smoothed_freqs.sum()
        self.words = words

    def sample(self, n: int, exclude: set = None) -> list:
        samples = []
        while len(samples) < n:
            candidate = np.random.choice(len(self.words),
p=self.probabilities)
            if exclude is None or candidate not in exclude:
                samples.append(candidate)
        return samples

def demonstrate_smoothing_effect():
    """توضيح أثر الأس 3/4 عمليًا: تقليل هيمنة الكلمات الشائعة جدًا."""
    word_freqs = {"ال" :10000 ,"في" :5000 ,"كتاب" :50 ,"يراعة" :2}

    freqs = np.array(list(word_freqs.values()), dtype=np.float64)
    raw_probs = freqs / freqs.sum()
    smoothed_probs = (freqs ** 0.75) / (freqs ** 0.75).sum()

    for word, raw, smooth in zip(word_freqs.keys(), raw_probs,
smoothed_probs):
        print(f"{word}: احتمال خام={raw:.4f}, احتمال مُنعَّم={smooth:.4f}")

demonstrate_smoothing_effect()
# ال: احتمال خام=0.6645, احتمال مُنعَّم=0.4850  ← انخفضت هيمنته
# في: احتمال خام=0.3322, احتمال مُنعَّم=0.3630
```

```
# كتاب: احتمال خام=0.0033, احتمال مُنعَّم=0.0125 ← زادت فرصته نسبيًا
# يراعة: احتمال خام=0.0001, احتمال مُنعَّم=0.0019 ← زادت فرصته بشكل ملحوظ نسبيًا
```

نهج إحصائي بديل — GloVe

```
def explain_glove_concept():
    """
    GloVe (Global Vectors, Pennington et al., 2014): بدلًا من
    نافذة سياق صغيرة كما في (التدريب التنبؤي المحلي Word2Vec)،
    يبني GloVe أولًا "مصفوفة تعايش عامة" (co-occurrence matrix)
    X حيث X[i][j] = عدد مرات ظهور الكلمة j في سياق الكلمة i
    عبر **المجموعة النصية بأكملها**، ثم يُدرّب Embeddings بحيث:

        v_i · v_j + b_i + b_j ≈ log(X[i][j])

    الفكرة الجوهرية: نسب تعايش الكلمات (وليس القيم الخام فقط)
    تحمل معلومات دلالية قوية جدًا. مثال كلاسيكي من الورقة:

        P(ice|solid) / P(steam|solid)  >> 1    (الجليد صلب فعلًا)
        P(ice|gas) / P(steam|gas)      << 1    (البخار غاز فعلًا)
        P(ice|water) / P(steam|water)  ≈ 1     (كلاهما متعلق بالماء)

    GloVe يستغل هذه النسب مباشرة في دالة الخسارة، بعكس Word2Vec
    الذي يتعلّمها بشكل ضمني فقط عبر التنبؤ المحلي المتكرر.
    """
    pass

def weighted_least_squares_glove_loss(co_occurrence_matrix,
word_vectors,
                                      context_vectors, biases,
context_biases,
                                      x_max=100, alpha=0.75):
    """
    دالة خسارة GloVe الكاملة: Weighted Least Squares.
    الوزن f(X_ij) يمنع الأزواج النادرة جدًا (ضوضاء إحصائية) والمتكررة
    جدًا (مهيمنة بشكل غير مفيد) من التأثير المفرط على التدريب.
    """
    total_loss = 0.0
    rows, cols = co_occurrence_matrix.nonzero()

    for i, j in zip(rows, cols):
        x_ij = co_occurrence_matrix[i, j]
```

```
        # دالة الوزن f(x): تتصاعد حتى x_max ثم تستقر عند 1
        weight = (x_ij / x_max) ** alpha if x_ij < x_max else 1.0

        prediction = (np.dot(word_vectors[i], context_vectors[j]) +
                      biases[i] + context_biases[j])
        target = np.log(x_ij + 1e-10)

        total_loss += weight * (prediction - target) ** 2

    return total_loss
```

التوضيح الحاسم — Contextual Embeddings مقابل Static

```
def demonstrate_static_vs_contextual_limitation():
    """
    القيد الجوهري لـ Word2Vec/GloVe (Static Embeddings):
    لكل كلمة متجه واحد ثابت بغض النظر عن السياق، لذا لا يمكنها
    التمييز بين المعاني المتعددة لنفس الكلمة (Polysemy).
    """
    sentences_with_bank = [
        "ذهبت إلى البنك لسحب المال",          # بنك = مؤسسة مالية
        "جلسنا على ضفة النهر (بنك النهر)",      # بنك = ضفة (استخدام قديم/مجازي)
    ]

    print("مع Word2Vec/GloVe (Static):")
    print("  كلمة 'بنك' → نفس المتجه بالضبط في كلا الجملتين")
    print("  (النموذج لا يستطيع التمييز بين المعنيين إطلاقًا)")

    print("\nمع BERT/GPT (Contextual، لاحقًا الفصل 12):")
    print("  كلمة 'بنك' في الجملة 1 → متجه قريب من [مال، حساب، قرض]")
    print("  كلمة 'بنك' في الجملة 2 → متجه قريب من [نهر، ماء، ضفة]")
    print("  (نفس الرمز، تمثيلان مختلفان تمامًا حسب السياق المحيط)")

demonstrate_static_vs_contextual_limitation()
```

---

4. مثال محلول خطوة بخطوة

المسألة: احسب يدويًا خطوة تدريب Skip-gram واحدة لجملة عربية، موضحًا كيف يتغيّر المتجه المركزي.

```
def manual_skipgram_step():
    vocab = {"4 :"السيارة" ,3 :"اللذيذ" ,2 :"الحليب" ,1 :"تحب" ,0 :"القطة"}
    vocab_size, embed_dim = len(vocab), 4

    np.random.seed(0)
    model = Word2VecSkipGram(vocab_size, embed_dim, learning_rate=0.1)

    # الجملة: "القطة تحب الحليب اللذيذ"
    # مع نافذة سياق = 1: (تحب → القطة), (تحب → الحليب)
    center_word = "تحب"
    context_word = "الحليب"
    negative_word = "السيارة"  # كلمة غير ذات صلة، عينة سلبية

    center_idx = vocab[center_word]
    context_idx = vocab[context_word]
    negative_idx = vocab[negative_word]

    v_center_before = model.W_center[center_idx].copy()

    loss = model.train_pair(center_idx, context_idx, [negative_idx])

    v_center_after = model.W_center[center_idx]
    change = np.linalg.norm(v_center_after - v_center_before)

    print(f"الخسارة: {loss:.4f}")
    print(f"مقدار التغيير في متجه '{center_word}': {change:.6f}")
    print(f"المتجه قبل: {v_center_before.round(4)}")
    print(f"المتجه بعد: {v_center_after.round(4)}")

manual_skipgram_step()
```

5. نصائح وحيل

النصيحة	التفصيل
استخدم Negative Sampling بدلًا من softmax الكامل	يُقلِّل التعقيد الحسابي من O(V) إلى O(k) لكل خطوة تدريب، حيث k≈5-20
اختر نافذة سياق مناسبة	نافذة صغيرة (2-5) تلتقط علاقات نحوية، نافذة كبيرة (10+) تلتقط علاقات موضوعية/دلالية عامة

النصيحة	التفصيل
استخدم Subsampling للكلمات شائعة جدًا	حذف عشوائي لجزء من ظهورات الكلمات شائعة جدًا (مثل "ال", "في") يُحسِّن جودة Embeddings للكلمات الأقل شيوعًا
قِس جودة Embeddings بمهام تناظر (Analogy)	`ملك - رجل + امرأة ≈ ملكة` اختبار كلاسيكي وسريع لجودة الفضاء الدلالي المُتعلَّم

6. أخطاء شائعة ومصائد

- **استخدام Static Embeddings لمهام تتطلب فهم السياق الدقيق**: تحليل المشاعر أو الإجابة على الأسئلة تحتاج Contextual Embeddings (الفصل 12)، وStatic Embeddings وحدها غير كافية لهذه المهام المعقدة.
- **إهمال تنعيم الأس 3/4 في Negative Sampling**: يؤدي لهيمنة الكلمات شائعة الاستخدام على العينات السلبية، مما يُضعف جودة تمثيل الكلمات النادرة.
- **الخلط بين حجم الـ Embedding وجودته**: زيادة `embedding_dim` بلا حدود لا تُحسِّن الجودة دائمًا، وقد تُسبب overfitting مع بيانات محدودة.

7. جدول مقارنة

الخاصية	Word2Vec	GloVe
منهجية التدريب	تنبؤية محلية (نافذة سياق)	إحصائية عامة (مصفوفة تعايش كاملة)
كفاءة استغلال البيانات	يحتاج مرور متكرر على السياقات	يستغل إحصائيات التعايش الكاملة مباشرة
قابلية التوسع لمجموعات ضخمة	جيدة (تدريب تدريجي/stream)	يحتاج بناء مصفوفة تعايش كاملة مسبقًا
الأداء التجريبي	ممتاز، خصوصًا Skip-gram للكلمات النادرة	ممتاز أيضًا، أحيانًا أفضل في مهام تناظر عامة

8. ملخص أفضل الممارسات

- استخدم Skip-gram مع Negative Sampling كخط أساس قوي ومُثبَت لتعلّم Embeddings ثابتة.
- طبّق تنعيم الأس 3/4 دائمًا في اختيار العينات السلبية.
- تذكّر أن Static Embeddings (Word2Vec/GloVe) تبقى مفيدة اليوم كخط أساس سريع أو لتهيئة أولية (initialization)، لكن Contextual Embeddings (BERT/GPT) هي المعيار لكل تطبيق حديث جاد.
- قِس جودة الـ Embeddings دائمًا بمهام تناظر أو تشابه قبل استخدامها في تطبيق فعلي.

9. أسئلة مقابلات

س: لماذا نستخدم Negative Sampling بدلًا من softmax الكامل في تدريب Word2Vec؟

ج: softmax الكامل يتطلب حساب مجموع الأسس (normalization term) عبر **كل** كلمات المفردات (قد تكون مئات الآلاف)، في كل خطوة تدريب واحدة — تكلفة حسابية O(V) لكل عينة، وهذا غير عملي إطلاقًا لمجموعات بيانات ضخمة. Negative Sampling يُحوِّل المسألة إلى عدة مسائل تصنيف ثنائي بسيطة ومستقلة: "هل هذا الزوج (كلمة مركزية، كلمة سياق) حقيقي أم عشوائي؟"، فنحسب فقط k+1 دوال sigmoid (حيث k≈5-20 عينة سلبية) بدلًا من V دالة أس كاملة، مما يُقلِّل التعقيد الحسابي بشكل جذري مع الحفاظ على جودة تعلّم مقاربة جدًا في الممارسة العملية.

س: ما سبب استخدام الأس 3/4 (وليس 1) في توزيع أخذ العينات السلبية؟

ج: التوزيع الخام لتكرار الكلمات في أي لغة طبيعية شديد الانحراف (Zipfian): كلمات قليلة جدًا (أدوات ربط، ضمائر) تهيمن على معظم الظهورات. إذا اخترنا عينات سلبية بناءً على هذا التوزيع الخام مباشرة، ستكون العينات السلبية دائمًا تقريبًا نفس الكلمات الشائعة جدًا، مما يُضعف تعلّم تمييز دلالي دقيق للكلمات الأقل شيوعًا (لأن النموذج "يتعلّم فقط" تمييز الكلمة

المركزية عن كلمات شائعة جدًا، وليس عن منافسين دلاليين أقرب). رفع التكرارات للأس 0.75 "يُسطِّح" التوزيع جزئيًا: يُقلِّل الاحتمال النسبي للكلمات شائعة جدًا بينما يزيد الاحتمال النسبي للكلمات متوسطة ونادرة الشيوع، مما يُحسِّن جودة العينات السلبية وبالتالي جودة Embeddings النهائية — وهذه القيمة (0.75) وُجدت تجريبيًا في الورقة الأصلية كالأفضل أداءً.

10. تمرين عملي

نفّذ `Word2VecSkipGram` كاملًا، ودرّبه على مجموعة نصوص عربية (يمكن استخدام مجموعة فقرات من ويكيبيديا العربية، 10,000+ كلمة)، ثم اختبر جودة الـ Embeddings المُتعلَّمة عبر حساب التشابه الكوني (من الفصل 2) بين كلمات متوقع تشابهها دلاليًا (مثل "ملك" و"أمير") ومقارنتها بكلمات غير مرتبطة.

11. مراجع للتعمّق

- Mikolov et al. (2013) — Efficient Estimation of Word Representations in Vector Space (Word2Vec)
- Mikolov et al. (2013) — Distributed Representations of Words and Phrases and their Compositionality (Negative Sampling)
- Pennington, Socher & Manning (2014) — GloVe: Global Vectors for Word Representation
- Firth (1957) — A Synopsis of Linguistic Theory (الفرضية التوزيعية الأصلية)

الجزء الثاني · الفصل 8

الشبكات المتكررة: RNN وLSTM وGRU

الجزء: الثاني **المستوى:** متوسط **القراءة:** 20 دقيقة **يسبقه:** الفصل 7

دفتر الكود `ch08-recurrent-networks.ipynb`

ملخص الفصل

قبل المحوّلات (2017)، كانت الشبكات المتكررة (Recurrent Neural Networks) هي المعمارية المهيمنة الوحيدة لمعالجة التسلسلات اللغوية.

مخرجات التعلم

بنهاية هذا الفصل ستكون قادرًا على:

- فهم LSTM — الحل عبر البوابات (Gates)
- فهم GRU — نسخة مُبسَّطة من LSTM
- فهم الانتقال لـ PyTorch ومعمارية Bidirectional

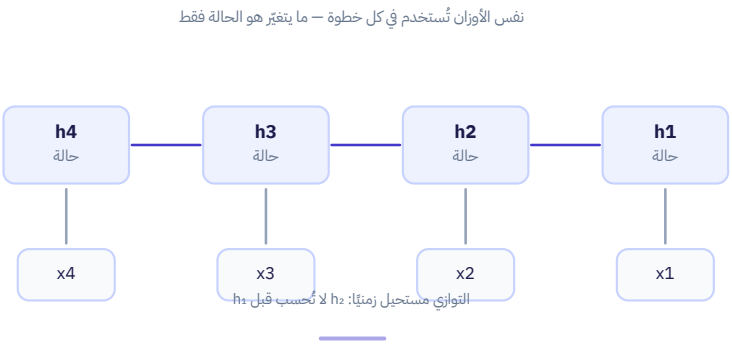
1. لماذا يهمّك هذا الموضوع؟

قبل المحوّلات (2017)، كانت الشبكات المتكررة (Recurrent Neural Networks) هي **المعمارية المهيمنة الوحيدة** لمعالجة التسلسلات اللغوية. فهمها بعمق — وتحديدًا **فهم أسباب فشلها** — هو ما يُفسِّر بدقة لماذا اختُرعت آلية Attention، ولماذا أحدثت المحوّلات ثورة حقيقية. بدون هذا الفهم التاريخي، ستتعامل مع Attention كـ"حيلة رياضية عشوائية" بدلًا من فهمها كـ**حل مباشر ومقصود** لمشاكل محددة وموثّقة جيدًا.

2. الخريطة الذهنية



3. الشرح المتدرج



تفكيك RNN عبر الزمن: نفس الخلية تتكرر، والحالة المخفية هي الذاكرة التي تنتقل.

🟢 طبقة الحدس — RNN البسيطة

تخيّل أنك تقرأ جملة كلمة كلمة، وتحتفظ في ذاكرتك بـ"ملخص" لكل ما قرأته حتى الآن (الحالة المخفية، Hidden State). عند كل كلمة جديدة، تُحدِّث هذا الملخص بدمج المعلومة الجديدة مع الملخص السابق. هذا بالضبط ما تفعله RNN: **نفس الأوزان** تُطبَّق بشكل متكرر عبر كل خطوة زمنية، مع تمرير حالة مخفية من خطوة لأخرى.

🟡 طبقة الآلية — معادلة RNN الأساسية

```
h_t = tanh(W_hh · h_{t-1} + W_xh · x_t + b_h)
y_t = W_hy · h_t + b_y

حيث:
h_t: الحالة المخفية عند الزمن t (الذاكرة المتراكمة)
x_t: مدخل الزمن t (مثلًا embedding الكلمة الحالية)
W_hh, W_xh, W_hy: مصفوفات أوزان مُشتركة (نفسها) عبر كل الخطوات الزمنية
```

🔴 طبقة التعمّق — RNN من الصفر ومشكلة تلاشي التدرجات

```python
import numpy as np

class SimpleRNNCell:
    """
    تطبيق خلية RNN الأساسية الآلية واحدة من الصفر، لتوضيح
    قبل الانتقال لتطبيق PyTorch الكامل.
    """
    def __init__(self, input_dim: int, hidden_dim: int):
        self.hidden_dim = hidden_dim
        scale = 1.0 / np.sqrt(hidden_dim)
        self.W_xh = np.random.randn(input_dim, hidden_dim) * scale
        self.W_hh = np.random.randn(hidden_dim, hidden_dim) * scale
        self.b_h = np.zeros(hidden_dim)

    def forward_step(self, x_t: np.ndarray, h_prev: np.ndarray) →
np.ndarray:
        z = x_t @ self.W_xh + h_prev @ self.W_hh + self.b_h
        h_t = np.tanh(z)
        return h_t

    def forward_sequence(self, X: np.ndarray) → list:
        """معالجة تسلسل كامل، خطوة زمنية تلو الأخرى (معالجة متسلسلة إلزامية!)."""
        seq_len = X.shape[0]
        h = np.zeros(self.hidden_dim)
        hidden_states = []
        for t in range(seq_len):
            h = self.forward_step(X[t], h)
            hidden_states.append(h.copy())
        return hidden_states

def demonstrate_vanishing_gradient_problem():
```

```
    """
    توضيح عملي لمشكلة تلاشي التدرجات في RNN: عبر قاعدة السلسلة،
    تدرج الخسارة بالنسبة لحالة مخفية بعيدة زمنيًا هو حاصل ضرب
    مضروبة (أقصاها 1، وعادة أقل بكثير) سلسلة طويلة من مشتقات tanh
    أقل من 1 W_hh تكراريًا. إذا كانت القيم الذاتية لـ W_hh في،
    هذا الحاصل يتلاشى أُسيًا مع طول التسلسل.
    """
    seq_lengths = [5, 20, 50, 100]
    # محاكاة مبسّطة: نفترض متوسط عامل انكماش لكل خطوة زمنية = 0.5
    # (في أغلب نطاقات التشغيل العملية tanh قيمة واقعية جدًا لمشتقة)
    decay_factor_per_step = 0.5

    print("تأثير طول التسلسل على مقدار التدرج المُتراكم (محاكاة):")
    for length in seq_lengths:
        gradient_magnitude = decay_factor_per_step ** length
        print(f"  طول التسلسل={length}: مقدار التدرج ≈
{gradient_magnitude:.2e}")

demonstrate_vanishing_gradient_problem()
# 3.13 ≈ مقدار التدرج :5=طول التسلسل-02e
# 9.54 ≈ مقدار التدرج :20=طول التسلسل-07e
# 8.88 ≈ مقدار التدرج :50=طول التسلسل-16e
# 7.89 ≈ مقدار التدرج :100=طول التسلسل-31e   ← تلاشى عمليًا تمامًا!
```

🔴 **هذا بالضبط سبب "النسيان" الذي تُعاني منه RNN البسيطة**: الكلمات الأولى في جملة أو فقرة طويلة لا تحصل عمليًا على أي تدرج تدريبي ذي معنى، فلا يتعلّم النموذج الاعتماديات طويلة المدى (Long-range dependencies) إطلاقًا.

الحل عبر البوابات (Gates) — LSTM

```
class LSTMCellFromScratch:
    """
    تطبيق كامل لخلية LSTM من الصفر (Hochreiter & Schmidhuber, 1997).
    الفكرة الثورية: فصل "حالة الخلية" (Cell State, C) عن
    "الحالة المخفية" (Hidden State, h). حالة الخلية تعمل كـ"حزام ناقل"
    (conveyor belt) يسمح للمعلومة بالتدفق عبر الزمن مع تدخل خطي
    بسيط فقط (جمع وضرب عنصري)، مما يحل مشكلة تلاشي التدرجات بشكل
    كبير جدًا مقارنة بـ RNN البسيطة.
    """
    def __init__(self, input_dim: int, hidden_dim: int):
        self.hidden_dim = hidden_dim
```

```
        concat_dim = input_dim + hidden_dim
        scale = 1.0 / np.sqrt(hidden_dim)

        # أوزانها الخاصة لها منها كل بوابات، أربع على تُطبَّق) [x_t, h_{t-1}] مُدمَجين)
        self.W_forget = np.random.randn(concat_dim, hidden_dim) * scale
        self.W_input = np.random.randn(concat_dim, hidden_dim) * scale
        self.W_candidate = np.random.randn(concat_dim, hidden_dim) *
scale
        self.W_output = np.random.randn(concat_dim, hidden_dim) * scale

        self.b_forget = np.ones(hidden_dim)  # تهيئة forget bias بـ1 (حيلة
مهمة!)
        self.b_input = np.zeros(hidden_dim)
        self.b_candidate = np.zeros(hidden_dim)
        self.b_output = np.zeros(hidden_dim)

    @staticmethod
    def sigmoid(x):
        return 1 / (1 + np.exp(-np.clip(x, -500, 500)))

    def forward_step(self, x_t: np.ndarray, h_prev: np.ndarray, c_prev:
np.ndarray):
        """
        أربع بوابات، كل منها بدور محدد بدقة:

        1. Forget Gate (f_t): "كم من الذاكرة القديمة (c_prev) نحتفظ بها؟"
        2. Input Gate (i_t): "كم من المعلومة الجديدة ندخلها للذاكرة؟"
        3. Candidate (c̃_t): "ما هي المعلومة الجديدة المُرشَّحة للإضافة؟"
        4. Output Gate (o_t): "كم من الذاكرة الحالية نُظهر كحالة مخفية؟"
        """
        combined = np.concatenate([x_t, h_prev])

        forget_gate = self.sigmoid(combined @ self.W_forget +
self.b_forget)
        input_gate = self.sigmoid(combined @ self.W_input +
self.b_input)
        candidate = np.tanh(combined @ self.W_candidate +
self.b_candidate)
        output_gate = self.sigmoid(combined @ self.W_output +
self.b_output)

        # "تحديث حالة الخلية: مزيج من "نسيان جزء من القديم" و"إضافة جزء من الجديد
        # (متكرر tanh جمع + ضرب عنصري، لا) **هذا التحديث **خطي بشكل أساسي
        # وهذا بالضبط ما يسمح للتدرج بالتدفق عبر مسافات زمنية طويلة دون تلاشٍ شديد
```

```
        c_t = forget_gate * c_prev + input_gate * candidate

        # الحالة المخفية: جزء "مُرشَّح للإظهار" من حالة الخلية الجديدة
        h_t = output_gate * np.tanh(c_t)

        return h_t, c_t, {
            "forget_gate": forget_gate, "input_gate": input_gate,
            "candidate": candidate, "output_gate": output_gate,
        }

    def forward_sequence(self, X: np.ndarray):
        seq_len = X.shape[0]
        h = np.zeros(self.hidden_dim)
        c = np.zeros(self.hidden_dim)
        hidden_states, cell_states = [], []

        for t in range(seq_len):
            h, c, gates = self.forward_step(X[t], h, c)
            hidden_states.append(h.copy())
            cell_states.append(c.copy())

        return hidden_states, cell_states
```

💡 **لماذا تُهيَّأ `b_forget` بالقيمة 1 (وليس صفر)؟** هذه حيلة هندسية مهمة جدًا ومُثبَتة تجريبيًا (Jozefowicz et al., 2015): في بداية التدريب، تُهيَّأ بوابة النسيان لتكون "منفتحة" (قريبة من 1، أي "احتفظ بمعظم الذاكرة القديمة") بدلًا من عشوائية تمامًا، مما يسمح للتدرجات بالتدفق بحرية أكبر عبر الزمن منذ الخطوات الأولى من التدريب ويُسرِّع التقارب بشكل ملحوظ.

LSTM نسخة مُبسَّطة من — GRU

```
class GRUCellFromScratch:
    """
    تطبيق GRU (Cho et al., 2014) من الصفر: دمج بوابتي Forget وInput
    وإزالة حالة الخلية المنفصلة تمامًا، في بوابة واحدة (Update Gate).
    نتيجة ذلك: معاملات أقل (الاعتماد فقط على الحالة المخفية h).
    وتدريب أسرع، مع أداء مقارب جدًا في أغلب المهام، بنسبة ~25% من LSTM.
    """
    def __init__(self, input_dim: int, hidden_dim: int):
        self.hidden_dim = hidden_dim
```

```
        concat_dim = input_dim + hidden_dim
        scale = 1.0 / np.sqrt(hidden_dim)

        self.W_update = np.random.randn(concat_dim, hidden_dim) * scale
        self.W_reset = np.random.randn(concat_dim, hidden_dim) * scale
        self.W_candidate = np.random.randn(concat_dim, hidden_dim) *
scale

        self.b_update = np.zeros(hidden_dim)
        self.b_reset = np.zeros(hidden_dim)
        self.b_candidate = np.zeros(hidden_dim)

    @staticmethod
    def sigmoid(x):
        return 1 / (1 + np.exp(-np.clip(x, -500, 500)))

    def forward_step(self, x_t: np.ndarray, h_prev: np.ndarray):
        combined = np.concatenate([x_t, h_prev])

        # Update Gate: يُحدّد "مزيج" الاحتفاظ بالقديم مقابل الجديد (يدمج دوري
forget+input من LSTM)
        update_gate = self.sigmoid(combined @ self.W_update +
self.b_update)

        # Reset Gate: يُحدّد "كم من الماضي نتجاهل" عند حساب المُرشَّح الجديد
        reset_gate = self.sigmoid(combined @ self.W_reset +
self.b_reset)

        combined_reset = np.concatenate([x_t, reset_gate * h_prev])
        candidate = np.tanh(combined_reset @ self.W_candidate +
self.b_candidate)

        # التحديث النهائي: استيفاء خطي بين الحالة القديمة والمُرشَّح الجديد
        h_t = (1 - update_gate) * h_prev + update_gate * candidate
        return h_t
```

الانتقال لـ PyTorch ومعمارية Bidirectional

```
import torch
import torch.nn as nn

class LSTMTextClassifier(nn.Module):
    """
```

```
    مُصنِّف نصوص كامل باستخدام LSTM ثنائي الاتجاه (Bidirectional):
    يُعالج التسلسل من البداية للنهاية ومن النهاية للبداية في آن واحد،
    مما يسمح لكل موضع بالوصول لمعلومات من كلا الاتجاهين — تحسين
    لكنه لا يزال أبطأ وأقل فعالية، Attention مهم جدًا قبل ظهور
    من Self-Attention الكامل (الفصل 10) في التقاط الاعتماديات
    البعيدة جدًا.
    """
    def __init__(self, vocab_size: int, embedding_dim: int = 128,
                 hidden_dim: int = 256, num_classes: int = 2,
                 num_layers: int = 2, dropout: float = 0.3,
                 pad_idx: int = 0):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim,
padding_idx=pad_idx)

        self.lstm = nn.LSTM(
            input_size=embedding_dim,
            hidden_size=hidden_dim,
            num_layers=num_layers,
            batch_first=True,
            bidirectional=True,      # معالجة في الاتجاهين
            dropout=dropout if num_layers > 1 else 0,
        )

        # ×2 لأن bidirectional يُركّب (concatenate) الاتجاهين معًا
        self.fc = nn.Linear(hidden_dim * 2, num_classes)
        self.dropout = nn.Dropout(dropout)

    def forward(self, input_ids: torch.Tensor, lengths: torch.Tensor =
None):
        embedded = self.embedding(input_ids)   # (batch, seq_len,
embedding_dim)

        if lengths is not None:
            # Packing: تحسين كفاءة حسابية مهم جدًا عند وجود padding
            # (يتجاهل الحسابات على مواضع الحشو، يُسرّع التدريب بشكل ملحوظ)
            packed = nn.utils.rnn.pack_padded_sequence(
                embedded, lengths.cpu(), batch_first=True,
enforce_sorted=False
            )
            packed_output, (h_n, c_n) = self.lstm(packed)
        else:
            _, (h_n, c_n) = self.lstm(embedded)
```

```
        # h_n شكلها: (num_layers*2, batch, hidden_dim)
        # نأخذ الحالة المخفية الأخيرة من كلا الاتجاهين للطبقة الأخيرة فقط
        forward_last = h_n[-2, :, :]   # آخر حالة للاتجاه الأمامي
        backward_last = h_n[-1, :, :]  # آخر حالة للاتجاه الخلفي
        combined = torch.cat([forward_last, backward_last], dim=1)

        combined = self.dropout(combined)
        logits = self.fc(combined)
        return logits
```

أبرز قيود RNN/LSTM التي حفّزت اختراع المحوّلات

```
def summarize_rnn_limitations():
    """
    ملخص القيود الجوهرية التي دفعت الباحثين لاختراع Attention
    والمحوّلات بالكامل (سنرى الحل الكامل في الفصل 10).
    """
    limitations = {
        "المعالجة المتسلسلة الإلزامية": (
            "أولًا، مما يمنع h_{t-1} دون معرفة h_t لا يمكن حساب "
            "التوازي الكامل عبر طول التسلسل أثناء التدريب — عنق "
            "زجاجة حاسوبي خطير جدًا لتدريب نماذج كبيرة على GPUs حديثة."
            "مُصمَّمة أساسًا للتوازي الضخم."
        ),
        "صعوبة الاعتماديات طويلة المدى": (
            "رغم تحسّن LSTM/GRU عن RNN الكبير البسيطة، لا تزال المعلومة "
            "تمر عبر 'عنق زجاجة' حالة مخفية ثابتة الحجم، ما يُصعّب "
            "الاحتفاظ بتفاصيل دقيقة من بداية تسلسل طويل جدًا (آلاف الكلمات)."
        ),
        "صعوبة بناء 'خريطة اعتماديات' مباشرة وصريحة": (
            "لا توجد آلية مباشرة لأي كلمة للوصول مباشرة (بخطوة واحدة) "
            "لأي كلمة أخرى بعيدة في التسلسل — كل المعلومة يجب أن تمر "
            "عبر سلسلة من التحديثات التسلسلية الوسيطة."
        ),
    }

    for problem, explanation in limitations.items():
        print(f"❌ {problem}:\n   {explanation}\n")

summarize_rnn_limitations()
```

4. مثال محلول خطوة بخطوة

المسألة: قارن عدد المعاملات القابلة للتدريب بين RNN بسيطة وLSTM وGRU لنفس الأبعاد (input_dim=100, hidden_dim=256)، لفهم تكلفة كل معمارية.

```
def compare_parameter_counts(input_dim=100, hidden_dim=256):
    # RNN بسيطة: W_xh + W_hh + b_h
    rnn_params = (input_dim * hidden_dim) + (hidden_dim * hidden_dim) +
hidden_dim

    # LSTM: 4 بوابات، كل منها بنفس حجم أوزان RNN تقريبًا
    concat_dim = input_dim + hidden_dim
    lstm_params = 4 * (concat_dim * hidden_dim + hidden_dim)

    # GRU: 3 "فقط" بوابات (update, reset, candidate)
    gru_params = 3 * (concat_dim * hidden_dim + hidden_dim)

    print(f"RNN البسيطة:  {rnn_params:,} معامل")
    print(f"LSTM:         {lstm_params:,} معامل
({lstm_params/rnn_params:.1f}× من RNN)")
    print(f"GRU:          {gru_params:,} معامل
({gru_params/rnn_params:.1f}× من RNN، "
          f"{gru_params/lstm_params:.1%} من LSTM)")

compare_parameter_counts()
# RNN البسيطة:  91,392 معامل
# LSTM:         365,568 معامل (من ×4.0 RNN)
# GRU:          274,176 معامل (من ×3.0 RNN، 75.0% من LSTM)
```

5. نصائح وحيل

النصيحة	التفصيل
استخدم `pack_padded_sequence` دائمًا مع تسلسلات متفاوتة الطول	يُسرِّع التدريب بشكل ملحوظ ويمنع تأثير الـ padding على التدرجات
ابدأ بـ GRU قبل LSTM إذا كانت الموارد محدودة	أداء مقارب جدًا بمعاملات أقل وتدريب أسرع

النصيحة	التفصيل
استخدم Bidirectional دائمًا للتصنيف (غير مناسب للتوليد)	يُحسِّن الأداء بشكل ملحوظ عندما يكون التسلسل الكامل متاحًا مسبقًا (ليس توليدًا تتابعيًا)
راقب تدرجات طبقات RNN المبكرة أثناء التدريب	قيم قريبة من الصفر باستمرار علامة على تلاشي التدرجات رغم استخدام LSTM/GRU

6. أخطاء شائعة ومصائد

- **استخدام RNN البسيطة لتسلسلات طويلة**: تفشل عمليًا بعد ~10-20 خطوة زمنية بسبب تلاشي التدرجات الشديد كما أثبتنا رياضيًا وعدديًا أعلاه.
- **نسيان Gradient Clipping مع RNN/LSTM**: هذه المعماريات عرضة بشكل خاص لمشكلة "انفجار التدرجات" (Exploding Gradients) المعاكسة لمشكلة التلاشي، خصوصًا مع تسلسلات طويلة.
- **مقارنة RNN بالمحوّلات دون فهم السياق التاريخي**: RNN لم تكن "خطأ"، بل كانت أفضل حل متاح في زمانها — فهم قيودها الدقيقة هو ما يُقدِّر جمال حل Attention لاحقًا.

7. جدول مقارنة

الخاصية	RNN البسيطة	LSTM	GRU
عدد البوابات	0	4 (forget, input, output, candidate)	3 (update, reset, candidate)
مشكلة تلاشي التدرجات	شديدة جدًا	محلولة جزئيًا بشكل جيد	محلولة جزئيًا بشكل جيد
عدد المعاملات	الأقل	الأعلى	متوسط (~75% من LSTM)
سرعة التدريب	الأسرع (نظريًا) لكن أداء ضعيف	الأبطأ	أسرع من LSTM بأداء مقارب

الخاصية	RNN البسيطة	LSTM	GRU
الاستخدام الحديث	نادر جدًا، تعليمي فقط	لا يزال مستخدَمًا في تطبيقات متخصصة	بديل خفيف لـ LSTM

8. ملخص أفضل الممارسات

- افهم RNN البسيطة ومشكلة تلاشي التدرجات رياضيًا بعمق — هذا يُؤسِّس فهمك لكل ما يليها.
- استخدم LSTM/GRU فقط لمهام تسلسلية بسيطة اليوم (المحوّلات هي المعيار لكل شيء آخر تقريبًا).
- طبّق `pack_padded_sequence` وGradient Clipping دائمًا عند استخدام RNN/LSTM/GRU فعليًا.
- احتفظ بفهم هذه المعماريات التاريخية جيدًا — بعض المعماريات الحديثة (Mamba، الفصل 16) تستعير مبادئ مشابهة للحالة المتكررة بكفاءة حديثة جديدة.

9. أسئلة مقابلات

س: كيف تحل LSTM مشكلة تلاشي التدرجات التي تُعاني منها RNN البسيطة؟

ج: المفتاح هو فصل "حالة الخلية" (Cell State، C_t) عن الحالة المخفية، وتحديثها عبر عملية **خطية بشكل أساسي**:
`C_t = forget_gate ⊙ C_{t-1} + input_gate ⊙ candidate`. هذا التحديث يتضمّن فقط جمعًا وضربًا عنصريًا (Hadamard product)، وليس ضربًا مصفوفيًا متكررًا عبر `tanh` كما في RNN البسيطة. عند اشتقاق التدرج عبر الزمن (Backpropagation Through Time)، هذا يعني أن مسار التدرج عبر حالة الخلية يمر بشكل أساسي عبر عمليات `forget_gate` (قيمة بين 0 و1، لكنها **مُتعلَّمة ومُتكيّفة** حسب المدخل الفعلي)، بدلًا من الضرب المتكرر بنفس مصفوفة الأوزان `W_hh` ومشتقة `tanh` الثابتة نسبيًا كما في RNN.

عندما تتعلّم بوابة النسيان أن تبقي قيمتها قريبة من 1 لمعلومة مهمة، يمر التدرج عبر هذا المسار بأقل انكماش ممكن، مما يسمح للشبكة بتعلّم الاحتفاظ بمعلومات لمسافات زمنية أطول بكثير مقارنة بـ RNN البسيطة.

س: لماذا تُعدّ المعالجة المتسلسلة الإلزامية في RNN عيبًا جوهريًا في عصر GPUs الحديثة، حتى لو كان أداء النموذج جيدًا؟

ج: GPUs الحديثة مُصمَّمة أساسًا للتوازي الضخم (آلاف النوى تعمل في آن واحد). في RNN، حساب `h_t` يعتمد بشكل إلزامي على `h_{t-1}` المحسوبة بالفعل، مما يعني أنه **لا يمكن حساب خطوتين زمنيتين متتاليتين في آن واحد** بغض النظر عن توفر موارد حوسبية متوازية هائلة — التسلسل بطبيعته الرياضية "متسلسل" (sequential dependency). هذا يعني أن وقت التدريب يتناسب خطيًا تقريبًا مع طول التسلسل بغض النظر عن حجم الـ GPU المستخدَم، بعكس المحوّلات (الفصل 10) حيث يمكن حساب Attention لكل المواضع في التسلسل **في آن واحد بشكل متوازٍ بالكامل** (نفس الضرب المصفوفي الكبير يُغطّي كل المواضع معًا)، مما يستغل قدرة GPUs الحديثة استغلالًا كاملًا ويُقلِّص زمن التدريب الفعلي بشكل جذري لمجموعات بيانات وتسلسلات ضخمة.

10. تمرين عملي

نفّذ `LSTMCellFromScratch` و`GRUCellFromScratch` كاملتين، وطبّقهما على نفس تسلسل بيانات صناعي طويل (100+ خطوة زمنية) حيث المعلومة المهمة موجودة في أول 5 خطوات فقط ويجب تذكّرها للتنبؤ بالمخرج النهائي. قارن قدرة كل من RNN البسيطة وLSTM وGRU على حل هذه المهمة (ستفشل RNN البسيطة بوضوح شديد).

11. مراجع للتعمّق

- Hochreiter & Schmidhuber (1997) — Long Short-Term Memory
- Cho et al. (2014) — Learning Phrase Representations using RNN Encoder-Decoder (GRU)

- Bengio, Simard & Frasconi (1994) — Learning Long-Term Dependencies with Gradient Descent is Difficult
- Jozefowicz, Zaremba & Sutskever (2015) — An Empirical Exploration of Recurrent Network Architectures

مشروع الجزء الثاني

تحليل مشاعر عربي بـ LSTM

الجزء: الثاني

دفتر الكود `project-02-arabic-sentiment-lstm.ipynb`

> مشروع ختامي يُدمج مفاهيم الجزء في خط إنتاج واحد قابل للتشغيل.

1. نظرة عامة

بناء محلل مشاعر عربي يستخدم Tokenizer مخصصًا وBiLSTM، مع تقييم شامل وتحليل للأخطاء.

> يُدمِج كل مفاهيم الجزء الثاني: BPE مخصص ← Embeddings مُتعلَّمة من الصفر ← LSTM ثنائي الاتجاه، في نظام واحد متكامل.

2. المخرجات

- Tokenizer BPE مدرَّب على corpus عربي.
- مجموعة بيانات تُحوِّل النص إلى تسلسلات معرّفات مع padding.
- نموذج BiLSTM مع Embeddings قابلة للتدريب.
- تقرير تقييم يضم F1 وتحليلًا نوعيًا.

3. المتطلبات

- إكمال الفصول 6–8.

- Python 3.10+ وPyTorch 2.x.
- corpus عربي غير مصنّف لتدريب Tokenizer، وبيانات مصنّفة للمشاعر.

4. خطة التنفيذ

1. تدريب Tokenizer وحفظ مفرداته.
2. بناء Dataset وDataLoader مع padding وlengths.
3. تدريب BiLSTM وضبط المعاملات الفائقة.
4. تقييم النموذج وتحليل الأخطاء.

5. التنفيذ

الكود مرجعي وتكاملي: يفترض توفر المكوّنات المساعدة التي بنيتها في فصول هذا الجزء. أضف الاستيرادات الناقصة أو انسخ التعريفات من ملفات الفصول قبل التشغيل، ثم اقرأ التعليقات العربية داخل الكود لفهم كل مرحلة.

```
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader


class ArabicSentimentDataset(Dataset):
    """
    مجموعة بيانات تُدمِج Tokenizer المخصص (الفصل 6) لتحويل
    النصوص الخام إلى تسلسلات معرّفات جاهزة للنموذج.
    """
    def __init__(self, texts: list, labels: list, tokenizer:
BPETokenizerFromScratch,
                 max_length: int = 32):
        self.texts = texts
        self.labels = labels
```

```
        self.tokenizer = tokenizer
        self.max_length = max_length

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, idx):
        token_ids = self.tokenizer.encode(self.texts[idx])

        # تقصير أو حشو (padding) للوصول لطول موحّد
        if len(token_ids) > self.max_length:
            token_ids = token_ids[:self.max_length]
        length = len(token_ids)
        token_ids = token_ids + [0] * (self.max_length - length)

        return {
            "input_ids": torch.tensor(token_ids, dtype=torch.long),
            "length": torch.tensor(length, dtype=torch.long),
            "label": torch.tensor(self.labels[idx], dtype=torch.long),
        }


def full_part2_pipeline():
    """
    خط الأنابيب الكامل: من النص الخام إلى نموذج مُدرَّب ومُقيَّم،
    يُدمج كل فصول الجزء الثاني (ضمنيًا عبر nn.Embedding، 6، 7، 8).
    """
    # ═══ بيانات توضيحية (في الواقع: مجموعة بيانات حقيقية أكبر) ═══
    positive_texts = ["الخدمة ممتازة والفريق محترف جدا",
                      "تجربة رائعة انصح بها بشدة للجميع",
                      "المنتج عالي الجودة ويستحق السعر"] * 40
    negative_texts = ["الخدمة سيئة جدا ولن اتعامل معهم مرة اخرى",
                      "تجربة محبطة ومضيعة للوقت والمال",
                      "المنتج رديء الجودة ولا يستحق الشراء"] * 40

    all_texts = positive_texts + negative_texts
    all_labels = [1] * len(positive_texts) + [0] * len(negative_texts)

    # ═══ الخطوة 1: تدريب Tokenizer مخصص (الفصل 6) ═══
    print("الخطوة 1: تدريب BPE Tokenizer...")
    tokenizer = BPETokenizerFromScratch(vocab_size=200)
    tokenizer.train(all_texts, verbose=False)
    print(f"✅ حجم المفردات النهائي: {len(tokenizer.vocab)}")
```

```
    # ═══ الخطوة 2: بناء مجموعة البيانات وتقسيمها ═══
    indices = torch.randperm(len(all_texts))
    split = int(0.8 * len(all_texts))
    train_idx, test_idx = indices[:split].tolist(),
indices[split:].tolist()

    train_dataset = ArabicSentimentDataset(
        [all_texts[i] for i in train_idx], [all_labels[i] for i in
train_idx], tokenizer
    )
    test_dataset = ArabicSentimentDataset(
        [all_texts[i] for i in test_idx], [all_labels[i] for i in
test_idx], tokenizer
    )

    train_loader = DataLoader(train_dataset, batch_size=16,
shuffle=True)
    test_loader = DataLoader(test_dataset, batch_size=16, shuffle=False)

    # ═══ الخطوة 3: بناء النموذج (Embeddings مُتعلَّمة من الصفر + LSTM، الفصلان 7-8)
═══
    print("\nالخطوة 2: بناء وتدريب LSTMTextClassifier...")
    model = LSTMTextClassifier(
        vocab_size=len(tokenizer.vocab),
        embedding_dim=64,
        hidden_dim=128,
        num_classes=2,
        num_layers=2,
        dropout=0.3,
    )

    # ═══ الخطوة 4: التدريب (يُدمج مفاهيم الفصل 4 من الجزء الأول: AdamW +
Clipping) ═══
    optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3,
weight_decay=0.01)
    criterion = nn.CrossEntropyLoss()

    for epoch in range(15):
        model.train()
        total_loss = 0.0
        for batch in train_loader:
            optimizer.zero_grad()
            logits = model(batch["input_ids"], batch["length"])
            loss = criterion(logits, batch["label"])
```

```
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(),
max_norm=1.0)
            optimizer.step()
            total_loss += loss.item()

        if epoch % 5 == 0:
            print(f"Epoch {epoch}: avg_loss=
{total_loss/len(train_loader):.4f}")

    # ═══ خطوة 5: التقييم الشامل (يُدمج مفاهيم الفصل 5 من الجزء الأول) ═══
    model.eval()
    all_preds, all_true = [], []
    with torch.no_grad():
        for batch in test_loader:
            logits = model(batch["input_ids"], batch["length"])
            preds = torch.argmax(logits, dim=1)
            all_preds.extend(preds.tolist())
            all_true.extend(batch["label"].tolist())

    import numpy as np
    all_preds, all_true = np.array(all_preds), np.array(all_true)
    print("\n" + "=" * 50)
    print("التقييم النهائي")
    print("=" * 50)
    print(f"Accuracy:  {accuracy_score(all_true, all_preds):.4f}")
    print(f"Precision: {precision_score(all_true, all_preds):.4f}")
    print(f"Recall:    {recall_score(all_true, all_preds):.4f}")
    print(f"F1-Score:  {f1_score(all_true, all_preds):.4f}")

    return model, tokenizer


if __name__ == "__main__":
    model, tokenizer = full_part2_pipeline()
```

6. التشغيل

1. احفظ الكود في `project_02.py`.

2. ثبّت `torch` و`numpy` و`scikit-learn`.

3. وفّر تعريف `BPETokenizerFromScratch` و`LSTMTextClassifier` من الفصلين 6 و8، ثم شغّل `python project_02.py`.

النتيجة المتوقعة: يُطبع حجم المفردات، ثم خسارة التدريب كل خمسة عصور، ثم مقاييس التقييم النهائية.

7. أفضل الممارسات

- درّب Tokenizer على corpus واسع قبل تقسيم البيانات، ثم احفظ المفردات وأعد استخدامها.
- استخدم padding ديناميكيًا أو حزم التسلسلات لتقليل الهدر الحسابي.
- طبّق gradient clipping وراقب norm التدرجات كل خطوة.
- قارن BiLSTM بخط أساس بسيط قبل الانتقال إلى المحوّلات.

8. تمرين عملي

أضف طبقة Attention فوق مخرجات BiLSTM، ثم قارن F1 وزمن التدريب مع النسخة الأساسية.

9. التقييم

- F1 مرجّح على مجموعة الاختبار.
- استقرار الخسارة عبر epochs.
- قابلية إعادة إنتاج النتائج.

10. التوسّعات

- استبدل BiLSTM بـ GRU أو أضف طبقة Attention.

- قارن مع AraBERT أو نموذج محوّل عربي جاهز.
- أضف تصنيفًا متعدد الفئات (إيجابي/سلبي/محايد).